

HACKATHON AGENTIC DEVELOPMENT FRAMEWORK (HADF)

Zero-to-Framework Team Learning + Ultimate 24-Hour Operating Workflow

A complete teammate handbook for understanding what an agentic workflow is, why the generic repository exists, how the baseline stack fits together, what must be learned before using the repository, and how a 3- or 4-member team executes the full 24-hour workflow.

CORE RULE: Parallelize implementation, not architecture. One product truth, one shared design, many bounded human-plus-agent implementation loops.

READING ORDER

PART A - Framework foundations and prerequisites (this prelude)

PART B - Zero-to-Framework Readiness and Team Implementation Handbook

PART C - Ultimate 24-Hour Multi-Member Agentic Development Workflow

0. What "This Framework" Means

Throughout this combined handbook, the phrase "this framework" means the Hackathon Agentic Development Framework (HADF). HADF is the team operating model that combines human decision-making, AI-assisted implementation, shared repository artifacts, contract-based parallel development, risk-based QA, incremental integration, and Golden-Path-driven scope control.

HADF is not a software library and it is not a specific AI product. It is a process framework. The repository is the implementation workspace that carries the framework. The tech stack is the set of technologies used to build the actual challenge solution. The agents are tools that perform bounded reasoning, implementation, or review inside that process.

FRAMEWORK = how the team works

REPOSITORY = where project truth + code live

TECH STACK = technologies used to build the product

AGENTS = bounded helpers inside the workflow

HUMANS = accountable decision makers

If a later chapter says "the framework prevents contract drift" or "the framework requires one plan.md", it is referring to HADF, not to FastAPI, Git, Codex, ChatGPT, or any single document.

1. What an Agentic Workflow Actually Is

An agentic workflow is a structured way of assigning goal-oriented work to AI agents while humans define authority, scope, checkpoints, evidence, and stop conditions. The important word is not AI; it is workflow. A chatbot can answer a question. A coding assistant can suggest code. An agentic workflow organizes multiple reasoning and implementation steps so that work progresses toward an engineering objective without giving uncontrolled authority to an agent.

Mode	What happens	Main risk
Normal chat	You ask a question; AI returns an answer.	No persistent project control.
Coding assistant	AI edits or suggests code around the current task.	Can optimize locally without whole-system awareness.
Uncontrolled autonomous agent	Agent decides what to build and how.	Architecture drift, scope drift, unverifiable changes.
HADF agentic workflow	Humans define shared truth; Builder plans and implements bounded work; Reviewer independently checks risk; evidence and Git integrate the result.	Requires discipline, but keeps speed and control together.

The central model is human-governed agency. Humans decide what the product means, what tradeoffs are acceptable, what the Golden Path is, which contracts are authoritative, when material changes are allowed, and whether the result is ready to merge or release. Agents accelerate bounded work beneath those decisions.

Human / Project Owner

- > Shared Control / Supervisor
 - > problem.md + Golden Path + plan.md
 - > Workstream assignment
 - > Builder Planning Phase
 - > Human approval
 - > Builder implementation + tests
 - > Evidence
 - > Reviewer when risk justifies it
 - > Incremental merge to main

2. Why Build a Generic Agentic Hackathon Repository?

A 24-hour hackathon should not spend its first hours repeatedly rebuilding the same engineering scaffolding and rediscovering the same coordination rules. The generic repository exists to preserve challenge-neutral infrastructure and operating knowledge before the challenge is known. On event day, the team fills that structure with challenge-specific requirements, architecture, code, tests, and evidence.

The repository solves two different problems at once: technical bootstrapping and coordination bootstrapping. Technical bootstrapping gives the team a known backend structure, configuration pattern, migration/test setup, documentation locations, and startup conventions. Coordination bootstrapping gives humans and agents a common source of truth for requirements, architecture, execution state, review findings, and operating rules.

Hackathon failure	Repository/framework response
Everyone interprets the challenge differently	One approved problem.md.
Team builds too much	MVP + Golden Path.
Four agents invent four architectures	One plan.md + material-change gates.
Frontend and backend disagree	Design-stable API contracts.
People overwrite or duplicate work	Workstream ownership + short-lived branches.
Database changes break teammates	Migration ownership + Alembic discipline.
Testing starts too late	Shift-left QA + Definition of Done.
Everything works separately but not together	Vertical-slice rendezvous + incremental integration.
End-of-event merge catastrophe	main stays the best-known integrated state.
Nobody can explain the AI-generated project	Post-workstream reconstruction + final whole-project reconstruction.

The generic repository must remain challenge-neutral. It may include reusable infrastructure, but challenge-specific domain models, core algorithms, business rules, or proprietary solution logic should be created after the official problem is known, subject to event rules.

3. The Five Layers of HADF

Layer	Purpose	Examples
1. Product truth	Defines what must be built and what matters most.	problem.md, MVP, Golden Path
2. System design	Defines how the complete	plan.md, architecture, entities,

	product will work.	contracts, invariants
3. Execution control	Defines what is being built now and by whom.	workstreams, dependencies, execute.md, branches
4. Verification	Defines how we know work is correct.	tests, QA matrix, evidence, Reviewer, regression, E2E
5. Release/explanation	Turns code into a demoable, explainable submission.	README, Feature Freeze, release decision, final reconstruction, Demo Freeze

A teammate does not need to memorize every artifact. They need to understand the chain: product truth constrains design; design constrains workstreams; workstreams produce code; verification produces confidence; integration produces one system; reconstruction produces human understanding.

4. Human Roles and Agent Roles Are Different

HADF deliberately separates human accountability from agent capability. A human may use several agents, and one person may fill several human roles in a 3-member or solo mode. The roles describe responsibilities, not specific products.

Role	Human or agent?	Core responsibility
Human / Project Owner	Human	Final accountability for scope, approvals, tradeoffs, and explanation.
Technical Lead / Integrator	Human	Maintains whole-system coherence, contracts, dependencies, merge/release decisions.
Backend / Data Owner	Human	Owens server, persistence, business correctness and backend verification.
Frontend / Client Owner	Human	Owens UI, client state, API consumption and frontend integration.
QA / Reliability Owner	Human	Owens risk-based independent verification, regression, E2E and evidence.
Control / Supervisor	Agent or human+agent role	Reasoning, architecture supervision, plan critique, scope/debug/reconstruction support.
Builder	Agent	Repo-grounded planning, implementation, tests, debugging and bounded evidence.
Reviewer	Agent or independent human	Independent verification and risk findings; does not silently become the Builder.

The separation prevents a common failure: asking one AI session to interpret the challenge, invent the architecture, implement everything, review itself, and declare success. HADF uses different checkpoints

and, when useful, different agent contexts so that implementation and verification are not the same uncontrolled activity.

5. The Repository Files Before Anyone Starts Coding

The repository is not just a folder of code. Its Markdown artifacts create shared memory for humans and agents. They are intentionally small enough to update under hackathon time pressure, but specific enough to stop parallel work from drifting.

Artifact	Question answered	Owner / authority
AGENTS.md	How are humans/agents allowed to operate in this repo?	Repo operating policy.
problem.md	WHAT must the product do?	Lead reconciles; team approves.
plan.md	HOW will the complete product be designed?	Lead reconciles; team reviews material decisions.
execute.md	What is happening now? Who owns it? What is blocked?	Workstream owners + lead.
review.md	What verified findings exist?	QA/Reviewer.
README.md	What is the actual project and how is it run/demoed?	Designated documentation owner; reconciled with reality.
Code/tests/migrations/runtime evidence	What is actually implemented?	Implemented truth.

Authority matters when documents disagree. Official rules and challenge statements come first. Then the approved problem.md and plan.md. Actual code, migrations, tests, and runtime evidence reveal implementation truth. execute.md and review.md track status and findings. The team should resolve drift rather than pretending every artifact is simultaneously correct.

6. Why This Baseline Technology Stack Exists

HADF is tool-agnostic, but a team benefits from a default stack so it does not spend the event debating technologies. The recommended baseline is optimized for fast implementation, explicit contracts, relational data correctness, testability, and good support from coding agents. It is a default, not a law.

Layer	Default	Why it fits a 24-hour hackathon	Replace when...
Frontend	HTML/CSS/JavaScript or a lightweight framework	Fast UI delivery, broad familiarity, direct browser debugging.	Challenge needs a platform-specific mobile/3D/native capability.
API backend	FastAPI	Fast Python API development, type-driven validation, OpenAPI/Swagger, async support.	Team has stronger expertise or challenge requires another ecosystem.
Validation	Pydantic	Executable request/response schemas and clear 422-	Chosen backend has equivalent native schema tooling.

		level validation.	
Application logic	Service/use-case layer	Separates HTTP from business rules and transaction orchestration.	Tiny prototype truly has no meaningful business logic - but keep boundaries clear.
Persistence mapping	SQLAlchemy	Mature ORM/SQL toolkit, works naturally with FastAPI/Python.	Data model is non-relational or another language stack is chosen.
Database	PostgreSQL	Relational integrity, constraints, transactions, concurrency, familiar tooling.	Challenge is naturally document/graph/time-series and tradeoff is justified.
Migrations	Alembic	Ordered schema evolution across teammates and environments.	Equivalent migration system exists in another stack.
Tests	pytest + API/integration/E2E tools	Low ceremony, repeatable evidence, easy automation.	Equivalent ecosystem-native test tooling is used.
Version control	Git + GitHub	Parallel branches, history, review, rollback, shared main.	Event mandates another VCS/hosting system.

User

- > Frontend
- > HTTP/JSON
- > FastAPI router
- > Pydantic validation
- > service/use case
- > business logic
- > SQLAlchemy/session
- > PostgreSQL
- > commit
- > response schema/JSON
- > frontend state
- > visible result

The stack is coherent because each layer has one job and one boundary. The frontend owns interaction, HTTP transports the request, FastAPI receives it, Pydantic validates structure, the service owns the use case, SQLAlchemy/session persists state, PostgreSQL enforces durable truth, and tests/QA verify behavior across those boundaries.

7. Minimum Prerequisites Before Touching the Agentic Repository

A teammate does not need to be an expert in every layer. They do need enough shared literacy to understand what their Builder is changing, recognize when a problem crosses a boundary, and communicate with the rest of the team.

Readiness level	Must understand	Can you demonstrate it?
Level 0 - Local computing	files/folders, terminal, processes, localhost, ports, environment variables, JSON	Start frontend/backend processes and explain which port each uses.
Level 1 - Full-stack basics	frontend vs backend vs DB, request/response, state vs persistence, CRUD	Trace one user action from click to DB and back.
Level 2 - HTTP/API	GET/POST/PATCH/DELETE, path/query/body, status codes, API contract	Write and explain one request/response contract.
Level 3 - Data	PK/FK, relations, constraints, transactions, migrations	Explain why a durable state change needs a transaction/migration.
Level 4 - Git	clone, branch, add/commit, pull/fetch, merge/conflicts, main	Create a feature branch and merge a safe change.
Level 5 - QA	happy/boundary/negative, unit/integration/E2E, regression, evidence	Create a small test matrix and reproduce a failure.
Level 6 - Framework vocabulary	MVP, Golden Path, workstream, vertical slice, dependency, freeze	Explain how one workstream contributes to the Golden Path.
Level 7 - Agent governance	Supervisor vs Builder vs Reviewer, Plan Mode, stop/escalation rules	Review a Builder plan and reject an unauthorized contract change.

8. What Everyone Must Know vs What Specialists Must Know Deeply

Shared literacy avoids silos; specialist depth avoids wasting time. Every teammate should understand the complete Golden Path and the boundaries between frontend, backend, data, Git, and QA. They do not all need identical depth.

Topic	Everyone	Lead	Backend	Frontend	QA
Problem/MVP/Golden Path	Working understanding	Deep	Working	Working	Working
Architecture/contracts	Working	Deep	Deep server side	Deep client side	Working + risk view
Database/transactions	Basic	Working	Deep	Basic	Working for validation
Frontend	Basic	Working	Basic	Deep	Working for

state/network					E2E
Git/branches/merge	Working	Deep coordination	Working	Working	Working
Testing/QA	Working	Working	Focused backend	Focused frontend	Deep
Agent workflow	Working	Deep governance	Working	Working	Deep reviewer/test use

9. Why MVP, Golden Path, Workstreams, and Vertical Slices Are Separate Concepts

These terms solve different planning problems. Treating them as synonyms makes the team either under-plan or over-plan.

Concept	Meaning	Why it exists
MVP	Smallest useful product scope.	Stops scope explosion.
Golden Path	Most important successful end-user journey proving MVP value.	Gives the team one integration/demo priority.
Workstream	Bounded engineering objective/capability.	Creates assignable ownership and dependency control.
Vertical slice	One capability implemented through only the layers it needs.	Creates early usable integration instead of layer silos.
Golden-Path E2E	Real proof the assembled critical journey works.	Demonstrates system-level value, not isolated component success.

problem.md -> defines required WHAT

MVP -> chooses smallest useful scope

Golden Path -> chooses most important successful journey

plan.md -> designs HOW

workstreams -> divide engineering objectives

vertical slices -> implement capabilities across needed layers

E2E -> prove the assembled journey works

10. One Framework, Three Operating Modes

The same repository can support solo, 3-member, and 4-member operation. Team mode adds coordination; it does not change the underlying engineering methodology.

Mode	Human mapping	What disappears or merges
SOLO	One person fills lead, backend, frontend, QA over time.	No teammate handoffs or multi-human ownership matrix; Reviewer remains useful for independence.
TEAM_3	A: lead/integration/backend; B: backend/data/core logic; C: frontend/QA/E2E.	Dedicated QA role merges; scope and parallelism are reduced.
TEAM_4	A: lead/integrator; B:	Highest role separation and

	backend/data; C: frontend/client; D: QA/reliability.	parallelism; requires strongest coordination discipline.
--	---	---

In every mode there is still one problem.md, one plan.md, one Golden Path, and one integrated main branch.

11. Mandatory Pre-Hackathon Practice Lab

Before the real event, the team should run one small challenge through the complete process. The objective is not to build an impressive app. It is to rehearse the coordination machinery until it feels normal.

1. Read a one-page fictional problem statement together and normalize it into problem.md.
2. Define a narrow MVP and one Golden Path.
3. Design one entity relationship and one design-stable API contract in plan.md.
4. Create two dependency-aware workstreams in execute.md.
5. Have the backend owner give a bounded Builder prompt, review Plan Mode, approve, implement and test.
6. Have the frontend owner build against the contract using a temporary mock if needed.
7. Perform the real frontend/backend rendezvous and remove the mock.
8. Have QA derive happy, boundary and negative cases before integration, then verify the real slice.
9. Introduce one deliberate contract mismatch and practice the NEED CONTRACT DECISION protocol.
10. Merge incrementally into main, run a Golden-Path E2E, refresh to prove persistence, and perform final reconstruction.

Readiness criterion: every member can explain why each step happened and what would have gone wrong if it were skipped.

12. How to Read the Remaining Handbook

The rest of the combined PDF is intentionally ordered in two layers. First comes the Zero-to-Framework Readiness handbook. It teaches the underlying engineering concepts and connects the stack to the framework. After that comes the Ultimate 24-Hour Workflow. That section is the full event-day operating system with role playbooks, prompts, synchronization, handoffs, QA, Git, release, and worked examples.

Reader	Recommended path
New teammate	Read this prelude -> complete all readiness/fundamentals chapters -> rehearse the practice lab -> read Ultimate Workflow.
Backend specialist	Skim general basics but fully read contracts, persistence, transactions, Git, QA, agent governance, and Ultimate Workflow.
Frontend specialist	Fully read HTTP/contracts, state vs persistence, API client, Git, QA, agent governance, and Ultimate Workflow.
QA specialist	Fully read system flow, contracts, DB state, Git, workstreams, Reviewer model, and Ultimate Workflow.

Technical Lead	Read everything; you are responsible for preserving the whole-system model.
----------------	---

At this point, a teammate should know exactly what HADF is, why the generic repository exists, what an agentic workflow means, how the baseline stack connects, which prerequisites are required, and how human authority differs from agent capability. The following two parts now teach the system in depth and then provide the complete 24-hour operating workflow.

PART B - ZERO-TO-FRAMEWORK READINESS HANDBOOK

The next document teaches the engineering foundations, stack, repository concepts, role model, and example implementation in greater depth. Throughout that document, every occurrence of "this framework" should be read as "the Hackathon Agentic Development Framework (HADF)" defined in Part A.

After Part B, continue directly to Part C - the Ultimate 24-Hour Multi-Member Agentic Development Workflow.

24-HOUR HACKATHON: ZERO-TO-FRAMEWORK READINESS & TEAM IMPLEMENTATION HANDBOOK

Why the framework exists, prerequisite engineering knowledge, why the baseline stack fits, and how a 4-member or 3-member agentic team builds one complete project

Primary audience: teammates who can program at a basic level but have not yet used this full-stack, QA-aware, multi-agent hackathon workflow.

Purpose: make every teammate understand the system before asking agents to build it. This is both a learning manual and an event-day operating reference.

CORE PRINCIPLE: Parallelize implementation, not architecture. One product truth, one shared design, many bounded implementation loops.

How to Use This Handbook

Read Parts I-VII before the event. They build the mental model and prerequisite knowledge required to use the repository correctly. Parts VIII onward are the operating manual: repository artifacts, team workflow, agent prompts, Git/QA runbooks, worked implementation, recovery playbooks, and final submission process.

Do not try to memorize every command. The goal is to understand why each layer exists, how data flows across the stack, where responsibility changes hands, what evidence proves correctness, and when an agent must stop and ask a human.

Stage	What teammates should be able to do after it
Foundations	Explain frontend, backend, database, HTTP, state, persistence, Git, tests, and localhost/ports.
Stack understanding	Explain why each baseline technology exists and when it may be replaced.
Tiny vertical slice	Trace one real user action from browser to database and back.
Framework understanding	Explain problem.md, Golden Path, plan.md, execute.md, workstreams, contracts, QA, Builder, Reviewer.
Team execution	Run a bounded workstream, integrate it, verify it, merge it, and explain it.
Event readiness	Operate the full 24-hour process without each member inventing a separate architecture.

PART I - Why This Framework Exists

1. The problem is not usually “we cannot write code”

In a 24-hour hackathon, teams often have enough raw coding ability to create pages, endpoints, and database tables. Projects still fail because the pieces do not converge. Requirements are interpreted differently, frontend and backend invent different data shapes, agents make uncoordinated architectural changes, branches remain separated for too long, testing starts late, or the final demo depends on paths nobody exercised end to end.

This framework is therefore a coordination and convergence system. It is designed to convert an unknown problem statement into one shared understanding, one bounded MVP, one Golden Path, one Master System Design, multiple coordinated workstreams, and finally one integrated product that can be demonstrated and defended.

2. Failure mode -> framework response

Typical failure	Why it happens	Framework response
Team builds the wrong product	People start coding their first interpretation	Challenge reconstruction + one problem.md
Scope explodes	Every idea is treated as mandatory	MVP + Golden Path + priority labels

Four architectures appear	Each person/agent designs locally	One shared plan.md + human-gated material changes
Frontend/backend mismatch	No agreed request/response model	Design-stable API contracts
People overwrite each other	Unclear ownership and shared files	Workstream ownership + branch protocol
AI agent changes too much	Prompt says “build the project”	Bounded Builder scopes + stop/escalate rules
Everything works separately	Integration delayed	Vertical-slice rendezvous + incremental merge
Critical bug found at the end	QA starts after development	Shift-left QA + risk-based Reviewer
No confidence in correctness	“It seems to work” is accepted	Evidence-based verification + E2E
Demo breaks	Last-minute changes destabilize app	Feature Freeze + Demo Freeze
Judges expose knowledge gaps	Team outsourced understanding to agents	Workstream reconstruction + final whole-project reconstruction

3. What the framework actually does

The framework turns a time-boxed hackathon into a sequence of controlled decisions and short implementation loops. The shared artifacts keep humans and agents synchronized. The Golden Path determines what matters most. Contracts let frontend and backend progress concurrently. Workstreams create accountable ownership. QA and Reviewer roles create confidence. Incremental merging keeps main close to a working system. E2E proves the critical user journey rather than merely proving isolated files compile.

```

Official challenge
-> shared understanding
-> problem.md
-> MVP + Golden Path
-> Master System Design / plan.md
-> contracts + workstreams + dependencies
-> parallel bounded Builder loops
-> vertical-slice rendezvous
-> QA / risk-based review
-> incremental merge to main
-> Golden Path assembled
-> systematic integration + regression + E2E
-> release/freeze + final review + demo

```

4. What this framework is NOT

- It is not “let AI build everything.” Humans remain responsible for requirements, tradeoffs, approvals, scope, and explanation.
- It is not backend-complete-first. Backend-first means core contracts/data/business logic get early attention, while frontend starts as soon as relevant contracts are clear.
- It is not QA-at-the-end. QA begins from requirements and contracts.
- It is not one giant up-front Waterfall plan. plan.md is shared architecture, while implementation remains iterative and can adapt through controlled decisions.
- It is not four permanent silos. Roles define primary accountability; critical blockers can be swarmed.

- It is not tied to one AI vendor. Supervisor, Builder, and Reviewer are logical roles.

5. Why this hybrid model fits a 24-hour event

A 24-hour event is too short for heavyweight ceremony and too long for pure improvisation. You need enough up-front alignment to prevent four people from diverging, but enough iteration to learn from the real implementation. The model borrows iterative delivery from Agile, risk prioritization from Spiral thinking, approval gates from stage-gated processes, shift-left quality from QA engineering, and frequent integration/release discipline from DevOps. The result is deliberately lightweight but controlled.

PART II - Prerequisite Knowledge Before Touching the Agentic Repository

6. Readiness levels

Level	Knowledge	Minimum proof
Level 0 - Computer/runtime basics	files, folders, terminal, processes, ports, localhost, environment variables, JSON	Run frontend/backend locally and explain which process listens on which port.
Level 1 - Full-stack basics	frontend/backend/database, HTTP, request/response, validation, state vs persistence	Trace one button click from browser to DB and back.
Level 2 - Collaboration/quality basics	Git branch/commit/merge, API contract, CRUD, tests, bug report, regression	Build and merge a tiny feature without breaking main.
Level 3 - Framework basics	problem.md, MVP, Golden Path, plan.md, workstream, vertical slice, Builder/Reviewer	Run one approved workstream from plan to evidence and closure.

7. Terminal, processes, localhost, and ports

A local application is usually several independent processes. The frontend development server may listen on port 5173, the FastAPI server on 8000, and PostgreSQL on 5432. “localhost” means the current computer; the port identifies which process on that computer should receive the network connection.

This matters because many “coding” failures are actually runtime failures: the backend process is not running, the frontend points to the wrong port, CORS blocks a cross-origin request, or the database server is unavailable.

```
Browser -> http://localhost:5173
Frontend JavaScript -> http://localhost:8000/api/...
FastAPI -> PostgreSQL on localhost:5432
```

7.1 Environment variables

Environment variables keep machine-specific or secret configuration outside source code. Typical examples are DATABASE_URL, API base URLs, and tokens. A .env.example documents the names without publishing private values. Team members must know that copying another developer’s exact local path or secret into Git is a mistake.

8. JSON, data types, and schemas

JSON is the common text format carried between browser and API. It can represent objects, arrays, strings, numbers, booleans, and null. An API contract is more than “some JSON”: it specifies which fields exist, which are required, their types, and what the response means.

```
{
  "recipient_id": 7,
  "amount": 2500,
  "note": "Emergency supplies"
}
```

The frontend may store amount as text while the user types. The API schema may require a numeric decimal. Understanding these transformations prevents subtle bugs.

9. State vs persistence

Frontend state is temporary client-side memory that controls what the user currently sees. Database persistence is durable server-side truth that survives refreshes and restarts. A form field changing on screen does not mean the database changed. A successful backend response does not automatically update every UI component unless the frontend changes state or refetches data.

Useful persistence test: perform action -> see result -> reload browser -> result should still be correct if the requirement says it is durable.

10. Error literacy

Symptom	Likely layer	First evidence to inspect
Button does nothing	Frontend event/JavaScript	Browser console
Request never appears	Frontend API client / URL	Network tab
422	Request schema mismatch	Network payload + FastAPI validation response
404	Wrong path or missing resource	API route + path parameter
409	Business/state conflict	Service logic + invariant
500	Unexpected backend failure	Backend traceback/logs
Data visible then disappears after reload	Persistence/state mismatch	DB + API GET response
Works in Swagger but not browser	Frontend/CORS/config	Network tab + backend CORS config

PART III - How a Full-Stack Application Actually Works

11. The end-to-end request flow

```
User
-> page/component
-> input/state
-> event handler
-> frontend API client
-> HTTP/JSON
-> FastAPI router
-> Pydantic validation
-> service/use case
-> business rules
```



```

-> SQLAlchemy/session
-> PostgreSQL
-> commit
-> response schema/JSON
-> browser
-> frontend state/refetch
-> rerender
-> visible result

```

Every teammate should be able to point at this diagram and describe which layer they own, which evidence exists at each boundary, and where a failure can occur.

12. Frontend: what it is responsible for

The frontend is the user-facing program running primarily in the browser. Its responsibilities are presentation, interaction, client-side state, calling APIs, and rendering success/error/loading states. It should not be trusted as the final authority for business rules because users can bypass or manipulate client-side code.

12.1 HTML, CSS, JavaScript

- HTML defines semantic structure: forms, buttons, headings, lists, input fields.
- CSS controls presentation and responsive layout.
- JavaScript controls behavior: events, state changes, API calls, validation feedback, and dynamic rendering.

12.2 React or another framework

A frontend framework is useful when the UI has many reusable components, stateful interactions, and route-based screens. It is not mandatory. For a small hackathon UI, vanilla HTML/CSS/JS may be faster and easier to debug. Choose a framework because the team can use it productively, not because it is fashionable.

12.3 Props, state, effects, and rerenders

In component frameworks, props are inputs from a parent, state is local/client memory that affects rendering, and effects are side effects such as data fetching. A rerender is a consequence of changed state/props. These concepts matter because API integration usually ends by updating state or refetching authoritative server data.

13. HTTP and REST: the conversation between frontend and backend

Method	Typical meaning	Example
GET	Read	GET /incidents/12
POST	Create/initiate	POST /transfers
PATCH	Partial update/state transition	PATCH /requests/9
PUT	Replace/update whole resource	PUT /profile/7
DELETE	Remove	DELETE /items/4
Status	Meaning in practice	
200	Successful request with response body	
201	Resource created	
204	Success with no response body	
400	Application-invalid request	
404	Resource not found	
409	State/invariant conflict	

422	Request did not match validated schema
500	Unexpected server error

REST is not magic; it is a convention for exposing resources and actions through predictable HTTP semantics. Consistency matters because it makes frontend integration, testing, documentation, and debugging easier.

14. API contracts: the bridge before either side is complete

The API contract is the shared agreement between client and server. It defines method, path, request body/query/path parameters, response shape, status codes, and important error semantics. This is what allows backend and frontend to progress in parallel.

14.1 Design-stable vs implementation-stable

A design-stable contract is sufficiently agreed for parallel work even if no real endpoint exists yet. An implementation-stable contract means the backend now implements and tests that agreement. The frontend waits for design clarity, not for every backend line of code.

```

DESIGN-STABLE
POST /transfers
request: { recipient_id, amount }
response: { id, recipient_id, amount, status }
errors: 404 recipient, 409 insufficient funds

IMPLEMENTATION-STABLE
real endpoint exists + tests pass + actual JSON matches the design

```

PART IV - Why This Baseline Tech Stack

15. The baseline is a default, not a law

The framework needs a reliable default so the team does not spend the first hours debating tooling. The recommended baseline is intentionally conventional, well documented, agent-friendly, and fast to debug. A challenge may justify replacing parts of it, but the replacement should be a deliberate Level-2 or Level-3 decision rather than a spontaneous preference.

Layer	Baseline	Why it fits a 24-hour hackathon	When to choose differently
Frontend	HTML/CSS/JS or familiar lightweight framework	Fast feedback, browser-native, easy API integration	Use team-standard framework if it clearly accelerates reusable UI/state.
Backend	FastAPI	Concise Python, type-driven validation, async-ready, automatic OpenAPI/Swagger	Choose another framework if team expertise or challenge ecosystem strongly favors it.
Validation	Pydantic	Schemas double as executable API contracts and clear errors	Alternative framework may have equivalent validation.
ORM	SQLAlchemy	Mature relational	Raw SQL can be fine for

		mapping, transactions, query control	very small systems; another ORM if team is stronger there.
Database	PostgreSQL	Relational integrity, transactions, constraints, concurrency features	SQLite for tiny local-only prototypes; NoSQL if data model truly demands it.
Migrations	Alembic	Repeatable schema evolution across teammates/environments	Skip only for very disposable prototypes where schema reset is acceptable.
Testing	pytest + API/manual/E2E checks	Fast Python tests and clear evidence	Use equivalent stack matching chosen language.
Version control	Git + GitHub	Parallel branches, history, rollback, merge review	Equivalent Git hosting is fine.
Agents	Supervisor / Builder / Reviewer roles	Separates reasoning, implementation, and independent verification	Specific product/vendor can vary.

16. Why FastAPI

A backend framework is needed because browser requests must enter the server through a clear HTTP boundary. FastAPI maps HTTP routes to Python functions, validates data through Pydantic, supports dependency injection, and produces interactive API documentation automatically. This shortens the feedback loop: backend engineers can test endpoints in Swagger before frontend integration begins.

```

from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class TransferCreate(BaseModel):
    recipient_id: int
    amount: float

@app.post("/transfers")
def create_transfer(payload: TransferCreate):
    return {"status": "accepted", **payload.model_dump()}

```

17. Why Pydantic

Pydantic converts an informal idea of input into an executable schema. If amount is required to be numeric and recipient_id an integer, the server can reject malformed requests before business logic runs. This is request validation, not complete business validation. “amount is a number” belongs in the schema; “amount cannot exceed the sender balance” belongs in business logic.

18. Why a service/use-case layer

Putting all logic inside route handlers makes endpoints difficult to test, reuse, and reason about. A service layer orchestrates one business use case: it loads data, checks invariants, invokes logic, writes changes, and returns a result. The router remains responsible for HTTP concerns.

```
Router: HTTP in/out
-> Service: use-case orchestration + transaction boundary
-> Pure logic: reusable decision rules
-> ORM/session: persistence
```

19. Why SQLAlchemy and PostgreSQL

Hackathon problems often involve users, assignments, resources, transactions, requests, statuses, or relationships. A relational database makes those relationships explicit and can enforce important constraints. PostgreSQL adds reliable transactional behavior and mature concurrency features. SQLAlchemy lets Python code express queries and model mappings while still allowing explicit transaction and locking decisions when correctness matters.

19.1 ORM does not replace database knowledge

An ORM is a translation layer, not a substitute for understanding primary keys, foreign keys, uniqueness, indexes, transactions, and query behavior. Teammates should know which rules are enforced by application code and which are enforced by the database.

20. Why Alembic

When one teammate adds a column and another still has the old schema, code can fail even if Git is synchronized. Migrations turn schema changes into ordered, reviewable files that can be applied consistently. In a team hackathon, one migration lane/owner reduces conflicting migration histories.

21. Why Git/GitHub

Every member has a local working copy. Git records snapshots, branches isolate work, and merging turns verified branches into one integrated product. Git is therefore not only backup; it is the coordination mechanism between multiple humans and their agents.

```
main = best-known integrated state
feat/ws03-transfer-api = isolated backend work
feat/ws03-transfer-ui = isolated frontend work
fix/ws03-idempotency = bounded correction
```

22. Why QA and evidence are built into the framework

A hackathon project is judged by what actually works, not by how many files were generated. Evidence converts claims into observable proof: passing tests, API responses, database state, screenshots, logs, and a successful E2E journey. QA begins early because ambiguity and contract defects are cheaper to fix before many dependent components exist.

23. Why separate Supervisor, Builder, and Reviewer roles

A single agent asked to understand, design, implement, test, and approve its own work has a conflict of roles and a very large scope. The framework separates responsibilities. The Supervisor helps the human reason and coordinate. The Builder operates on a bounded workstream. A Reviewer independently checks high-risk work or the final repository. Humans approve material decisions and remain accountable.

PART V - Database, Correctness, Transactions, Concurrency, and Idempotency

24. Relational modeling essentials

Every teammate does not need to become a database specialist, but everyone should understand entity, row, column, primary key, foreign key, relationship, uniqueness, and nullability. These concepts determine what states the system can represent.

Concept	Meaning	Example
Primary key	Unique identity of a row	incident.id
Foreign key	Reference to another row	assignment.incident_id -> incident.id
Unique constraint	Prevents duplicate value/state	one username or one idempotency key
NOT NULL	Field must exist	transfer.amount
Index	Speeds selected lookup patterns	request status or user_id

25. Transactions: all-or-nothing state changes

A transaction groups related database operations so they either commit together or roll back together. If a transfer debits one account and credits another, partial success is unacceptable. This is an atomicity requirement.

```
BEGIN
  validate sender and recipient
  debit sender
  credit recipient
  insert transaction record
COMMIT
```

If any required step fails -> ROLLBACK

26. Concurrency: correct when two things happen at once

Sequential manual testing can hide race conditions. Two requests may read the same balance before either writes it, or two workers may attempt to allocate the same resource. Concurrency control is selected according to the invariant: row locks, optimistic version checks, stronger isolation, uniqueness constraints, or other mechanisms.

The important learning is not to memorize one lock pattern. It is to identify the invariant first: “this balance must never go negative,” “this request can be paid once,” or “this resource can only have one active allocation.” Then choose a mechanism that makes the invariant true under concurrent execution.

27. Idempotency and retries

Networks fail in ambiguous ways. A client may not know whether the server processed a request, so it retries. If the operation is not safe to repeat, the same user action may execute twice. Idempotency gives repeated requests with the same key one logical effect. This is particularly important for money-like, booking, provisioning, and allocation operations.

PART VI - QA, Testing, Debugging, and Evidence

Fundamentals

28. Testing levels

Level	Question	Example
Unit	Does isolated logic behave correctly?	priority_score() returns expected ranking
Integration	Do components cooperate?	service writes DB and returns correct response
Workstream verification	Does this capability meet its exit criteria?	create request + persist + retrieve
E2E	Can the real user journey succeed through the whole system?	browser action -> DB -> visible persisted result

29. Happy, boundary, negative, and adversarial cases

A useful QA mindset asks not only “does the normal input work?” but also where behavior changes. Boundary-value analysis targets edges such as amount=0, exact available balance, maximum capacity, empty lists, first/last valid timestamp, or state-transition boundaries. Negative testing sends invalid or forbidden inputs. Adversarial/reliability testing asks about duplicate clicks, retries, concurrency, stale state, and partial failure.

30. Bug report anatomy

Field	Example
Title	Double-click creates duplicate allocation
Precondition	One unallocated resource available
Steps	Open incident -> double-click Allocate
Expected	One allocation
Actual	Two allocation records
Evidence	Network requests + DB rows + screenshot
Severity	P1 because core state integrity is broken

31. Debug from the boundary inward

Do not guess. Trace the real request. Start at the browser, then HTTP, then backend logs, then service behavior, then database state. Evidence narrows the failing layer.

1. Did the click handler run?
2. Did Network show a request?
3. Was URL/method/body correct?
4. What status/response returned?
5. Did backend log receive it?
6. Did validation pass?
7. Did service/invariant fail?
8. Did DB commit?
9. Did frontend update/refetch?

PART VII - Tiny End-to-End Practice Before the Real Framework

32. Practice goal: one complete “Incident” vertical slice

Before teammates attempt the multi-agent repo workflow, they should build or at least walk through one very small full-stack slice. The goal is not sophistication. The goal is to make every boundary concrete.

32.1 Backend contract

```
POST /incidents
request: { "type": "fire", "severity": 4, "description": "Warehouse" }
response 201: { "id": 1, "type": "fire", "severity": 4, "description": "Warehouse" }
```

32.2 Pydantic schema

```
class IncidentCreate(BaseModel):
    type: str
    severity: int
    description: str | None = None
```

32.3 Persistence model

```
class Incident(Base):
    __tablename__ = "incidents"
    id = mapped_column(primary_key=True)
    type = mapped_column(String(100), nullable=False)
    severity = mapped_column(Integer, nullable=False)
    description = mapped_column(Text, nullable=True)
```

32.4 Service shape

```
def create_incident(db, payload):
    incident = Incident(**payload.model_dump())
    db.add(incident)
    db.commit()
    db.refresh(incident)
    return incident
```

32.5 Frontend shape

```
async function submitIncident(form) {
    const res = await fetch(`${API}/incidents`, {
        method: "POST",
        headers: {"Content-Type": "application/json"},
        body: JSON.stringify(form)
    });
    if (!res.ok) throw new Error("Create failed");
    return await res.json();
}
```

32.6 Proof

1. Open the real UI and submit a valid incident.
2. Observe POST /incidents in Network.
3. Confirm backend returns 201.
4. Query or retrieve the stored incident.
5. Reload the UI and confirm the incident still exists.
6. Try an invalid severity/type and inspect the failure path.

If a teammate cannot explain this small loop, they are not yet ready to supervise an agent implementing a much larger workstream.

33. Pre-framework practice exercises

Exercise	Outcome
Trace an existing feature	Draw browser -> API -> service -> DB -> response -> UI.
Write one API contract	Specify method/path/request/response/errors before coding.
Break the contract intentionally	Cause a 422 or wrong field and diagnose it.
Create a Git branch and merge it	Understand local branch, remote, main, and conflict basics.
Write 8 QA cases	Include happy, boundary, negative, persistence, and integration cases.
Simulate duplicate submit	Understand why client disable alone is not authoritative.
Explain transaction rollback	Describe why partial multi-row updates are dangerous.

PART VIII - Framework Concepts Teammates Must Know Before Agents Start

34. problem.md, MVP, Golden Path, workstream, vertical slice

Concept	Meaning	Question it answers
problem.md	Normalized product requirements	WHAT must the product do?
MVP	Smallest useful product scope	What must exist to deliver value?
Golden Path	Most important successful user journey	What journey proves the MVP works?
plan.md	Master System Design	HOW will the product be built?
Workstream	Bounded engineering objective/capability	What unit of work can one owner drive?
Vertical slice	Capability implemented through required layers	Does this user capability actually cross UI/API/logic/data?
Golden-Path E2E	Real end-to-end proof	Does the assembled critical journey actually work?

35. Why one shared problem.md and plan.md

Each teammate has a local Git copy, but there is one canonical project interpretation. If backend, frontend, and QA each create their own problem.md or Master Design, agents will faithfully implement conflicting truths. Individuality belongs in bounded workstream plans, not competing architectures.

36. Builder Planning Phase vs plan.md

plan.md persists the whole-system architecture and contracts. Builder Planning Phase is temporary, repo-grounded planning for one bounded workstream. A Builder should first reconstruct the current repository, propose which files it will change and how it will verify them, and then stop for human approval before implementing material work.

37. Material changes and escalation

A Builder may make local design-preserving choices, but architecture, public API contracts, shared schemas, core invariants, major dependencies, Golden Path, and MVP changes require escalation. This keeps agent autonomy useful without allowing silent divergence.

38. Readiness gate before teammates use the real repo

Question	Ready when teammate can...
Full-stack flow	Trace one user action to DB and back without hand-waving.
API contract	Write request/response/error contract before implementation.
State/persistence	Explain why reload distinguishes client state from durable truth.
Git	Create branch, sync main, commit, resolve a simple conflict, merge.
Backend	Explain router -> schema -> service -> DB.
Frontend	Explain event -> state -> API client -> state/refetch -> render.
QA	Create risk-based cases and distinguish unit/integration/E2E.
Agents	Explain Supervisor vs Builder vs Reviewer and when Builder must stop.
Framework	Explain problem.md -> Golden Path -> plan.md -> workstream -> merge -> E2E.

If several answers are “I only know because the agent will do it,” pause and rehearse before the event.

PART IX - The Repository Framework and Team Operating System

The remaining chapters move from prerequisite learning into the full operational framework. The following material is intentionally detailed and can be used during rehearsals and the event itself.

PART III - The Generic Repository Framework

15. One repository, multiple operating modes

The generic starter should support SOLO, TEAM_3 and TEAM_4 modes. The engineering core does not change. Team modes add coordination rules. A solo developer should not simulate four humans; a team should not abandon the shared artifacts that make the solo workflow coherent.

GENERIC HACKATHON STARTER

Core engineering system

problem.md
plan.md
execute.md
review.md
README.md
AGENTS.md
backend/
frontend/
tests/
migrations/

Operating modes

SOLO
TEAM_3
TEAM_4

16. Authority hierarchy

1. Official rulebook and organizer clarifications
2. Official challenge statement
3. Approved problem.md
4. Approved plan.md
5. Actual code, migrations, tests and runtime evidence
6. execute.md, review.md and explanatory documentation

This ordering matters when artifacts disagree. For example, if execute.md claims a capability is done but runtime evidence fails, the code/runtime truth wins. If plan.md proposes a feature that official rules prohibit, official rules win.

17. problem.md - WHAT

problem.md is the normalized product requirement document. It should be challenge-specific but implementation-neutral enough to avoid locking into technology prematurely. The lead reconciles it; the team approves it. There is one canonical version, even though each Git clone contains a local copy.

Recommended structure

```
# Problem
## Product purpose
## Users / actors
## Required workflows
## Inputs / outputs
## Frontend requirements
## Backend requirements
## Persistence requirements
## Business rules and invariants
## Constraints and official clarifications
## Assumptions
## Acceptance criteria
## MVP boundary
## Explicit / inferred / optional separation
```

18. plan.md - HOW

plan.md is the Master System Design. It records architecture decisions that all workstreams must obey unless a formal shared decision changes them. It is not a per-agent implementation plan.

```
# Master System Design
## Golden Path
## Architecture
## Frontend structure
## Backend structure
## Entities / relationships
## Business invariants
## API contracts
## Workstream map
## Dependency graph
## Ownership
## Integration strategy
## Test strategy
## Release criteria
```

19. execute.md - LIVE execution state

execute.md is the live board for workstreams, blockers, evidence and next actions. It should answer “What is happening now?” rather than restating the design.

```
## WS-03 - Direct Transfer
Primary Owner: Member B
Frontend Owner: Member C
Verification Owner: Member D
Integration Owner: Member A
Golden Path Relevance: CRITICAL
Dependencies: WS-01
Approved Contract: POST /transfers
Status: IN PROGRESS
Backend: [x]
Frontend: [~]
Persistence: [x]
Integration: [ ]
Verification: [ ]
Evidence: 18 backend tests pass
Blocker: frontend still using mock
Next: integrate against real endpoint
```

20. review.md, README.md, AGENTS.md

File	Purpose	Not for
review.md	Verified findings, severity, verdict and resolution history	Live task tracking
README.md	Project-facing explanation, setup, features, demo, limitations	Architectural promises that do not exist
AGENTS.md	Operating constraints for humans/agents in this repository	Challenge requirements themselves

21. Workstream state machine

PLANNED

- > READY
- > IN PROGRESS
- > VERIFYING
- > COMPLETE

At any active stage:

- > BLOCKED
- > decision / dependency resolution
- > return to READY or IN PROGRESS

A workstream is complete only when its relevant implementation, integration and verification exit criteria are satisfied. “My code is done” is not a completion state.

PART IV - The 4-Member 24-Hour Operating Model

22. Role A - Technical Lead / System Integrator

The lead is the keeper of coherence. They facilitate requirement normalization, protect the Golden Path, own architecture reconciliation, coordinate shared contracts, watch dependencies, manage merge order, supervise critical integration, and control scope/release decisions. They can code, but should not disappear into a feature for hours while the team loses a system-level owner.

- Own/reconcile problem.md and plan.md
- Maintain workstream/dependency map
- Approve or escalate Level 2/3 decisions
- Coordinate shared API/data contracts
- Protect main and merge order
- Lead integration swarms and release decisions
- Ensure final reconstruction and demo readiness

23. Role B - Backend / Data / Core Logic Engineer

The backend owner implements server-side contracts and protects data/business correctness. This role usually owns migrations and transaction-sensitive capabilities. They should publish implementation-stability evidence so frontend and QA know when the real endpoint is safe to integrate.

- FastAPI routers and schemas
- service/use-case orchestration
- business invariants
- SQLAlchemy models and sessions
- migrations and data integrity
- transactions/concurrency when relevant
- backend/API tests
- runtime evidence and logs

24. Role C - Frontend / UX / Client Integration Engineer

The frontend owner turns approved workflows and contracts into usable interaction. They should not wait for backend completion; once a contract is design-stable, they can build against a mock or typed interface, then rendezvous with the real endpoint as soon as it becomes implementation-stable.

- Page/component structure
- forms and client validation
- API client
- loading/error/success states
- responsive behavior
- mock-to-real transition
- browser Network inspection
- frontend integration evidence

25. Role D - QA / Integration / Reliability / Utility Engineer

The QA owner begins at requirements. They derive test scenarios, challenge contracts, verify high-risk behavior, reproduce bugs, check database state, coordinate fresh Reviewer agents, run regression and E2E, and help the team distinguish P0/P1/P2. They also act as a utility engineer when critical-path work needs another pair of hands.

- Risk-based test matrix
- API and negative testing
- DB/persistence checks
- contract drift detection
- integration and regression
- Reviewer orchestration
- E2E and deployment verification
- defect evidence and severity

26. Responsibility matrix

Area	A Lead	B Backend	C Frontend	D QA
Challenge intake	A/R	C	C	C
problem.md	A/R	C	C	C
Golden Path	A/R	C	C	C
plan.md	A/R	C	C	C
API design	A	R	R/C	C
DB/migrations	C	A/R	I	C
Frontend	C	I	A/R	C
Backend	C	A/R	I	C
QA matrix	C	C	C	A/R
Merge/integration	A/R	C	C	C
E2E	A	C	C	R
Release	A/R	C	C	C

A = accountable, R = responsible, C = consulted, I = informed. The matrix prevents ambiguity but does not create walls. If the Golden Path is blocked, the team swarms the blocker.

27. First 90 minutes - collaborative intake

1. Every member reads official challenge/rules independently.
2. Lead facilitates a 10-15 minute shared interpretation: users, required workflows, constraints, technically risky behavior.
3. Shared Supervisor performs Challenge-Intake Understanding Reconstruction.
4. Lead drafts problem.md; team checks explicit vs inferred vs optional requirements.
5. Team agrees MVP.
6. Team identifies Golden Path.
7. QA begins a risk/testability list before implementation decisions.

The reason everyone participates is practical: backend must understand invariants, frontend must understand the user journey, QA must understand expected behavior, and the lead must understand the whole product. If only one person understands the challenge, later parallelism becomes guesswork.

28. Hours 1.5-3 - Master System Design

1. Define entities, relationships and important invariants.
2. Define architecture boundaries.
3. Define design-stable API contracts for Golden-Path capabilities.
4. Define frontend screens and state flow.
5. Decompose into capability workstreams.
6. Add dependencies and Golden-Path relevance.
7. Assign primary, frontend, verification and integration ownership.
8. Initialize execute.md.
9. Agree branch naming and shared-file ownership.
10. Stop and confirm the team can explain the design before coding spreads out.

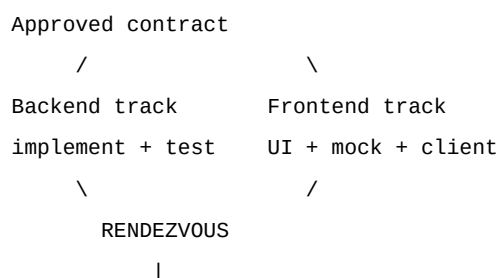
29. Parallel execution after Hour 3

Parallelism is dependency-aware. It is normal for all members to be active, but not required that everyone writes code continuously. The team optimizes for Golden-Path progress, not keyboard utilization.

Member	Early parallel work
A Lead	integration skeleton, auth/foundation or next critical contract, dependency/merge supervision
B Backend	DB foundation, first API/service, migrations, core logic
C Frontend	frontend shell, routes/pages, API client, contract-compatible mocks
D QA	test matrix, API contract checks, environment verification, first negative cases

30. The frontend/backend rendezvous

Backend and frontend start from a design-stable contract, progress independently, and meet as soon as both sides are sufficiently ready. This meeting is a technical synchronization point, not necessarily a formal meeting.



```

remove mock / point to real API
|
Network + server logs + DB inspection
|
fix contract/integration bugs
|
QA integrated verification
|
merge / close slice

```

31. When people finish at different times

Situation	What happens next
Backend first	Verify endpoint -> merge safe backend -> frontend syncs main -> integrate real API. Backend moves to next READY dependency-aware workstream.
Frontend first	Finish UI/loading/error states against mock -> avoid claiming integration -> move to another READY UI/support task. When backend lands, sync and replace mock.
QA test design first	Begin API tests when endpoint exists, verify UI when available, expand regression plan, inspect ambiguous contract behavior.
Both finish together	Immediate rendezvous, real integration, focused bug fixing, QA verification.
Lead finishes a task early	Do not start random feature. Check dependency graph, merge queue, blockers, Golden Path and team understanding.

32. Contract drift and contract evolution

Contract drift means implementation disagrees with the approved contract. Contract evolution means the approved contract itself must change because a real requirement was missed. Treat them differently.

DRIFT

```

plan.md: recipient_id
frontend: recipient_id
backend: receiver_id  <-- implementation wrong
Action: fix backend implementation.

```

EVOLUTION

```

Approved activity UI requires created_at, but contract omitted it.
Action: NEED CONTRACT DECISION -> affected owners + lead -> approve -> update plan.md -> notify -> implement.

```

33. Decision authority

Level	Examples	Authority
1 Local	function names, internal helper, local refactor, test organization	Workstream owner
2 Shared	public API, shared schema, DB relationship, new shared dependency, cross-workstream behavior	Lead + affected owners
3 Product	MVP, Golden Path, architecture, major	Team + lead + shared Supervisor

	scope, release/deployment strategy	
--	------------------------------------	--

34. Team synchronization cadence

Use 5-10 minute syncs every 2-3 hours, plus immediate coordination for shared-contract/schema decisions. The purpose is not reporting activity; it is exposing blockers and integration risk.

WORKSTREAM:

STATUS:

DONE:

BLOCKED:

CONTRACT CHANGE:

NEED DECISION:

TESTS / EVIDENCE:

MERGE STATE:

NEXT:

PART V - The Agentic Layer: How Humans and AI Agents Cooperate

35. Roles: Supervisor, Builder, Reviewer

The agentic roles are logical responsibilities, not vendor names. A reasoning agent can serve as Control/Supervisor, a repository-capable coding agent can serve as Builder, and a fresh reasoning/coding session can serve as Reviewer. Humans remain accountable for scope, architecture-changing decisions, acceptance and explanation.

Agent role	Primary job	Must not silently do
Control / Supervisor	requirements reasoning, design supervision, prompts, teaching, debugging strategy, scope/release	become default implementation agent or invent unapproved product requirements
Builder	reconstruct repo, plan bounded work, implement, test, debug, report evidence	redesign shared architecture/contracts/schema without approval
Reviewer	independently verify requirements, risks, contracts, tests, evidence	rewrite everything or expand scope before reporting findings

36. One shared Supervisor, optional personal Supervisors

The Technical Lead should maintain one primary team-level Supervisor context for requirements, architecture, Golden Path, workstream conflicts and release. Individual members may use personal Supervisors for workstream teaching, Builder-plan review and debugging. Personal Supervisors are subordinate to repository authority and must not create alternative architectures.

PRIMARY SHARED SUPERVISOR

-> problem.md / Golden Path / plan.md / dependencies / major decisions

Member B Supervisor -> Backend Builder

Member C Supervisor -> Frontend Builder

Member D Supervisor -> Reviewer / QA assistant

Member A -> integration + team Supervisor

37. Builder Planning Phase vs plan.md

plan.md is persistent whole-system design. Builder Planning Phase is a repo-grounded plan for one bounded workstream. The Builder should first reconstruct current code, identify files and dependencies, propose steps/tests/conflict risks, and stop for human approval before implementation.

38. Universal Builder prompt

ROLE:

You are the Builder for one bounded workstream in a shared multi-person hackathon repository.

WORKSTREAM: WS-XX - <name>

HUMAN OWNER: <member>

OBJECTIVE: <bounded capability>

GOLDEN PATH RELEVANCE: Critical / Supporting / Optional

READ FIRST:

- AGENTS.md
- problem.md
- plan.md
- execute.md
- relevant code/tests/migrations

TEAM CONTEXT:

Other humans and Builder agents are working concurrently on separate workstreams.

Do not silently change shared architecture, public contracts, business invariants, shared schemas, or another member's owned scope.

DEPENDENCIES: ...

APPROVED CONTRACTS: ...

IN SCOPE: ...

OUT OF SCOPE: ...

FIRST reconstruct relevant repository state.

THEN enter Builder Planning Phase / Plan Mode.

Return:

1. current-state summary
2. files likely to change
3. implementation steps
4. testing plan
5. integration implications
6. dependency concerns
7. merge/conflict risks

STOP before implementation.

After human approval: implement only approved scope, test/debug, preserve contracts, record evidence.

STOP AND REPORT material architecture/schema/API/business-policy/dependency/scope changes.

39. Reviewer prompt

ROLE: Independent Reviewer

WORKSTREAM: WS-XX

Read problem.md, plan.md, execute.md, implementation diff, relevant tests/migrations.

Verify requirement coverage, invariants, contract adherence, persistence, important negative paths, concurrency if relevant, integration and missing verification.

Do not implement initially.

Classify: BUG / DESIGN ISSUE / CONTRACT DRIFT / INTEGRATION FAILURE / MISSING VERIFICATION / DOC DRIFT / DEFERRED / IMPROVEMENT

Severity: P0 / P1 / P2

Verdict: PASS / PASS WITH NON-BLOCKING FINDINGS / BLOCKED

Return finding, evidence, impact and recommended action.

40. Escalation rule

Stop instead of “helpfully” redesigning

If a Builder discovers that the approved architecture, public API, shared schema, business invariant, major dependency or another member’s workstream must change, it stops and reports the issue. The human owner escalates through the decision authority. This single rule prevents multi-agent architectural drift.

PART VI - Integration, Git, QA, Debugging and Release Runbooks

41. Branching protocol

main = best-known integrated state

feat/ws01-persistence

feat/ws03-transfer-api

feat/ws03-transfer-ui

test/ws03-transfer

fix/ws03-idempotency

1. Sync main before starting a workstream.
2. Create a short-lived bounded branch.
3. Commit coherent changes with clear message.
4. Before merge, fetch main and reconcile conflicts on the feature branch.
5. Run focused tests again after conflict resolution.
6. Open a concise PR or use team merge protocol.
7. Integration owner confirms contract/scope/risk.
8. Merge and immediately make the integrated state visible to the team.

42. Shared-file ownership

Artifact	Default owner
problem.md	Lead
plan.md	Lead
execute.md	Owners edit their sections or Lead reconciles
review.md	QA/Reviewer
README.md	Designated documentation owner
Alembic migrations	Backend/persistence owner
Central frontend API client	Frontend owner

43. Merge readiness checklist

- scope matches approved workstream
- relevant tests pass
- no accidental files or secrets
- shared contract unchanged or formally updated
- migration impact reconciled
- feature branch includes latest required main changes
- real integration performed if the slice requires it
- QA evidence present for critical Golden-Path behavior

- execute.md status/evidence updated
- no known P0 blocker

44. QA test matrix template

Capability	Happy	Boundary	Negative	Persistence	Integration	Concurrency/Retry	Regression	E2E
Create incident	valid create	max severity	missing type	row persists	UI->API	N/A	list still works	yes
Allocate resource	valid allocation	exact remaining qty	insufficient qty	allocation + stock	UI->service	two simultaneous allocations	incident view	yes

The QA owner should prioritize tests by impact and likelihood. P0 risks such as data corruption, impossible state, security bypass or total Golden-Path failure deserve more attention than cosmetic edge cases.

45. Debugging full-stack failures

Debugging should follow the runtime chain rather than random edits. Start where the symptom is observable, then walk boundary by boundary.

1. Browser UI: did the user action fire?
2. Browser Network: correct method/path/payload? response/status?
3. Backend logs: request arrived? validation/service exception?
4. API contract: request/response fields match plan.md?
5. Service/business logic: invariant rejected? wrong branch?
6. Database: expected rows/state? transaction committed?
7. Frontend state: response consumed? stale cached state? rerender?
8. Reload: does persistent state survive?

46. Common failure signatures

Symptom	Likely area	First evidence
422	request shape/type mismatch	Network payload + Pydantic schema
404	wrong path/ID/resource	Network URL + router + DB
409	business/state conflict	service rule + current DB state
500	unhandled backend failure	server traceback
UI says success but reload loses it	persistence/commit/mock	DB + Network + frontend mock check
Backend works in Swagger but UI fails	CORS/API client/contract	Browser Network + console
Two simultaneous actions oversubscribe	concurrency/transaction	DB transaction test/logs

47. Incremental merging

- main v1 - starter/foundation
- main v2 - persistence + migrations
- main v3 - first Golden-Path API
- main v4 - first integrated vertical slice
- main v5 - second slice
- main v6 - critical business logic
- main v7 - Golden Path assembled
- main v8 - hardened/release candidate

Do not wait until the end to combine frontend and backend. Incremental merging makes integration failures small, recent and attributable.

48. Golden-Path E2E runbook

1. Reset or know the starting state.
2. Start real frontend, backend and database.
3. Open the actual UI.
4. Perform the critical user action through the browser.
5. Observe request and response in Network.
6. Observe server logs if useful.
7. Verify important database state.
8. Confirm visible UI result.
9. Reload/refetch and confirm persistence.
10. Run one high-value invalid path.
11. Record pass/fail evidence and defects.

49. Regression strategy

Every new merge can break older behavior. Maintain a small critical regression set: the earliest Golden-Path entry, one important state transition, one persistence check, one negative path and any previously fragile bug. Run the smallest high-value set frequently; run the fuller set before release.

50. Feature Freeze, release decision and Demo Freeze

Feature Freeze means the team stops adding optional capability and focuses on correctness, integration, tests, docs and release. Demo Freeze is later and stricter: stop risky modifications so the known demo state cannot be destabilized.

Release decision questions

- Is deployment required by rules?
- Does local Golden-Path E2E pass?
- Is hosting already understood?
- Do we have enough time for production config, migrations and deployed E2E?
- Would a local demo be more reliable if allowed?

51. Suggested 24-hour timeline

Hours	Primary objective
0-1.5	Challenge intake, problem.md, MVP, Golden Path
1.5-3	Master design, contracts, workstreams, ownership, execute.md
3-6	foundation, DB, backend skeleton, frontend shell, QA

	matrix, first API/UI contact
6-10	core capability vertical slices
10-13	major business/decision logic and high-risk behavior
13-16	complete critical Golden Path
16-18	systematic integration/hardening
18-20	local Golden-Path E2E + regression
~20	Feature Freeze target
20-21	P0/P1 corrections
21	release decision
21-22.5	deployment if selected + deployed E2E
22.5-23	final independent review
23-23.5	README/execute/review reconciliation
23.5-23.8	whole-project reconstruction
23.8-24	demo rehearsal, Git/source checkpoint, Demo Freeze, submission

52. Fatigue as an engineering risk

A 24-hour event is not a contest to keep everyone awake continuously. Fatigue increases defect rate, poor architecture decisions and communication failures. Stagger meals and short recovery windows; keep at least one system-level owner and one implementation-capable member active; avoid major architecture changes late when everyone is exhausted; have the full team awake and aligned for final integration and demo.

PART VII - Full Worked Example: RapidRelief Resource Allocation System

This fictional example demonstrates the complete framework with concrete API, database, frontend, QA and Git behavior. The exact technology choices are examples; the generic methodology remains tool-agnostic.

53. Official challenge statement (fictional)

Challenge

Build a web application for a city emergency operations center. Dispatchers report incidents, see available response resources, receive a recommended allocation, confirm deployment, and monitor remaining capacity. The system should prevent over-allocation, preserve a history of decisions, and remain correct if two dispatchers act at nearly the same time. Use fake/demo data only. A working user-facing web experience and backend persistence are required.

54. Challenge-intake reconstruction

Question	Answer
Who uses it?	Emergency dispatcher / operations coordinator.
Core problem	Allocate limited resources to incidents without over-committing shared capacity.
Critical data	Incidents, resources, allocations, statuses, timestamps.
Hard engineering risk	Concurrency: two dispatchers may allocate the same remaining units.
Frontend responsibility	Create incident, show recommendation, confirm allocation, show dashboard/error states.
Backend responsibility	Validate requests, rank resources, enforce availability, perform atomic allocation, expose history.
Persistence responsibility	Durably store incidents, resources and allocations; enforce relational integrity.
Optional ideas	Map view, analytics, advanced optimization, notifications.

55. problem.md summary

Problem - RapidRelief

Users: dispatchers.

Required workflows:

1. Create incident with type, severity, affected_people, description.
2. View available resources.
3. Request recommendation for an incident.
4. Confirm an allocation.
5. View incident/allocation status and remaining capacity.

Invariants:

- allocation quantity > 0

- resource must exist and be active
- allocation cannot exceed available quantity
- confirmed allocation decrements availability exactly once
- failed allocation leaves availability unchanged
- incident/resource/allocation relationships remain consistent
- concurrent confirmations cannot over-allocate

MVP boundary:

incident -> recommendation -> allocation -> updated dashboard/history.

56. MVP and Golden Path

MVP: incident reporting, resource inventory, recommendation, confirm allocation, dashboard/history.

Golden Path: dispatcher creates a high-severity incident -> receives ranked resource recommendation -> confirms one allocation -> backend atomically verifies/decrements capacity and stores allocation -> UI shows allocated incident and updated remaining resource -> reload confirms persistence.

57. Master architecture

React/Vite example frontend

```
pages: Dashboard, NewIncident, IncidentDetail
api/client.ts
  | HTTP/JSON
  v
```

FastAPI backend

```
routers: incidents, resources, allocations
schemas: Pydantic request/response
services: incident_service, allocation_service
logic: recommendation.py
models: Incident, Resource, Allocation
database: SQLAlchemy Session
  |
  v
```

PostgreSQL

```
incidents
resources
allocations
```

58. Data model

Entity	Important fields	Relationship
Incident	id, type, severity, affected_people, description, status, created_at	has many allocations
Resource	id, name, category, total_qty, available_qty, active	has many allocations
Allocation	id, incident_id, resource_id, quantity, status, created_at	belongs to incident and resource

Example SQLAlchemy model

```
class Resource(Base):
    __tablename__ = "resources"

    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str] = mapped_column(String(120), nullable=False)
    category: Mapped[str] = mapped_column(String(60), nullable=False)
    total_qty: Mapped[int] = mapped_column(nullable=False)
    available_qty: Mapped[int] = mapped_column(nullable=False)
    active: Mapped[bool] = mapped_column(default=True, nullable=False)
```

59. Design-stable API contracts

Method / path	Request	Response
POST /incidents	type, severity, affected_people, description	IncidentRead, 201
GET /resources	-	ResourceRead[]
GET /incidents/{id}/recommendations	-	RecommendationRead[]
POST /allocations	incident_id, resource_id, quantity	AllocationRead, 201
GET /incidents/{id}	-	IncidentDetail

Allocation contract

POST /allocations

Request

```
{
  "incident_id": 12,
  "resource_id": 4,
  "quantity": 2
}
```

201 Response

```
{
  "id": 31,
  "incident_id": 12,
  "resource_id": 4,
  "quantity": 2,
  "status": "confirmed",
  "remaining_available": 3
}
```

409 examples

- insufficient available quantity
- resource inactive
- allocation already processed where applicable

60. Workstream map

WS	Capability	Owner(s)	Dependency	Golden Path
01	Persistence + seed resources	B, A integration	-	Supporting
02	Create incident	B API, C UI, D verify, A integrate	01	Critical
03	Recommendation engine	B, D review	01/02	Critical
04	Confirm allocation	B API, C UI, D verify, A integrate	01/02	Critical
05	Dashboard/history	C UI, B read API, D verify	02/04	Critical
06	Concurrency hardening	B, D Reviewer, A decision	04	Critical risk
07	Golden Path integration	A lead, all members	02-06	Critical
08	Optional map/analytics	whoever free	07	Optional

61. Hour 3 parallel startup

Member B starts WS-01 persistence and the POST /incidents backend. Member C creates the frontend shell and NewIncident form against the approved contract. Member D writes create-incident and allocation test matrices, including concurrency risk. Member A keeps the dependency map, helps integrate database/session foundation and prepares the next shared contract.

62. Backend implementation - create incident

```
class IncidentCreate(BaseModel):
    type: str = Field(min_length=1, max_length=100)
    severity: int = Field(ge=1, le=5)
    affected_people: int = Field(ge=0)
    description: str | None = None

@router.post("/incidents", response_model=IncidentRead, status_code=201)
def create_incident(payload: IncidentCreate, session: Session = Depends(get_session)):
    return incident_service.create(session, payload)

def create(session: Session, payload: IncidentCreate) -> Incident:
    incident = Incident(**payload.model_dump(), status="open")
    session.add(incident)
    session.commit()
    session.refresh(incident)
    return incident
```

Focused backend evidence: valid incident returns 201; severity 6 returns 422; persisted row exists; GET detail returns the created incident.

63. Frontend implementation while backend is still building

```
// api/incidents.ts
export type IncidentCreate = {
  type: string;
  severity: number;
  affected_people: number;
  description?: string;
```

```
};

export async function createIncident(payload: IncidentCreate) {
  // Initially a temporary contract-compatible mock may be used.
  const res = await fetch(`${API_BASE}/incidents`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(payload),
  });
  if (!res.ok) throw new Error(await res.text());
  return res.json();
}
```

Member C can build the form, loading state, success navigation and error message before the endpoint is runtime-ready. As soon as Member B reports implementation-stable, Member C points the client at the real backend and performs rendezvous testing.

64. Recommendation engine

For the MVP, the recommendation can be deterministic and explainable rather than over-engineered. Example scoring: match category to incident type, prefer active resources with enough availability, then rank by remaining capacity and predefined suitability weight. The important engineering rule is that recommendation is advisory; the allocation transaction must re-check current availability because data can change between recommendation and confirmation.

```
def recommend(resources, incident):
    candidates = [r for r in resources if r.active and r.available_qty > 0]
    scored = []
    for r in candidates:
        match = category_match_score(r.category, incident.type)
        capacity = min(r.available_qty, 10)
        score = match * 100 + capacity
        scored.append((score, r))
    return [r for _, r in sorted(scored, reverse=True)]
```

Why QA cares

A correct recommendation function is not enough. QA should test empty resources, inactive resources, tie behavior, changing availability, and that a stale recommendation cannot force an invalid allocation.

65. Allocation transaction and concurrency

WS-04 is high risk because availability is shared mutable state. The service must verify the resource inside the transaction and ensure two concurrent confirmations cannot both consume the same remaining units. The exact locking strategy depends on database and architecture, but the invariant is fixed: available quantity never becomes negative and successful allocations are reflected exactly once.

```
def allocate(session, payload):
    # Conceptual example: lock resource row in the transaction.
```

```

resource = session.execute(
    select(Resource)
    .where(Resource.id == payload.resource_id)
    .with_for_update()
).scalar_one_or_none()

if resource is None:
    raise NotFound("resource")
if not resource.active:
    raise Conflict("resource inactive")
if resource.available_qty < payload.quantity:
    raise Conflict("insufficient availability")

resource.available_qty -= payload.quantity
allocation = Allocation(
    incident_id=payload.incident_id,
    resource_id=resource.id,
    quantity=payload.quantity,
    status="confirmed",
)
session.add(allocation)
session.commit()
session.refresh(allocation)
return allocation

```

The example demonstrates the idea, not a universal concurrency recipe. The team still needs transaction/error handling appropriate to the chosen database and session lifecycle.

66. QA scenarios for allocation

Risk	Test	Expected
Happy path	available=5, allocate=2	201, allocation row, available=3
Boundary	available=2, allocate=2	success, available=0
Negative	available=2, allocate=3	409, no allocation, available stays 2
Validation	quantity=0	422 or defined validation error
Missing	unknown resource ID	404
Inactive	inactive resource	409
Concurrency	two allocate=2 requests while available=3	only valid total succeeds; availability never negative
Persistence	successful allocation then reload	UI/GET reflects durable remaining quantity

67. Example contract drift at rendezvous

Suppose plan.md and frontend use resource_id, but the backend accidentally implements responder_id. Swagger looks fine to the backend developer because its own schema is internally consistent. The browser sends resource_id and receives 422. QA compares runtime evidence to the approved contract. This is contract drift, so backend implementation changes; plan.md does not change.

68. Example legitimate contract evolution

The IncidentDetail page is approved to show “allocated at 14:32,” but the original AllocationRead omitted created_at. Frontend reports NEED CONTRACT DECISION. Lead, backend and frontend agree created_at is required by the already-approved workflow. plan.md is updated; backend response schema and frontend type both change. This is controlled evolution, not drift.

69. Git progression in the example

```
main v1  starter + environment
main v2  WS-01 DB models + migration + seeds
main v3  WS-02 incident backend
main v4  WS-02 integrated NewIncident slice
main v5  WS-03 recommendation API + tests
main v6  WS-04 allocation backend + concurrency evidence
main v7  WS-04 integrated allocation UI
main v8  WS-05 dashboard/history
main v9  WS-07 Golden Path assembled
main v10 hardened release candidate
```

70. Full Golden-Path E2E

1. Seed two resource types with known capacities.
2. Open real frontend.
3. Create high-severity fire incident.
4. Incident detail loads recommendation list.
5. Choose recommended fire unit and allocate quantity 2.
6. Browser POST /allocations returns 201.
7. Backend transaction decrements capacity and persists allocation.
8. UI shows confirmed allocation and new remaining capacity.
9. Refresh page.
10. GET/detail still shows allocation and reduced capacity.
11. Attempt allocation above remaining capacity.
12. System shows defined error and database state remains unchanged.

71. Reviewer finding example

P1 - Missing concurrency verification

Reviewer sees row locking in allocation service but no automated or reproducible concurrency test. Impact: the core integrity claim is not evidenced. Recommendation: add a focused concurrent integration test or a repeatable two-request verification procedure before release. Verdict: PASS WITH IMPORTANT FINDING until evidence is

added.

72. Final demo story

The demo should tell the product story, not wander through screens. “A dispatcher reports an incident. RapidRelief ranks available resources. When the dispatcher confirms an allocation, the backend re-checks current capacity inside a transaction so two simultaneous dispatchers cannot over-allocate. The allocation is persisted, remaining capacity updates, and a reload proves the state is durable. We also test an insufficient-capacity case to show the system rejects invalid operations without corrupting inventory.”

PART VIII - 3-Member Alternative

73. Role redistribution

4-member responsibility	3-member owner
Lead / integration	Member A - Lead + Integration + Backend
Backend/data/core logic	Member B - Backend + DB + Core Logic
Frontend/UX/client integration	Member C - Frontend + QA + E2E
Dedicated QA	Distributed: C primary, A/B self-verify; fresh Reviewer agents for independence

The 3-member model has less parallel capacity, so scope discipline becomes more important. Optional work should be cut earlier, and the lead must protect time for integration rather than becoming a second full-time backend developer.

74. How review independence is preserved

Even without a dedicated QA human, a developer should not be the only source of confidence for their own high-risk work. Member C can independently exercise APIs and E2E; Member A can review Member B's core logic at a high level; a fresh Reviewer agent can inspect concurrency/state-machine code; and the team can perform cross-member verification before merge.

75. 3-member parallel pattern

Member A: architecture + integration + one critical backend/foundation task

Member B: database + core backend + transaction/business logic

Member C: frontend + test matrix + API/E2E verification

When critical blocker appears:

A traces system/contract

B debugs backend/data

C reproduces from UI/API and records evidence

76. 3-member scope policy

- Protect Golden Path over optional analytics/polish
- Avoid duplicate backend ownership between A and B; split by workstream
- Use fewer simultaneous workstreams
- QA scenarios are written before C becomes consumed by UI polish
- Reviewer agents are used more selectively on high-risk logic
- Cross-train so at least two people understand every critical subsystem

PART IX - Teammate Learning Curriculum Before the Event

77. Minimum shared curriculum

Module	What to learn	Practice outcome
A. Full-stack flow	browser -> HTTP -> API -> service -> DB -> response	Trace one request end to end.
B. Contracts	method/path/request/response/status	Write and critique one API contract.
C. Git	clone, branch, pull, commit, merge conflict, PR	Merge two small branches into main.
D. Repo artifacts	problem.md, plan.md, execute.md, review.md	Explain what each file controls.
E. Backend basics	FastAPI, Pydantic, service, SQLAlchemy	Build one CRUD-ish endpoint.
F. Frontend basics	form/state/fetch/loading/error	Consume a real API.
G. DB basics	models, relationships, migration, transaction	Persist and reload state.
H. QA basics	happy/boundary/negative/integration/E2E	Create a test matrix.
I. Agent workflow	Supervisor/Builder/Reviewer/human gates	Run one bounded Builder cycle.
J. Integration	rendezvous, Network/logs/DB	Fix one intentional contract mismatch.

78. Suggested 2-session team rehearsal

Session 1 - 2 to 3 hours

1. Explain the handbook mental model.
2. Give a tiny challenge and jointly create problem.md, Golden Path and plan.md.
3. Agree two API contracts.
4. Split into backend/frontend/QA/lead work.
5. Use agents with bounded prompts.
6. Integrate one vertical slice.
7. Intentionally introduce a contract mismatch and debug it.

Session 2 - 3 to 4 hours

1. Run a mini-hackathon with 3-4 workstreams.
2. Use real branches and incremental merges.
3. Require QA matrix before implementation is complete.
4. Run one Reviewer on high-risk work.
5. Perform Golden-Path E2E and regression.
6. End with 5-minute judge explanation from each member.

79. Member-specific readiness

Technical Lead

- Can normalize challenge without overdesigning
- Can explain Golden Path vs MVP
- Can define API contracts with backend/frontend
- Can read dependency graph and reassign work
- Can resolve Level 2/3 decisions
- Can trace full-stack bugs and control release

Backend

- Can implement contract in FastAPI/Pydantic
- Understands service vs router responsibility
- Can use SQLAlchemy session and migration workflow
- Can explain transaction/invariant behavior
- Can provide focused test/runtime evidence
- Can recognize when concurrency matters

Frontend

- Can implement from design-stable contract
- Can use mock temporarily and remove it
- Can inspect Network/console
- Can handle loading/error/success states
- Can sync branch with backend merge
- Can verify reload/persistence behavior

QA/Integration

- Can derive tests from requirements/invariants
- Can distinguish 422/409/404/500 style failures
- Can reproduce and document bugs
- Can inspect DB/API/browser evidence
- Can run regression/E2E
- Can use Reviewer agent without turning review into redesign

PART X - Event-Day Checklists and Cheat Sheets

80. Challenge intake checklist

- All members read official statement
- Official clarifications captured
- Users/actors identified
- Required workflows identified
- Inputs/outputs identified
- Frontend/backend/persistence responsibilities identified
- Business invariants identified
- Explicit vs inferred vs optional separated
- MVP approved
- Golden Path approved

81. plan.md readiness checklist

- Architecture understandable by all members
- Entities/relationships sufficient for MVP
- Critical invariants explicit
- Golden-Path API contracts design-stable
- Frontend screens/state flow identified
- Workstreams capability-oriented
- Dependencies visible
- Owners assigned
- Verification strategy present
- Release criteria understood

82. Workstream readiness checklist

- Objective bounded
- Golden Path relevance known
- Dependencies satisfied or preparation-only work understood
- Approved contracts linked
- Owner(s) known
- Out-of-scope explicit
- Builder plan reviewed
- Tests/exit criteria known
- Branch created from current main

83. Frontend/backend rendezvous checklist

- Backend endpoint implementation-stable
- Frontend contract-compatible UI/client ready
- Mocks clearly identified
- Both branches synchronized with required main
- Real API base URL configured
- Network request inspected

- Response/status matches contract
- Server logs clean
- DB state correct
- UI state/rerender correct
- Mock removed
- QA integrated test passes

84. Merge checklist

- No unauthorized shared-design changes
- Focused tests pass
- Integration done where required
- QA evidence adequate for risk
- No secrets/temp debug artifacts
- Migration impact reconciled
- Conflicts resolved and retested
- execute.md evidence updated
- PR/merge note identifies contract changes
- main remains runnable

85. Final 4-hour checklist

- Critical Golden Path assembled
- Feature Freeze active
- Systematic integration pass complete
- Regression set passes
- Local E2E passes
- Release/deployment decision made
- If deployed: deployed E2E passes
- Final Reviewer complete
- P0/P1 addressed
- README matches actual implementation
- Whole-project reconstruction done
- Demo rehearsed
- Source checkpoint/push confirmed
- Demo Freeze active

86. Common failure modes and prevention

Failure	Why it happens	Prevention
Everyone starts coding immediately	requirements/design feel like delay	time-box intake; explain cost of wrong parallel work
Four agents create four architectures	no shared authority	one problem.md/plan.md; stop-escalate rule
Frontend waits for all backend	contract not treated as design artifact	design-stable contract + mocks
Mocks survive until demo	integration delayed	rendezvous as soon as endpoint stable
Silent API changes	local convenience	NEED CONTRACT DECISION protocol
Long-lived layer branches	ownership by layer	short workstream branches + incremental merge
QA starts at end	testing viewed as phase	testability during intake; QA matrix early
"My part is done"	completion defined locally	workstream DoD includes

		integration/evidence
Migration conflicts	multiple generators	single migration owner/reconciliation
Late optional features break core	scope drift	Golden Path relevance + Feature Freeze
Deployment before local E2E	demo pressure	local E2E release gate

87. Judge-readiness questions

- What exact problem are you solving and for whom?
- What is your Golden Path?
- Why did you choose this architecture?
- Show one frontend request all the way to the database and back.
- What are your most important business invariants?
- How do you prevent invalid or concurrent state changes?
- How did frontend and backend work in parallel?
- How did you test negative cases?
- What did QA/Reviewer find?
- What would you improve with another day?
- Which features were intentionally deferred and why?
- How do you know the demo result is persisted rather than mocked?

88. The causal chain - why the process works

problem.md prevents building the wrong product
 Golden Path prevents uncontrolled scope
 plan.md prevents fragmented architecture
 API contracts enable safe parallelism
 workstreams create bounded ownership
 Builder agents accelerate bounded execution
 human gates protect material decisions
 short branches isolate parallel work
 incremental merging exposes integration failures early
 shift-left QA finds ambiguity and defects earlier
 Reviewer agents add independent risk checks
 Golden Path assembly proves system cohesion
 E2E proves real user value and persistence
 Feature/Demo Freeze protect the known-good state
 final reconstruction makes every member able to explain the product

89. Final operating rules - print this page

1

One challenge interpretation. One problem.md. One Golden Path. One plan.md. One main.

2

Parallelize implementation, not architecture.

3

Frontend waits for contract clarity, not backend completion.

4

Every workstream has an accountable owner and explicit exit criteria.

5

Agents are bounded. Material design changes stop and escalate.

6

Everyone verifies their own work; QA independently challenges critical behavior.

7

Merge small and often. main is the best-known integrated state.

8

When the Golden Path is blocked, swarm the blocker.

9

Cut optional scope before cutting core verification.

10

Near the end, stop thinking in individual features and verify one complete system.

APPENDIX A - Copy-Ready Templates

A1. problem.md template

```
# Problem
## Product purpose
## Users / actors
## Required workflows
## Inputs / outputs
## Frontend requirements
## Backend requirements
## Persistence requirements
## Business rules / invariants
## Constraints / official clarifications
## Assumptions
## Acceptance criteria
## MVP boundary
## Explicit requirements
## Logically inferred requirements
## Optional / deferred ideas
```

A2. plan.md template

```
# Master System Design
## Golden Path
## Architecture
## Frontend
## Backend
## Database / entities / relationships
## Business invariants
## API contracts
## Workstreams
## Dependencies
## Ownership
## Integration strategy
## QA / test strategy
## Release criteria
## Major decisions / tradeoffs
```

A3. execute.md workstream template

```
## WS-XX - <Capability>
Primary Owner:
Frontend Owner:
Verification Owner:
Integration Owner:
Golden Path Relevance:
```


Dependencies:

Approved Contract(s):

Status: PLANNED / READY / IN PROGRESS / BLOCKED / VERIFYING / COMPLETE

Backend:

Frontend:

Persistence:

Integration:

Verification:

Evidence:

Blocker:

Need Decision:

Next:

Deferrals:

A4. review.md finding template

Finding <ID>

Category: BUG / DESIGN ISSUE / CONTRACT DRIFT / INTEGRATION FAILURE / MISSING VERIFICATION / DOC DRIFT / DEFERRED / IMPROVEMENT

Severity: P0 / P1 / P2

Workstream:

Evidence:

Impact:

Recommended action:

Status:

Resolution evidence:

Verdict impact:

APPENDIX B - Glossary

Term	Meaning
Acceptance criteria	Observable conditions that prove a requirement is satisfied.
API contract	Agreed client/server interface: method, path, request, response, errors and semantics.
Builder	Agent role that plans and implements one bounded workstream.
Contract drift	Implementation differs from approved contract without an approved design change.
E2E	End-to-end verification through the real user-facing flow and system components.
Golden Path	Most important successful user journey proving MVP value.
Invariant	Rule that must remain true across operations, including failures/concurrency.
MVP	Smallest useful product scope that satisfies core challenge value.
Reviewer	Independent verification role that reports findings and verdict.
Supervisor	Control role for requirements, architecture supervision, teaching and major decisions.
Vertical slice	One capability implemented through only the layers it needs.
Workstream	Bounded engineering objective with owner, dependencies and exit criteria.

Closing note

The framework is deliberately strict about shared truth and integration, but flexible about implementation details. A 24-hour hackathon rewards speed, yet uncontrolled speed creates incompatible pieces. The purpose of this system is to spend a small amount of time making the important agreements explicit so that humans and agents can move quickly in parallel without losing coherence. The end goal is not maximum code volume. It is a working, tested, explainable product whose Golden Path survives the real demo.

PART XX - Deep Event-Day Runbooks, Recovery Playbooks, and Printable Operations

This final layer expands the core workflow into execution-grade runbooks: first-three-hours command sequence, role operating manuals, handoffs, Git/migration conflict handling, QA/debug escalation, three-member capacity strategy, prompt library, templates, failure recovery, final war room, and cheat sheets.

PART XI - FIRST-THREE-HOURS COMMAND SHEET AND STARTUP SEQUENCE

The first three hours determine whether the remaining twenty-one hours are spent building one coherent product or repairing four different interpretations. This section is intentionally operational: it tells the team what to do, who speaks, what must be written down, what can start in parallel, and what must not start yet.

Operating rule

No member starts challenge-specific implementation merely because they have free hands. The team first creates a shared product truth and enough design stability to make parallel work safe.

65. Minute 0-15: Rules, Environment, and Evidence Capture

The team begins by separating official authority from assumptions. The Technical Lead opens the official rulebook, challenge statement, organizer clarifications, and submission requirements. Member B confirms backend/runtime constraints, Member C confirms required user-facing platform constraints, and Member D creates a short list of testable submission obligations. This is not bureaucracy: a technically excellent project can still fail if it violates a format, platform, runtime, or submission rule.

Owner	Action	Output	Why it matters
A - Lead	Capture official rules and clarifications	Authority notes / challenge source	Prevents building against an invalid assumption
B - Backend	Check runtime, DB, networking, provided starter constraints	Environment risk notes	Prevents late infrastructure surprises
C - Frontend	Check required web/mobile form, demo/browser constraints	UI/runtime notes	Prevents wrong client target
D - QA	Convert submission requirements into checks	Submission checklist	Makes compliance verifiable rather than remembered

- Confirm repository access for all members.
- Confirm every member can run the starter locally.
- Confirm Git identity and remote access.
- Confirm one team communication channel.
- Confirm where official clarifications will be recorded.

- Do not create feature branches yet unless the challenge has already been normalized.

66. Minute 15-45: Independent Read, Then Shared Reconstruction

Each member reads the problem independently before the lead explains it. Independent reading reduces anchoring: if the lead misunderstands one sentence, the entire team is less likely to inherit the same mistake. After the read, each person summarizes the product in one minute from their role perspective.

Member	One-minute answer
A	What product are we actually being asked to deliver, and what is the smallest credible outcome?
B	What state, invariants, data, and hard backend behavior appear unavoidable?
C	What user actions and screens must exist for the product to feel complete?
D	What claims will need proof, and what failure cases could invalidate the demo?

The shared Supervisor then performs Challenge-Intake Understanding Reconstruction. The output is explanation first, not code: actors, core workflow, inputs, outputs, business rules, persistence, edge cases, explicit requirements, inferred requirements, optional ideas, and risk. Any disagreement is labeled rather than silently resolved.

67. Minute 45-75: problem.md Approval Gate

The Technical Lead reconciles the challenge into one canonical problem.md. The team reviews it against the official statement line by line for mandatory behavior. This is the first formal human gate.

Approval questions:

- Can every member explain the product in plain language?
- Does every official requirement appear somewhere in problem.md?
- Are inferences clearly distinguished from explicit requirements?
- Are optional ideas clearly marked optional?
- Are important business invariants stated without prematurely choosing implementation mechanisms?
- Would a judge reading problem.md recognize the intended product?

Stop condition

If the team cannot agree on a requirement that materially changes architecture or MVP, do not hide the uncertainty. Record it as NEED CLARIFICATION / ASSUMPTION and make the safest reversible choice until organizer clarification is available.

68. Minute 75-100: MVP and Golden Path Lock

The MVP is not simply a list of features. It is the smallest useful product boundary that satisfies the challenge. The Golden Path is the one successful user journey that proves the MVP produces value. The team writes both explicitly because they become the reference for every later scope decision.

MVP: <smallest useful product boundary>

GOLDEN PATH:

1. <user begins from known state>

2. <user performs critical action>
3. <backend validates/processes>
4. <data changes persist>
5. <result becomes visible>
6. <reload/revisit still shows correct state>

Member D adds at least one important invalid-path companion to the Golden Path. A system that only works when every input is perfect is not yet trustworthy.

69. Minute 100-150: Master System Design Workshop

Now the team designs one architecture. Member A facilitates rather than dictating every decision.

Member B proposes data and backend boundaries, Member C proposes screens and client state, Member D challenges testability and failure behavior. The shared Supervisor helps identify hidden coupling and missing dependencies.

Design topic	Primary voice	Questions that must be answered
Entities/data	B	What persists? What relates? Which invariants belong in data/business logic?
API contracts	A+B+C	Which method/path/request/response/error behavior lets both sides work independently?
UI/user flow	C	Which screen owns each user action? How does loading/error/success appear?
Quality/risk	D	What would be costly to discover late? How will we prove it?
Dependencies/order	A	Which capability unlocks others? What can safely run in parallel?

The output is plan.md: the shared HOW. The team does not attempt to specify every private function. It specifies the architecture and contracts strongly enough that independent Builders cannot accidentally invent different systems.

70. Minute 150-180: Workstream Assignment and Agent Launch

Only after the design is stable enough does the team split. The lead creates dependency-aware workstreams and marks each READY or BLOCKED. Each member receives a bounded assignment, not their own architecture. Every member syncs the same main branch, creates a short-lived workstream branch, and starts their Builder Planning Phase.

At Hour 3	Member A	Member B	Member C	Member D
Primary action	Lead/integration foundation or first cross-cutting capability	Persistence/backend foundation	Frontend shell + contract-based first UI	QA matrix + contract/requirement checks
Agent mode	Supervisor + bounded Builder	Builder Plan Mode	Builder Plan Mode	QA/Test agent; Reviewer only

	if coding			when risk justifies
Must not do	Rewrite approved architecture silently	Change API/schema unilaterally	Invent response fields independently	Wait for a finished app before testing

71. First-Three-Hours Exit Criteria

- problem.md approved and committed.
- MVP and Golden Path written and understood by all members.
- plan.md contains architecture, entities, contracts, workstreams, dependencies, and verification strategy.
- execute.md has ownership and status for the first wave of workstreams.
- Shared API contracts for the first critical slices are design-stable.
- At least one person owns migrations/shared schema changes.
- Every active member knows their current branch and bounded scope.
- No one is maintaining a private competing project plan.

PART XII - DETAILED 4-MEMBER ROLE OPERATING MANUALS

The four roles below are primary ownership models, not permanent silos. Each role has a control loop: what it watches continuously, what artifacts it owns, what decisions it may make locally, when it must escalate, and how it hands work to the next person.

72. Member A - Technical Lead / System Integrator Operating Manual

Member A is the convergence owner. The lead may write code, but their irreplaceable job is to prevent parallel speed from becoming parallel divergence. A lead who spends six uninterrupted hours inside one feature branch can accidentally leave the team without architecture supervision.

72.1 Continuous lead dashboard

Signal	Question	Action if unhealthy
Golden Path	What exact step is not executable yet?	Move capacity toward that blocker or cut optional scope
Dependency graph	Which READY capability unlocks the most downstream work?	Prioritize/assign it
Contracts	Did any request/response/schema change?	Run shared-decision protocol immediately
main	Is it runnable and better than the previous checkpoint?	Stop merges until restored
Team load	Who is blocked, idle, overloaded, or duplicating work?	Reassign/swarm
Clock	Are we still on the planned phase?	Reduce scope before reducing core verification

72.2 Lead rhythm during implementation

7. Check execute.md and branch status every 45-90 minutes.
8. Review Level-2/Level-3 decisions as they arise.
9. Coordinate merge order around dependencies, not personality or arrival time.
10. Pull and run main at meaningful checkpoints.
11. Trigger rendezvous when both sides of a slice are sufficiently ready.
12. Prevent the team from opening optional workstreams while a critical Golden-Path step is blocked.
13. Protect Feature Freeze and Demo Freeze.

72.3 Lead anti-patterns

- Becoming the sole coder and neglecting integration.
- Rewriting teammates work instead of establishing contracts.
- Allowing every disagreement to become a full-team meeting.
- Approving architecture changes through chat but failing to update plan.md.
- Waiting until the final hours to run main personally.

73. Member B - Backend / Data / Core Logic Operating Manual

Member B owns server-side correctness and usually the migration lane. The backend engineer is not simply an endpoint factory. They translate business invariants into authoritative server behavior and durable state transitions.

73.1 Before implementing an endpoint

- Confirm the endpoint belongs to an approved workstream.
- Confirm method/path/request/response/error expectations in plan.md.
- Identify authoritative business rules.
- Identify transaction boundary and persistence effects.
- Identify concurrency, retry, idempotency, or ordering risks if relevant.
- Identify which tests can prove the invariant independently of the UI.

73.2 Backend completion packet

WORKSTREAM: WS-XX
BACKEND STATUS: READY FOR INTEGRATION
CONTRACT: unchanged | changed with approved decision D-XX
MIGRATION: none | <revision>
TESTS: <command + result>
MANUAL CHECK: <endpoint / docs evidence>
KNOWN LIMITATION: <none or explicit>
NEXT CONSUMER: frontend / integration / QA

This packet lets Member C and Member D consume the implementation without needing a verbal reconstruction of every commit.

74. Member C - Frontend / UX / Client Integration Operating Manual

Member C owns the user-facing proof that the system is usable. They work from design-stable contracts and user journeys, not from guesses about whatever backend code happens to exist today.

74.1 Contract-first frontend loop

14. Read Golden Path step and approved contract.
15. Build the UI state machine: idle, input, loading, success, failure.
16. Implement the API-client boundary with one known request/response shape.
17. Use a narrow mock/stub only if the real endpoint is unavailable.
18. Tag the mock clearly so it cannot be mistaken for integrated completion.
19. When backend becomes implementation-stable, sync main and replace the mock.
20. Use browser Network + visible UI + server evidence to verify the real path.

74.2 Frontend completion packet

WORKSTREAM: WS-XX
FRONTEND STATUS: READY FOR RENDEZVOUS
SCREEN/COMPONENT: <name>
CONTRACT EXPECTED: <method path + version/decision>
MOCKS REMAINING: yes/no
LOADING/ERROR/SUCCESS STATES: verified
BROWSER CHECK: <what was observed>
BLOCKER: <none or dependency>

75. Member D - QA / Integration / Reliability / Utility Operating Manual

Member D owns independent confidence, not all quality. They start before implementation by turning requirements and contracts into testable claims, then move toward the areas with highest consequence and highest uncertainty.

75.1 QA queue priority

21. P0 Golden-Path blockers and data corruption risks.
22. New or changed shared contracts.
23. High-risk state transitions, concurrency, retry, authorization, or persistence.
24. Newly integrated vertical slices.
25. Regression on previously working Golden-Path steps.
26. Optional feature polish only after critical confidence is established.

75.2 QA evidence packet

CAPABILITY: WS-XX
BUILD/COMMIT TESTED: <sha or branch>
ENVIRONMENT: local/deployed
HAPPY PATH: PASS/FAIL
NEGATIVE/BORDER CASES: <results>
PERSISTENCE: PASS/FAIL/N-A
INTEGRATION: PASS/FAIL/N-A
REGRESSION IMPACT: <none / details>
FINDINGS: P0/P1/P2
VERDICT: PASS / PASS WITH NON-BLOCKING FINDINGS / BLOCKED

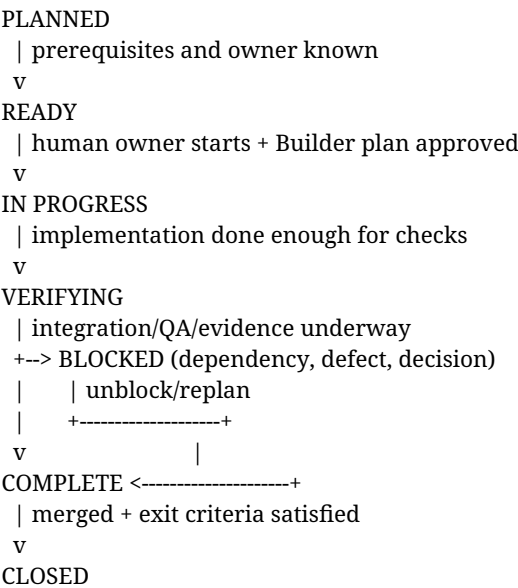
76. Role-to-Role Handoff Map

From	To	Handoff object	Receiver checks first
A Lead	B Backend	Approved workstream + contract + dependencies	Is scope READY and contract design-stable?
A Lead	C Frontend	Golden Path step + UI responsibility + contract	Can UI proceed with real API or temporary contract-faithful mock?
B Backend	C Frontend	Implementation-stable endpoint + test evidence	Does actual response match plan.md?
B/C	D QA	Integrated or testable capability + commit evidence	Which claims are highest risk?
D QA	A Lead	Findings + severity + reproduction	Fix, defer, replan, or release?
A Lead	Whole team	Merged main checkpoint	What Golden Path step is now executable?

PART XIII - SYNCHRONIZATION, HANDOFF, RENDEZVOUS, AND COMPLETION-TIME PROTOCOLS

A multi-member hackathon rarely proceeds at equal speed. The team therefore needs explicit synchronization rules. The goal is to avoid idle waiting without allowing dependent work to drift.

77. Workstream State-Transition Protocol



A frontend track may be COMPLETE while its parent workstream remains IN PROGRESS because backend or integration is not ready. Track-level completion is not workstream completion.

78. Completion-Time Matrix: What Each Person Does Next

Situation	Backend	Frontend	QA	Lead
Backend finishes first	Merge verified server work; move to next READY dependency	Sync endpoint when ready; continue UI	Start API/persistence verification	Confirm contract, merge order, next dependency
Frontend finishes first	Continue backend	Finish states, mark mock, move to READY task	Verify UI logic/mock assumptions	Prevent fake-data path from being mistaken as integrated
QA test design finishes first	Expose endpoint as soon as testable	Expose UI as soon as testable	Test incrementally; prepare regression	Use QA findings to refine risk order
One member blocked >20-30 min	Share evidence, not vague status	Assist if cross-boundary	Reproduce independently	Swarm/reassign/replan
Critical slice ready on both sides	Freeze contract	Remove mock/connect real API	Run focused integration checks	Coordinate rendezvous and merge
Optional task	Move to critical	Move to critical	Prioritize blocker	Do not open

finishes while critical blocked	support	support		another optional feature
---------------------------------	---------	---------	--	--------------------------

79. The Frontend/Backend Rendezvous Runbook

A rendezvous is a short synchronization event for one capability, not a general meeting. It occurs when the backend contract is implemented enough to be consumed and the frontend path is ready enough to use it.

27. Backend owner posts implementation-stable contract evidence and any approved decision IDs.
28. Frontend owner updates their branch from main/merge-ready backend state.
29. Frontend removes or bypasses the temporary mock for this path.
30. Run the real request from browser to backend.
31. Inspect browser Network request/response.
32. Inspect backend log/service behavior.
33. Inspect database state where relevant.
34. Compare actual request/response with plan.md.
35. Fix contract drift or integration defects in the owning layer.
36. QA independently reruns the happy path and the most important negative path.
37. Record evidence and mark Integration/Verification complete.
38. Merge remaining work and close the workstream only when exit criteria are met.

Rendezvous success

The capability is no longer two separately working halves. A real user action crosses the actual client/server boundary and produces the intended durable/visible outcome.

80. Handoff Message Templates

```

BACKEND -> FRONTEND
WS: WS-03
Endpoint: POST /transfers
Contract decision: D-04
Implementation status: stable for integration
Tests: 21 passed
Migration: rev_003 applied
Known limitation: none
Branch/merge: merged main @ <sha>

FRONTEND -> QA
WS: WS-03
Screen: TransferForm
Real backend connected: yes
Mock remaining: no
Happy-path manual check: pass
Error state implemented: yes
Commit: <sha>
Please prioritize: insufficient funds + duplicate submit

QA -> LEAD
WS: WS-03
Verdict: BLOCKED
Finding: duplicate submit creates two transfers
Severity: P0
Reproduction: <steps>
Evidence: <test/log/db state>

```

Suggested owner: backend
Golden Path impact: direct

81. Communication Without Constant Meetings

The team should communicate state, not stream-of-consciousness. A five-line structured update is more useful than fifteen minutes of narration. Sync meetings are for decisions and cross-boundary risks; routine progress belongs in `execute.md` and status messages.

WS: WS-04
STATUS: IN PROGRESS
DONE: request create endpoint + tests
BLOCKED: incoming list contract needs decision
CONTRACT CHANGE: proposed created_at field
NEED DECISION: D-07
NEXT: frontend rendezvous after decision

PART XIV - GIT, BRANCH, MERGE, MIGRATION, AND CONFLICT-RESOLUTION RUNBOOK

Git is the coordination substrate for parallel human and agent work. The purpose is not to imitate enterprise ceremony; it is to preserve a runnable integrated truth while allowing bounded work to move independently.

82. Repository Ground Rules

- main is the best-known integrated state, not a shared scratchpad.
- One workstream branch should contain one coherent capability or bounded correction.
- Agents must inspect git status/diff before reporting completion.
- Generated junk, secrets, local DB files, editor metadata, and unrelated changes must not enter commits.
- Shared design changes are committed before dependent branches continue when practical.
- No force-push to shared main.
- No destructive reset of another member's published work without explicit coordination.

83. Standard Branch Lifecycle

```
git checkout main
git pull --ff-only
git checkout -b feat/ws03-transfer-api

# work + focused tests
git status
git diff
git add <intentional-files>
git commit -m "implement WS-03 transfer API"
git fetch origin
git merge origin/main # or team-approved rebase strategy
# resolve on feature branch, rerun tests
git push -u origin feat/ws03-transfer-api
```

A team may use rebase instead of merge-to-update, but it should choose one habit before the event. During a hackathon, familiarity and recoverability matter more than history aesthetics.

84. Merge Readiness Gate

Check	Minimum evidence
Scope	Diff contains only approved workstream/supporting changes
Tests	Relevant automated/manual checks pass
Contract	No silent public API/schema drift
Migration	Revision order known and applied successfully if relevant
Integration	Required consumer path tested or explicitly marked pending
Docs/status	execute.md/evidence updated according to ownership policy
Secrets	No credentials, local env values, or private artifacts added

85. Conflict Resolution Protocol

39. Stop and identify whether the conflict is textual or semantic.
40. Pull the latest authoritative plan/decision before choosing a side.
41. For purely textual conflicts, preserve both intended changes and rerun tests.
42. For semantic conflicts involving API/schema/business rules, do not guess. Escalate to Level-2/Level-3 decision authority.
43. Resolve on the feature branch, not by experimenting directly on main.
44. Run the affected slice after resolution, not just unit tests.
45. Record a decision if the resolution changes shared behavior.

A merge tool telling you which lines conflict does not tell you which architecture is correct. Git can reveal collision; only the shared design and human owners can resolve meaning.

86. Migration Lane Protocol

Database migrations create ordering and merge risks, so one person - normally Member B - should act as migration coordinator. Other workstreams may request schema changes, but simultaneous independent migration histories should be minimized.

```
SCHEMA CHANGE REQUEST
Workstream: WS-05
Needed change: add request.status
Reason: approved request lifecycle requires durable state
Compatibility impact: existing rows need default PENDING
Requested by: C
Decision owner: A+B
Migration owner: B
```

- Before generating a migration, sync latest migration head.
- Avoid two unrelated branches creating migrations from the same old head when possible.
- If parallel migrations are unavoidable, explicitly reconcile/merge heads before release.
- Test both fresh database setup and upgrade path when feasible.

- Never manually edit production-like data as a substitute for a missing migration without documenting the exception.

87. When to Merge Partial Work

Partial work may merge when it is safe, non-misleading, and useful to downstream work. A stable backend endpoint can merge before frontend completion. A reusable frontend shell can merge before one API exists. A temporary fake-success path should generally not merge into main unless clearly gated and impossible to confuse with real behavior.

Partial item	Usually safe to merge?	Condition
Verified backend endpoint	Yes	Contract approved; tests pass
Frontend component shell	Often	Does not pretend integration is complete
Temporary mock data path	Usually no	Unless feature-gated/test-only and explicit
Migration needed by other workstreams	Yes, carefully	Migration owner + compatibility verified
Half-refactor across shared modules	No	Finish or revert; high merge risk

PART XV - QA, DEBUGGING, REVIEW, EVIDENCE, AND INCIDENT-RESPONSE RUNBOOK

Quality engineering in this workflow is continuous confidence-building. The team does not ask only "did the code run?" It asks whether the approved behavior is correct, reproducible, integrated, persistent, and still correct after later changes.

88. QA Test-Design Ladder

Layer	Question	Example
Requirement	What claim must be true?	A paid request cannot be paid twice
Equivalence	Which input classes behave similarly?	valid positive amount / invalid non-positive amount
Boundary	Where does behavior change?	balance exactly equal to amount
State	Which transitions are legal?	PENDING -> PAID allowed; PAID -> PAID rejected
Persistence	Does restart/reload preserve truth?	paid status remains after refresh
Integration	Do layers agree?	frontend field names match backend schema
Concurrency/retry	What if actions overlap/repeat?	double click / network retry
Regression	What old behavior can new change break?	new auth blocks existing transfer

89. Debugging Escalation Ladder

46. Reproduce with exact steps and known starting state.
47. Classify the failing layer: UI, Network/contract, backend validation, service/business logic, DB/persistence, environment/config.
48. Collect evidence before changing code.
49. Compare actual behavior against problem.md/plan.md, not against what one developer remembers.
50. Use the smallest owner capable of fixing it.
51. If local fix preserves design, fix/test locally.
52. If fix requires shared contract/schema/invariant change, stop and run decision protocol.
53. Rerun focused test plus one nearby regression.
54. Record evidence and close only when reproduction no longer fails.

DEBUG TRIAGE

Symptom: <visible failure>

Reproduction: <steps>

Expected: <contract/requirement>

Actual: <response/log/db/UI>

Likely layer: frontend | contract | backend | logic | DB | env

Owner: <member>

Shared-design change required: yes/no

Regression set after fix: <tests>

90. Browser-to-Database Trace

For a full-stack defect, trace the request without skipping layers: user event -> frontend state -> event handler -> API client -> browser Network -> HTTP route -> request validation -> service/use case -> business rule -> ORM/session -> database -> commit -> response serialization -> Network response -> client state -> rerender -> visible result. The first layer where actual behavior diverges from expected behavior is usually the best starting point.

91. Reviewer Trigger Matrix

Change/risk	Independent Reviewer?	Reason
Simple text/UI polish	Usually no	Low correctness consequence
New critical API contract	Often yes	Cross-team boundary
Money/resource invariant	Yes	High consequence if wrong
Concurrency/idempotency	Yes	Difficult failure modes
Migration affecting core entities	Often yes	Durable state risk
Small design-preserving bug fix	Maybe not	Owner tests may be sufficient
Pre-release whole project	Yes	Independent final confidence

92. Evidence Standard

Evidence should answer "why do we believe this is complete?" It can include automated test output, API response, browser Network evidence, DB row/state evidence, migration success, E2E steps, or Reviewer verdict. Evidence is lightweight but concrete.

Claim	Weak evidence	Stronger hackathon evidence
Endpoint works	"I tested it"	pytest output + example response
Persistence works	UI looked right	DB state + refresh/reload result
Contract matches	frontend compiles	real Network request/response matches plan.md
Concurrency handled	code has a lock	concurrent/retry test + invariant result
Deployment works	build succeeded	deployed Golden-Path E2E

93. P0/P1/P2 Operational Meaning

Severity	Meaning	Team action
P0	Blocks core Golden Path, corrupts critical state, security/integrity catastrophe, cannot submit/demo reliably	Stop optional work; swarm immediately
P1	Important incorrect behavior or major reliability/usability defect that should be fixed before release	Schedule before freeze/release; may interrupt current task
P2	Non-blocking issue, polish, minor docs drift, low-risk improvement	Fix only if time remains; document otherwise

PART XVI - DETAILED 3-MEMBER OPERATING MODEL AND REDUCED-CAPACITY STRATEGY

The three-member model is not the four-member process with QA deleted. It is a deliberate redistribution of ownership with more selective parallelism and stronger use of fresh Reviewer agents for independence.

94. Three-Member Responsibility Split

Responsibility	Member A - Lead/Integration/Backend	Member B - Backend/Data/Logic	Member C - Frontend/QA/E2E
Challenge/problem.md	Primary	Contribute invariants	Contribute user/testability
plan.md	Primary	Data/backend/contracts	UI/contracts/test design
Migrations	Backup	Primary	Request changes
Core backend	Shared critical/cross-cutting	Primary	API consumer/testing
Frontend	Integration support	Support when free	Primary
QA design	Risk prioritization	Backend tests	Primary manual/integration/E2E
Reviewer	Trigger/orchestrate	May request fresh reviewer	May run fresh reviewer
Merges/main	Primary	Prepare merge-ready branches	Prepare merge-ready branches

95. Three-Member Parallelism Rules

- Do not run three unrelated feature tracks if one critical dependency is unstable.
- Member A should keep at least 30-40% attention on system integration rather than disappearing into backend implementation.
- Member C begins QA design before frontend completion, but uses fresh Reviewer agents for high-risk backend logic to preserve independence.
- Member B should absorb backend support and testing work before starting optional server features.
- Cut optional features earlier than the four-member model because verification capacity is lower.

96. Three-Member 24-Hour Adjustment

Window	Primary focus	Capacity rule
0-3h	Same shared intake/design	All three collaborate; no premature split
3-7h	Foundation + first Golden-Path slice	A integration/backend, B data/core, C UI + QA matrix
7-13h	Critical capabilities	Only 2-3 active critical workstreams; merge frequently
13-16h	Golden Path completion	Defer optional scope aggressively
16-20h	Integration + regression + local E2E	C QA-heavy; A integration; B correctness/fixes
20-24h	Freeze/release/review/demo	All converge; no isolated feature ownership

97. Preserving Review Independence with Three People

When Member C helped implement a frontend capability, they should not be the only final authority on that same capability. For high-risk work, use a fresh Reviewer agent with repository context, then have another human confirm findings. Independence is about a fresh perspective and evidence, not a fourth job title.

98. Lead Overload Protection

- Member A owns decisions, not every implementation detail.
- Member B may directly approve Level-1 backend implementation choices.
- Member C may directly approve Level-1 frontend/test choices.
- Use structured decision requests instead of interrupting A with every question.
- If A becomes the critical coding bottleneck, reassign coding and return A to integration oversight.

PART XVII - EXPANDED AGENT PROMPT LIBRARY

These prompts are templates. Replace placeholders with the current workstream and repository evidence. The important property is role clarity: an agent is either supervising, building, reviewing, testing, integrating, or investigating - not doing everything without a boundary.

99. Team-Level Supervisor Startup Prompt

ROLE

You are the shared Control/Supervisor for a 24-hour multi-member hackathon team.

AUTHORITY

Official rules/clarifications > official challenge > approved problem.md > approved plan.md > actual code/tests/migrations/runtime evidence > execute.md/review.md/README.

TEAM MODEL

Four members by default: Lead/Integrator, Backend/Data, Frontend/Client, QA/Integration.

YOUR JOB

- reconstruct the challenge before implementation;
- help the humans create one problem.md, MVP, Golden Path, and one Master System Design;
- detect requirement ambiguity and architecture fragmentation;
- create dependency-aware capability workstreams;
- keep API/data contracts explicit;
- prepare bounded Builder/Reviewer prompts;
- supervise cross-workstream decisions and scope;
- prioritize Golden Path movement;
- supervise integration, E2E, release, and judge readiness.

DO NOT

Become the default repository Builder. Do not invent separate plans for members. Do not approve silent contract/schema/business-rule changes.

100. Backend Builder Workstream Prompt

ROLE: Backend Builder for WS-XX.

Read AGENTS.md, problem.md, plan.md, execute.md, relevant migrations/tests.

Reconstruct current backend/data state before proposing code.

OBJECTIVE: <capability>.

APPROVED CONTRACT: <method/path/request/response/errors>.

INVARIANTS: <list>.

DEPENDENCIES: <list>.

OUT OF SCOPE: frontend ownership, unrelated refactors, unapproved schema/contract changes.

FIRST RETURN A PLAN:

- files;
- data/migration impact;
- service/transaction boundary;
- tests including negative/concurrency cases if relevant;
- integration implications;
- merge-conflict risk.

STOP BEFORE IMPLEMENTATION.

AFTER APPROVAL: implement, test, debug, record evidence. Stop and escalate material shared-design changes.

101. Frontend Builder Workstream Prompt

ROLE: Frontend Builder for WS-XX.

Read problem.md, plan.md, execute.md, approved API contract, current API client and UI patterns.

OBJECTIVE: <user-facing capability>.

CONTRACT: <method/path/request/response/errors>.

BACKEND STATUS: design-stable only | implementation-stable.

PLAN MUST COVER:

- components/pages;
- local/server state;
- loading/success/error states;
- API-client changes;
- temporary mock strategy if needed;
- exact mock removal/rendezvous step;
- frontend verification;
- integration risks.

STOP BEFORE IMPLEMENTATION.

Do not invent response fields or silently compensate for backend contract drift.

102. QA Test-Design Prompt

ROLE: QA/Test Designer for WS-XX.

Use problem.md and plan.md as expected behavior.

Return a risk-prioritized test matrix covering happy path, boundaries, negative inputs, state transitions, persistence, integration, concurrency/retry where relevant, and regression impact.

Separate tests that can run before integration from tests requiring the full slice.

Flag ambiguous requirements/contracts as NEED DECISION instead of guessing.

103. Contract-Drift Reviewer Prompt

Compare the approved contract in plan.md with actual backend schemas/routes/responses and frontend API-client/request usage for WS-XX.

Identify every mismatch.

Classify each as implementation drift, approved evolution, or ambiguous.

Do not change code. Return evidence, impact, owner, and whether a shared decision is required.

104. Integration Rendezvous Prompt

ROLE: Integration Supervisor for WS-XX.

Trace the real user action from frontend event through API client, browser HTTP, backend route/validation/service,

persistence, response, and rerender.

The goal is to replace assumptions/mocks with real behavior.

Check contract adherence, configuration/CORS, error mapping, persistence, stale client state, and reload behavior.

Return: first divergence, reproduction, owner, smallest design-preserving fix, and regression checks. Do not add unrelated features.

105. Blocker Investigation Prompt

ROLE: bounded investigator, not automatic fixer.

BLOCKER: <symptom>.

WORKSTREAM: WS-XX.

EXPECTED: <requirement/contract>.

REPRODUCTION: <steps>.

Read relevant code/tests/log evidence.

Return hypotheses ranked by evidence, likely failing layer, minimal experiments to distinguish them, and whether any fix would change shared design. Stop before broad refactor.

106. Final Whole-Repository Reviewer Prompt

Review the integrated main branch as an independent release Reviewer. Prioritize Golden Path correctness, business invariants, data integrity, API contract consistency, critical negative paths, migration/runtime correctness, deployment if used, regression evidence, and misleading documentation.

Classify findings P0/P1/P2 and return PASS / PASS WITH NON-BLOCKING FINDINGS / BLOCKED. Avoid style-only suggestions unless they affect correctness, maintainability, or judge explanation.

PART XVIII - COPY-READY REPOSITORY ARTIFACT TEMPLATES

These templates make the workflow executable under time pressure. They are intentionally lightweight: enough structure to synchronize humans and agents without turning the repository into documentation work.

107. problem.md Template

```
# Problem
## Plain-English Product
## Users / Actors
## Required User Workflows
## Inputs and Outputs
## Frontend Requirements
## Backend Requirements
## Data / Persistence Requirements
## Business Rules and Invariants
## Constraints
## Explicit Requirements
## Logically Inferred Requirements
## Assumptions / Clarifications Needed
## Optional Ideas (Not MVP Commitments)
## Acceptance Criteria
## MVP Boundary
```

108. plan.md Template

```
# Master System Design
## MVP and Golden Path
## Architecture Overview
## Frontend Structure
## Backend Structure
## Entities / Relationships
## Business Invariants
## API Contracts
### <METHOD /path>
Request:
Response:
Errors:
Consumer:
## Business / Decision Logic
## Workstream Map
## Dependency Graph
## Ownership
## Integration Strategy
## Verification Strategy
## Release / Deployment Strategy
## Major Decisions
```

109. execute.md Workstream Template

```
## WS-XX - <Capability>
Golden Path Relevance: CRITICAL | SUPPORTING | OPTIONAL
Primary Owner:
Frontend Owner:
Verification Owner:
Integration Owner:
```

Dependencies:

Approved Contracts:

Status: PLANNED | READY | IN PROGRESS | BLOCKED | VERIFYING | COMPLETE

Backend:

- []

Frontend:

- []

Persistence:

- []

Integration:

- []

Verification:

- []

Evidence:

Blocker:

Need Decision:

Next:

Exit Criteria:

110. review.md Finding Template

Finding R-XX

Workstream / Scope:

Category: BUG | DESIGN ISSUE | CONTRACT DRIFT | INTEGRATION FAILURE | MISSING VERIFICATION | DOC DRIFT | DEFERRED | IMPROVEMENT

Severity: P0 | P1 | P2

Evidence:

Expected:

Actual:

Impact:

Owner:

Resolution / Deferral:

Verification after fix:

Status:

111. API Contract Template

<Capability>

Method + Path:

Authentication:

Request Headers:

Path Params:

Query Params:

Request Body:

Success Status:

Success Response:

Validation Errors:

Business Conflict Errors:

Not Found Behavior:

Idempotency / Retry Notes:

Frontend Consumer:

Persistence Effect:

Contract Decision ID:

112. Lightweight Decision Record

D-XX - <short decision>

Time:

Context:

Options considered:

Decision:

Reason:

Affected workstreams:

Contract/schema/doc updates required:

Owner responsible for propagation:

PART XIX - FAILURE-MODE AND RECOVERY PLAYBOOKS

The workflow should tell the team what to do when reality deviates from the plan. Recovery quality is often more important than perfect planning because a hackathon guarantees surprises.

113. Backend Is Two Hours Late

- Do not let frontend invent a second contract.
- Frontend completes contract-faithful UI states and moves to another READY task.
- QA prepares tests and can test any already-live backend subparts.
- Lead asks whether the backend scope can be split into an earlier integration-safe subset.
- Swarm if the late endpoint is the current Golden-Path blocker.
- Cut optional backend behavior before cutting core correctness.

114. Frontend Is Two Hours Late

- Merge verified backend contracts so other work can proceed.
- Backend moves to next dependency-aware core capability.
- QA exercises APIs directly and builds regression evidence.
- Lead prevents backend from overbuilding features the UI cannot expose.
- If Golden Path is blocked by UI, swarm basic integration before polish.

115. Contract Was Wrong

55. Identify the approved requirement that makes the current contract insufficient.
56. Create NEED CONTRACT DECISION with impact.
57. Lead + affected owners approve the smallest coherent change.
58. Update plan.md/decision record first or in the same coordinated change.
59. Backend and frontend update against the new contract.
60. QA adds regression around the changed boundary.
61. Do not preserve backward compatibility unless it is genuinely needed; unnecessary dual contracts waste hackathon time.

116. main Is Broken After Merge

62. Stop additional merges.
63. Identify the last known good commit and the merge that introduced failure.

64. Reproduce on main.
65. Decide revert vs forward fix based on time/risk.
66. Restore runnable main before continuing unrelated merges.
67. Rerun Golden-Path smoke test.
68. Record the failure if it reveals a missing merge gate.

117. Two Builders Changed the Same Shared File

First decide whether both changes are legitimately required. If yes, appoint one human owner to reconcile on a branch with both contexts. If one change violates ownership or shared design, discard/rework it. Do not ask an agent to "merge both" without explaining semantic intent.

118. Database Migration Conflict

- Pause deployment/main migration until heads are understood.
- Have migration owner inspect latest revision graph.
- Rebase/regenerate or create a proper merge migration according to the migration tool.
- Verify fresh setup and upgrade path.
- Notify dependent branches of the canonical migration head.

119. QA Finds P0 Near Feature Freeze

Freeze all optional work immediately. Lead owns triage. Assign one fix owner, one reproducer/verifier, and one integration observer. If the fix requires architecture redesign, prefer the smallest safe behavior-preserving workaround that satisfies the core requirement, provided it remains explainable and does not falsify project claims.

120. Deployment Fails Late

- Do not destroy a working local runtime while chasing hosting.
- Determine whether deployment is mandatory.
- If optional and local demo is acceptable, preserve local release as fallback.
- If mandatory, reduce deployment variables: one known DB, explicit env, migrations, start command, CORS, frontend API URL.
- Run deployed Golden-Path E2E; a green build alone is not release proof.

121. One Member Becomes Unavailable

Cross-understanding and shared artifacts are contingency controls. Lead checks execute.md, branch, latest handoff packet, and workstream evidence. Another member reconstructs the bounded scope from repository truth rather than from memory. Critical work is reassigned; optional owned work may be deferred.

122. Team Is Behind at Hour 14

Keep	Reduce/Defer
Golden Path critical capabilities	Optional dashboards/analytics
Core invariants and persistence	Aesthetic polish beyond demo clarity
Real frontend/backend integration	Nice-to-have secondary workflows
Critical negative path	Large refactors
Regression on working core	Extra technology demonstrations

The rule is to reduce breadth before reducing confidence in the core.

PART XX - FINAL FOUR-HOUR WAR ROOM, DEMO, SUBMISSION, AND JUDGE-DEFENSE PROTOCOL

The final four hours are not simply the end of coding. The operating mode changes from feature throughput to release confidence. Everyone should know exactly which activities are allowed and which are too risky.

123. Hour 20-21: Feature Freeze and Defect Queue

- Stop optional capability development.
- Snapshot current main and verify startup commands.
- Run risk-based regression on all critical Golden-Path steps.
- Sort findings P0/P1/P2.
- Assign P0/P1 owners and explicit verification owner.
- Do not mix unrelated cleanup into critical fixes.

124. Hour 21-22: Release / Deployment Decision

Question	If YES	If NO
Deployment mandatory?	Deploy immediately using known plan	Evaluate value vs risk
Local E2E passing?	Proceed	Do not deploy yet; fix local system
Production config rehearsed?	Proceed cautiously	Deployment risk high
Enough buffer for deployed E2E?	Proceed	Prefer stable local release if allowed

Whichever runtime is chosen becomes the release candidate. From this point, all fixes must be justified against release confidence.

125. Hour 22-23: Final Independent Review and Documentation Reconciliation

69. Run final Reviewer against integrated main/release candidate.
70. Fix P0 and justified P1 findings.
71. Update README to actual features and actual run commands.
72. Reconcile execute.md and review.md; remove false COMPLETE claims.
73. Verify migrations, env template, seed/demo data, and startup instructions.
74. Tag or record the release commit.

126. Hour 23-23:30: Whole-Project Reconstruction

Each member explains the project from their role, while the lead checks that explanations agree. The point is not memorization; it is discovering understanding gaps while there is still time to inspect the repository.

Member	Must be able to explain
A	problem, MVP, Golden Path, architecture, workstream decisions, tradeoffs, integration, release
B	entities, transactions, business logic, invariants,

	migrations, concurrency/retry, API behavior
C	user journey, component/state flow, API client, loading/error handling, real integration
D	test strategy, highest risks, key failures found, regression, E2E evidence, release verdict

127. Hour 23:30-24: Demo Freeze Protocol

- No new features.
- No dependency upgrades.
- No broad refactors.
- No schema changes unless submission-blocking.
- Keep the demo dataset/known state reproducible.
- Run the exact demo once from clean start.
- Confirm source pushed and correct branch/commit submitted.
- Keep local fallback even if deployed demo is primary.
- Prepare one sentence for each known limitation rather than hiding it.

128. Demo Story Structure

75. Problem: one concise sentence about the real pain.
76. User: who experiences it.
77. Golden Path: perform the central workflow live.
78. Technical value: explain one or two non-trivial engineering choices.
79. Quality proof: mention how correctness/integration was verified.
80. Tradeoff: explain one deliberate scope choice.
81. Outcome: show persistent/visible final state.

129. Judge Question Bank

Question	Strong answer direction
Why this architecture?	Tie the answer to hackathon constraints, domain invariants, simplicity, and future extension - not fashion.
How do frontend and backend coordinate?	Explain design-stable contracts, independent implementation, rendezvous, and real integration.
How do you prevent conflicting balances/state?	Explain the specific invariant and actual transaction/concurrency mechanism used in this project.
What did AI agents do?	Explain bounded Builder/Reviewer roles and human approval of important decisions.
How do you know it works?	Point to focused tests, QA findings, integrated Golden-Path E2E, persistence/reload, and release evidence.
What would you build next?	Choose the highest-value deferred capability or hardening step, not a random technology.
What failed during development?	Describe one real integration/contract/test failure and how evidence led to the fix.

PART XXI - PRINTABLE AT-THE-TABLE CHEAT SHEETS

These pages are intentionally compressed. They are designed to remain visible during the event while the rest of the handbook serves as the deeper reference.

130. One-Page Team Operating Loop

CHALLENGE -> problem.md -> MVP -> GOLDEN PATH -> plan.md -> execute.md

FOR EACH READY WORKSTREAM:

assign owner -> sync main -> branch -> Builder plan -> human approve -> implement/test -> backend/frontend rendezvous if needed -> QA -> Reviewer if risk justifies -> merge -> evidence -> close

ALWAYS ASK:

1. What blocks the Golden Path?
2. Did any contract/schema/invariant change?
3. Is main runnable?
4. What is the next READY dependency?
5. Are we adding optional scope while core is blocked?

131. Member Quick Cards

Member	Always watching	Immediate escalation trigger
A Lead	Golden Path, dependencies, contracts, main, clock	architecture/API/schema/MVP/release change
B Backend	invariants, transactions, data, endpoint evidence	shared schema/contract/business-rule change
C Frontend	user flow, UI states, API client, mocks	contract insufficient/drift, backend dependency blocked
D QA	highest-risk claims, regression, integration evidence	P0/P1, ambiguous expected behavior

132. Decision Quick Guide

Question	Decision level
Can I rename a private helper?	Level 1 - local
Can I change endpoint response field?	Level 2 - shared
Can I add/change DB relationship used by other workstreams?	Level 2 - shared
Can I replace core architecture or database?	Level 3 - product
Can I cut an optional feature?	Lead/team scope decision; usually safe if Golden Path unaffected
Can I change Golden Path?	Level 3 - product

133. Integration Quick Guide

BEFORE RENDEZVOUS

- ☐ contract design-stable
- ☐ backend endpoint testable
- ☐ frontend consumer ready
- ☐ mock identified

RENDEZVOUS

- ☐ sync latest code
- ☐ remove/bypass mock

- ☐ real browser request
- ☐ compare Network with plan.md
- ☐ inspect backend/log/DB
- ☐ verify visible result
- ☐ verify error path
- ☐ QA independent check
- ☐ evidence + merge

134. Merge Quick Guide

BEFORE MERGE

- ☐ scope clean
- ☐ tests pass
- ☐ no silent contract/schema drift
- ☐ migrations coherent
- ☐ required integration checked
- ☐ no secrets/junk
- ☐ feature branch updated with main
- ☐ conflicts resolved and retested

AFTER MERGE

- ☐ pull/run main
- ☐ smoke Golden Path if affected
- ☐ update execute.md/evidence

135. QA Quick Guide

FOR EVERY CRITICAL CAPABILITY ASK:

- Happy path?
- Boundary?
- Negative/invalid?
- State transition?
- Persistence/reload?
- Frontend-backend contract?
- Retry/concurrency if relevant?
- Regression risk?
- Evidence?
- Verdict?

136. Hour-by-Hour Phase Card

Hours	Mode	Do not do
0-3	Understand + design + assign	blind feature coding
3-6	Foundation + first contact	isolated long-lived branches
6-13	Core slices + high-risk logic	silent contract changes
13-16	Complete Golden Path	open large optional features
16-20	Integrate + harden + E2E	treat layer tests as enough
20-22.5	Freeze + release/deploy	large refactors
22.5-24	Final review + docs + demo	risky last-minute changes

137. Final Submission Card

- Correct repository/branch/commit submitted.
- README run instructions tested from a clean terminal.
- Required credentials/demo data handled safely.

- Local fallback preserved.
- Golden Path rehearsed.
- Known limitations stated accurately.
- Every member can explain the project and their own work.
- No last-minute unpushed change exists only on one laptop.

138. Closing Principle

A strong 24-hour team does not win by maximizing simultaneous typing. It wins by converting uncertainty into shared decisions, shared decisions into bounded work, bounded work into integrated capabilities, and integrated capabilities into a verified Golden Path. Agents increase implementation throughput; human governance keeps that throughput pointed at one product.

FINAL APPENDIX - Suggested Team Training Sequence Before the Hackathon

A. Session 0 - Individual prerequisite catch-up (60-120 minutes each)

- Terminal/processes/ports/env/JSON
- Frontend/backend/database mental model
- HTTP methods/status codes
- Git branch/commit/merge
- Unit vs integration vs E2E

B. Session 1 - Shared stack walkthrough (2-3 hours)

82. Run the generic repo together.
83. Trace one GET and one POST end to end.
84. Inspect Swagger, browser Network, backend logs, and DB.
85. Change one API field intentionally and observe contract failure.
86. Apply/revert a migration.
87. Write a small QA matrix.

C. Session 2 - One complete mini workstream (3-4 hours)

88. Create problem.md for a tiny challenge.
89. Define MVP and Golden Path.
90. Create plan.md with one contract.
91. Split backend/frontend/QA roles.
92. Run separate Builder agents on bounded scopes.
93. Rendezvous on the real API.
94. QA verifies and record evidence.
95. Merge incrementally to main.
96. Run E2E and explain the system without the agents.

D. Session 3 - Failure drill (2 hours)

- 97. Introduce a wrong response field, a migration mismatch, and one broken business rule.
- 98. Make the team diagnose using evidence instead of guessing.
- 99. Practice escalation: local fix vs shared contract decision vs product decision.
- 100. Practice P0/P1 triage and a mock Feature Freeze.

E. Readiness standard

The team is ready when every person understands the shared architecture and Golden Path, can operate their own branch and agent safely, can identify where a failure sits in the full-stack flow, and can explain what their workstream changed and how it was verified.

Closing Principle

The framework is not valuable because it creates more documents. It is valuable because each artifact removes a specific source of uncertainty: requirements, scope, architecture, contracts, ownership, quality, integration, or release. The purpose of agents is to accelerate bounded engineering work while humans preserve one coherent product and remain capable of explaining it.

PART C

ULTIMATE 24-HOUR OPERATING WORKFLOW

You now have the framework identity, prerequisites, full-stack fundamentals, baseline stack rationale, repository mental model, and readiness material.

The following section is the event-day operating manual for the complete 4-member workflow, with a 3-member alternative.

CORE RULE: Parallelize implementation, not architecture.

24-HOUR HACKATHON

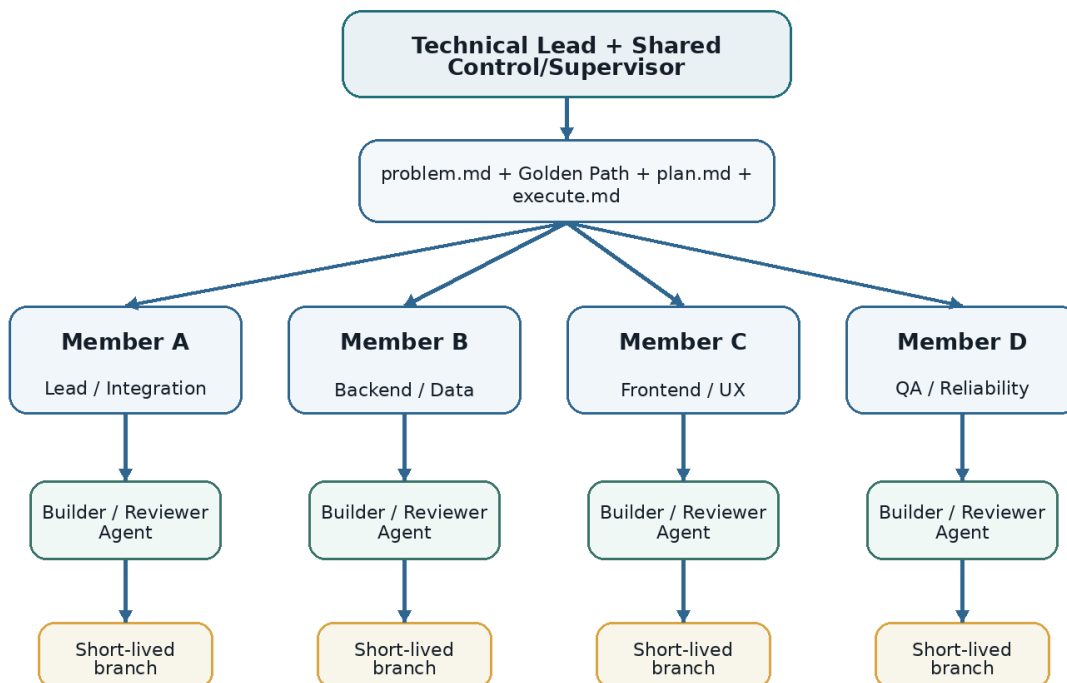
MULTI-MEMBER AGENTIC DEVELOPMENT WORKFLOW

Complete 4-Member Team System + 3-Member Alternative

Operating thesis

Parallelize implementation, not architecture. One product truth, one Master System Design, multiple bounded implementation loops, frequent integration, and one Golden Path that becomes demonstrably real.

One Shared Design, Multiple Bounded Human-Agent Loops



Definitive operating manual for a human-governed, multi-agent 24-hour hackathon team - expanded execution edition.

How to Use This Handbook

This handbook is an operating system for a 24-hour software hackathon, not merely a role chart. It describes how a team receives an unknown challenge, establishes one shared interpretation, turns it into an approved product and system design, then runs several human-supervised coding-agent loops in parallel without allowing the architecture to fragment. The workflow continues through vertical-slice integration, QA, incremental merging, Golden-Path end-to-end proof, release, final review, documentation reconciliation, and judge readiness.

The primary model assumes four members: a Technical Lead/System Integrator, a Backend/Data/Core Logic Engineer, a Frontend/UX/Client Integration Engineer, and a QA/Integration/Reliability Engineer. A complete three-member alternative appears later, with explicit redistribution of responsibilities rather than pretending the fourth role disappears. This expanded execution edition also includes minute-by-minute startup guidance, role operating manuals, completion-time handoffs, merge/conflict/migration protocols, debugging and recovery playbooks, copy-ready repository templates, final-four-hour release procedures, and printable quick-reference cards.

The mental model

The repository is the shared memory. `problem.md` defines WHAT. `plan.md` defines HOW. `execute.md` tells the team what is happening now. Git branches isolate bounded work. Contracts let independent tracks move safely. `main` remains the best-known integrated product.

Contents

- Part I - Process model, principles, and team roles
- Part II - From official challenge to one shared `problem.md`, MVP, Golden Path, and `plan.md`
- Part III - Contracts, workstreams, parallel frontend/backend implementation, and different completion times
- Part IV - Builder agents, supervisors, decision authority, Git, branches, and incremental merging
- Part V - Shift-left QA, Reviewer agents, evidence, swarming, regression, and E2E
- Part VI - Recommended 24-hour timeline, rest, freeze, release, and final review
- Part VII - Full 4-member case study
- Part VIII - Full 3-member adaptation
- Part IX - Prompt library
- Part X - Checklists, failure modes, analogies, and judge-readiness questions
 - Part XI - First-three-hours command sheet and startup sequence
 - Part XII - Detailed 4-member role operating manuals
 - Part XIII - Synchronization, handoff, rendezvous, and completion-time protocols
 - Part XIV - Git, branch, merge, migration, and conflict-resolution runbook
 - Part XV - QA, debugging, review, evidence, and incident-response runbook
 - Part XVI - Detailed 3-member operating model and reduced-capacity strategy
 - Part XVII - Expanded Supervisor, Builder, Reviewer, QA, integration, and emergency prompt library
 - Part XVIII - Copy-ready repository artifact templates and status boards
 - Part XIX - Failure-mode and recovery playbooks
 - Part XX - Final four-hour war room, demo, submission, and judge-defense protocol
 - Part XXI - Printable at-the-table cheat sheets and quick decision guides

PART I - THE PROCESS MODEL AND TEAM

1. Core Process Model

The methodology is best described as a human-governed hybrid iterative-incremental software process with Golden-Path-driven scope control, capability-oriented vertical-slice delivery, shared-contract-based parallel implementation, human-gated important decisions, risk-driven review, evidence-based verification, and incremental integration.

It borrows useful ideas from several established disciplines without pretending to be a textbook implementation of any one of them. From Agile and iterative development it takes short feedback loops and progressive delivery. From stage-gated systems it takes explicit human approval before material architectural decisions. From Spiral-style thinking it takes risk-first prioritization. From shift-left QA it takes early testability and risk analysis. From DevOps it takes frequent integration, reproducible runtime setup, and release verification. From vertical slicing it takes the idea that a capability should become real across only the layers it actually needs.

WHAT	An end-to-end process that keeps human judgment over product and architecture while using agents to accelerate bounded implementation.
WHY	A 24-hour event is long enough for parallel engineering but short enough that architectural divergence, late integration, or uncontrolled scope can destroy the submission.
ANALOGY	A film crew can shoot scenes in parallel only after agreeing on the script, characters, visual language, and final story. More cameras do not help if each crew is filming a different movie.
TECHNICAL REALITY	The shared artifacts define the product and interfaces. Each member receives bounded work. Agents implement locally. Verified increments merge to main. The whole system is repeatedly reassembled and tested.
TEAM RESPONSIBILITY	The Technical Lead protects shared decisions; domain owners execute and verify their capabilities; QA challenges risk; all humans remain accountable for agent output.
PROCESS CONNECTION	This model is the backbone for every later section: intake, design, parallelism, integration, release, and explanation.
IF SKIPPED	The team can become fast but incoherent: multiple architectures, incompatible APIs, long-lived branches, and a final-hour integration crisis.

2. The Non-Negotiable Principle: Parallelize Implementation, Not Architecture

The team should parallelize bounded implementation after establishing one shared product interpretation and one sufficiently stable system design. Four people and four coding agents must not become four architects inventing the application independently.

One shared truth

One challenge interpretation + one problem.md + one Golden Path + one plan.md + one execute.md + one repository + one integrated main branch. Local clones are copies of the same truth, not separate versions of the product.

Architecture can still evolve. The point is that material changes are coordinated. If a workstream discovers that the approved API contract, database relationship, invariant, or Golden Path is wrong, the agent stops, the human escalates, the affected owners decide, and the shared design is updated before implementation continues.

3. Primary 4-Member Team Model

Member	Primary ownership	Detailed responsibilities
A - Technical Lead / System Integrator	Whole-system ownership	Challenge interpretation; problem.md; MVP; Golden Path; plan.md; contracts; dependency map; integration; merge order; scope; release; major

Member	Primary ownership	Detailed responsibilities
		debugging; reconstruction; demo readiness.
B - Backend / Data / Core Logic	Authoritative server-side behavior	FastAPI; schemas; services; business rules; SQLAlchemy; migrations; transactions; concurrency/integrity; backend tests; migration coordination.
C - Frontend / UX / Client Integration	User-facing behavior	Screens; components; forms; state; API client; error/loading states; responsive UX; contract-based work; mocks; real API integration.
D - QA / Integration / Reliability	Independent confidence and failure discovery	Testability; test design; API/DB checks; negative paths; regression; bug reproduction; Reviewer orchestration; E2E; deployment verification; utility support.

Primary ownership creates accountability, not walls. The Backend Engineer does not own every line under backend/. The QA Engineer is not the dumping ground for broken features. The Frontend Engineer does not wait for a finished backend. The Lead does not become the only person allowed to think. Roles create a default home for decisions while the Golden Path remains a shared outcome.

4. Why Everyone Does Not Need to Be Coding at the Same Time

The objective is not maximum keyboard activity. The objective is maximum movement of the critical product path. During intake and Master System Design, all four people may be reading, questioning, diagramming, or deriving risks instead of coding. During execution, multiple tracks can run in parallel. When a critical blocker appears, the team may temporarily swarm one issue. When one person finishes early, they move to the next READY workstream, help integration, or reduce risk rather than creating optional scope for the sake of staying busy.

Optimization target

Do not optimize for “everyone busy.” Optimize for “the Golden Path is becoming executable and safer.”

5. 3-Member Alternative

Member	Combined role	Operating guidance
A	Technical Lead + Integration + Backend	Own architecture, contracts, merge/integration, and one critical backend lane. Avoid becoming the sole implementer.
B	Backend + Database + Core Business Logic	Own persistence, main server-side capabilities, migrations, high-risk correctness, and deep backend tests.
C	Frontend + QA + E2E	Own user-facing implementation plus test matrix, integration checks, regression, and E2E. Reviewer agents preserve some independence.

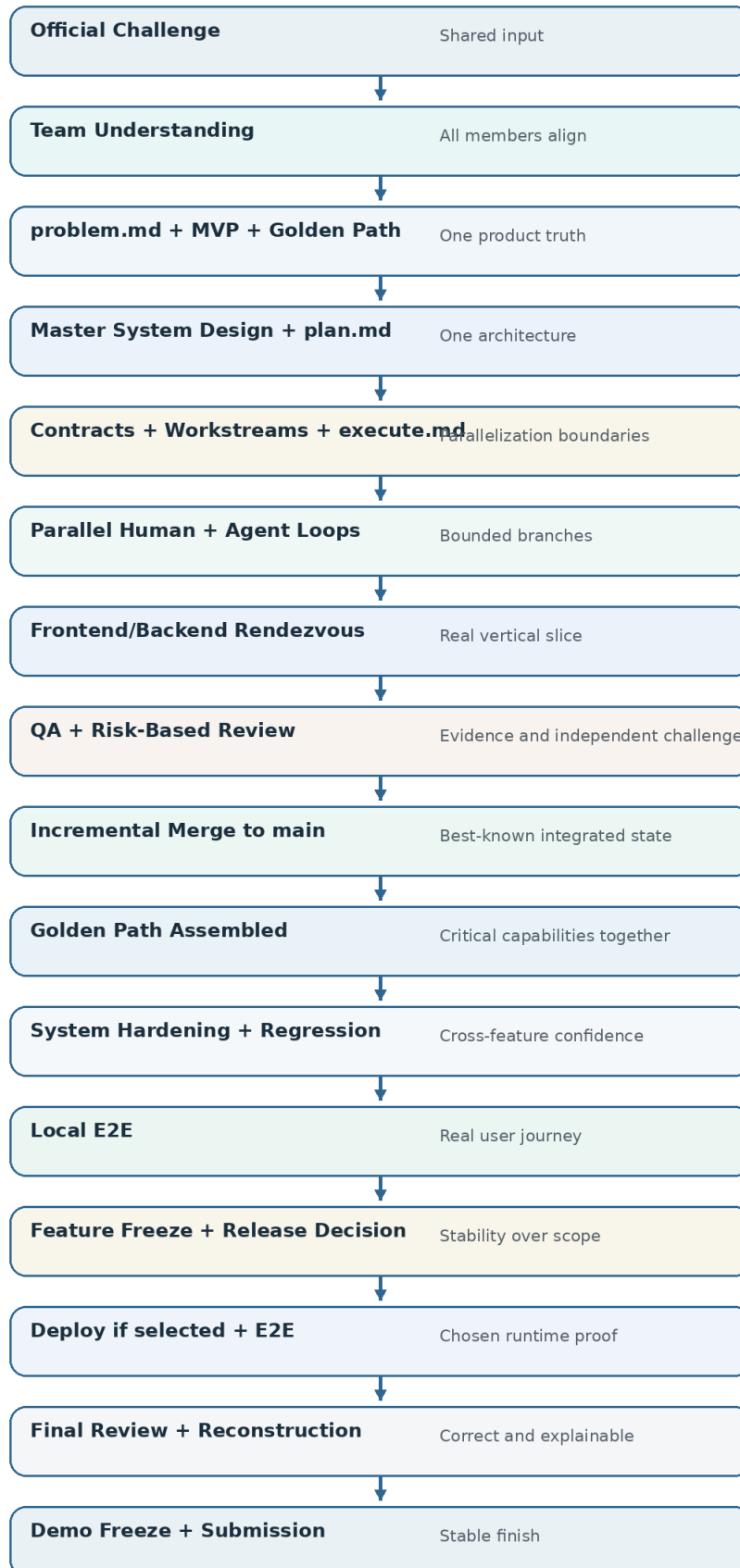
The three-member model reduces parallel capacity, so it should cut optional scope earlier. QA does not vanish; it is redistributed. Member C owns most integration-level verification, Members A and B must still test their own work, and fresh Reviewer agents are used selectively for high-risk backend logic or final release checks. The Lead should protect time for integration and decisions instead of taking every unowned feature.

6. 4-Member vs 3-Member Responsibility Matrix

Responsibility	4-member model	3-member model
Challenge intake	A leads; all participate	A leads; all participate
Master architecture	A owns; team reviews	A owns; B/C review
Backend foundation	B primary; A backup	B primary; A secondary

Responsibility	4-member model	3-member model
Core business logic	B primary; D risk-tests	B primary; A reviews
Frontend	C primary	C primary
QA test design	D primary; all contribute	C primary; A/B contribute
Independent review	D orchestrates fresh Reviewer	Fresh Reviewer initiated by C or A
Integration	A owns; C/D active	A owns; C active
E2E	D leads; A/C/B observe relevant layers	C leads; A/B observe relevant layers
Documentation/reconstruction	A coordinates; all explain	A coordinates; all explain

24-Hour Multi-Member Agentic Delivery Loop



PART II - FROM CHALLENGE TO ONE SHARED DESIGN

7. Beginning of the Hackathon: Do Not Code the First Interpretation

The event begins with coordinated understanding, not immediate implementation. Every member reads the official challenge because each role sees different failure modes: backend notices invariants and persistence implications, frontend sees user-flow ambiguity, QA sees testability gaps, and the Lead sees scope and dependency consequences.

```
Official Rules / Clarifications
-> Official Problem Statement
-> Every member reads
-> Team discussion
-> Challenge-Intake Understanding Reconstruction
-> proposed problem.md
-> team approval
-> MVP + Golden Path
-> team approval
-> Master System Design
-> plan.md
-> workstreams + owners + execute.md
-> implementation
```

8. Challenge-Intake Understanding Reconstruction

Before formalizing problem.md, the team reconstructs the challenge in plain language. This is deliberately educational and operational: the goal is to make sure the team can say what the system is supposed to accomplish before an agent starts producing code.

Reconstruction lens	Question to answer
Plain-English product	What is the product in one paragraph?
Purpose	Why does this product exist and what pain does it remove?
Users/actors	Who performs actions and who receives outcomes?
Core flow	What does a normal successful journey look like?
Analogy	What familiar real-world process helps explain the flow?
Frontend	What must the user see, enter, and understand?
Backend	What authoritative actions and rules must execute?
Persistence	What state must survive refresh/restart?
Inputs/outputs	What enters the system and what must be returned or displayed?
Rules/invariants	What must always remain true?
Hard parts	Where are concurrency, optimization, security, state transitions, or integration risks?
Requirement class	What is explicit, logically inferred, or merely optional?
MVP	What is the smallest useful product?
Golden Path	Which successful journey best proves that MVP?

Why it matters

A technically impressive implementation of the wrong interpretation is still a failed hackathon submission. This phase makes later architecture and testing traceable to an agreed problem.

9. problem.md - The Canonical WHAT

problem.md is the normalized statement of what the complete product must do. It is intentionally separated from implementation choices. The Lead reconciles edits, but the whole team approves the meaning. Each local Git clone has a copy of the same canonical file; members do not create role-specific versions with different requirements.

- Product scope and actors
- User workflows
- Frontend requirements
- Backend requirements
- Persistence/data requirements
- Inputs and outputs
- Business rules and invariants
- Constraints and assumptions
- Acceptance criteria
- MVP boundaries

Do not put premature implementation here

problem.md should say “a successful payment must update both balances atomically,” not “use PostgreSQL SELECT FOR UPDATE.” The invariant belongs to the WHAT; the mechanism belongs to plan.md.

10. MVP, Golden Path, Workstream, Vertical Slice, and E2E

Term	Meaning
problem.md	The complete normalized WHAT the challenge requires.
MVP	The smallest useful product scope that can credibly solve the core problem.
Golden Path	The most important successful user journey that proves the MVP has value.
Workstream	A bounded engineering objective/capability with ownership, dependencies, and exit criteria.
Vertical slice	The implementation shape of a capability across only the layers it needs.
Golden-Path E2E	A real end-to-end proof that the assembled critical journey works in the chosen runtime.

The Golden Path controls priorities. It influences which contracts are designed first, which workstreams become READY first, which integration points receive attention, what gets deferred when time slips, what QA attacks first, and what the team eventually demonstrates to judges.

11. Master System Design and plan.md - The Shared HOW

Once the problem, MVP, and Golden Path are approved, the team creates the Master System Design. plan.md is the persistent shared HOW. It should be detailed enough that several agents can work independently without inventing incompatible architecture, but not so detailed that the team spends the first six hours writing a specification instead of shipping software.

- Architecture and runtime boundaries
- Frontend structure and main screens
- Backend modules and responsibilities
- Database entities, relationships, constraints, and migration approach
- Business invariants and decision logic
- API contracts including success/error behavior
- Workstream map and dependencies
- Owners and integration responsibilities
- Testing/verification strategy
- Release/deployment strategy and decision criteria

One plan.md

Members receive different bounded prompts, not different Master System Designs. A local Builder Planning Phase is allowed to choose implementation details inside the approved workstream; it cannot silently overwrite the shared architecture.

12. Workstream and Dependency Map

Workstreams should represent capabilities or enabling foundations, not organizational departments. A workstream may cross backend, frontend, persistence, and QA. Some foundation workstreams are not user-facing vertical slices, but they exist only to enable later capabilities.

ID	Capability	Golden Path relevance	Dependencies
WS-01	Persistence Foundation	Supporting / prerequisite	None
WS-02	Authentication / User Entry	Critical or supporting	WS-01
WS-03	Core Create/Submit Capability	Critical	WS-01, WS-02
WS-04	Main Decision / Processing Capability	Critical	WS-03
WS-05	State Transition / Approval	Critical	WS-03, WS-04
WS-06	Dashboard / Results	Critical/supporting	relevant API contracts
WS-07	Integrity / Retry / Concurrency Hardening	Critical if domain requires	core service exists
WS-08	Golden Path Integration	Critical	all Golden-Path slices

Recommended execution states are PLANNED, READY, IN PROGRESS, BLOCKED, VERIFYING, and COMPLETE. A dependency can block real integration without blocking all preparation. For example, a frontend component can be built against an approved contract even while the backend endpoint is still under construction.

13. execute.md - The Live Execution Board

```

## WS-03 - Direct Transfer

Primary Owner: Member B
Frontend Owner: Member C
Verification Owner: Member D
Integration Owner: Member A

Golden Path Relevance: CRITICAL
Dependencies: WS-01
Approved Contract: POST /transfers
Status: IN PROGRESS

Backend:
- [ ] router/schema/service
- [ ] persistence/transaction

Frontend:
- [ ] form/state/API client

Integration:
- [ ] real UI -> real API

Verification:
- [ ] happy path
- [ ] important negative paths

Evidence:
Blocker: None
Next: ...

```

Artifact	Primary purpose	Typical lifecycle
problem.md	What the product must do	Stable after approval; changes only through deliberate requirement decisions

Artifact	Primary purpose	Typical lifecycle
plan.md	How the whole system is designed	Shared Master System Design
execute.md	What is happening now	Live workstream status, blockers, evidence, next actions
review.md	What independent review found	Verified findings and review history
README.md	How the actual project is understood/run	Project-facing truth reconciled near release

PART III - CONTRACTS AND PARALLEL FULL-STACK IMPLEMENTATION

14. API Contracts: The Safe Parallelization Boundary

Frontend and backend do not need to finish sequentially. They need an agreed interface. The API contract is that interface: method, path, request schema, response schema, important status codes, and any business-level error behavior that the client must understand.

14.1 Design-Stable vs Implementation-Stable

State	Meaning
Design-stable	Method/path/request/response/error expectations are agreed in plan.md. Enough for backend, frontend, and QA to start independently.
Implementation-stable	The real endpoint exists, focused tests pass, and runtime behavior matches the approved design. Enough for real integration.

```

POST /transfers

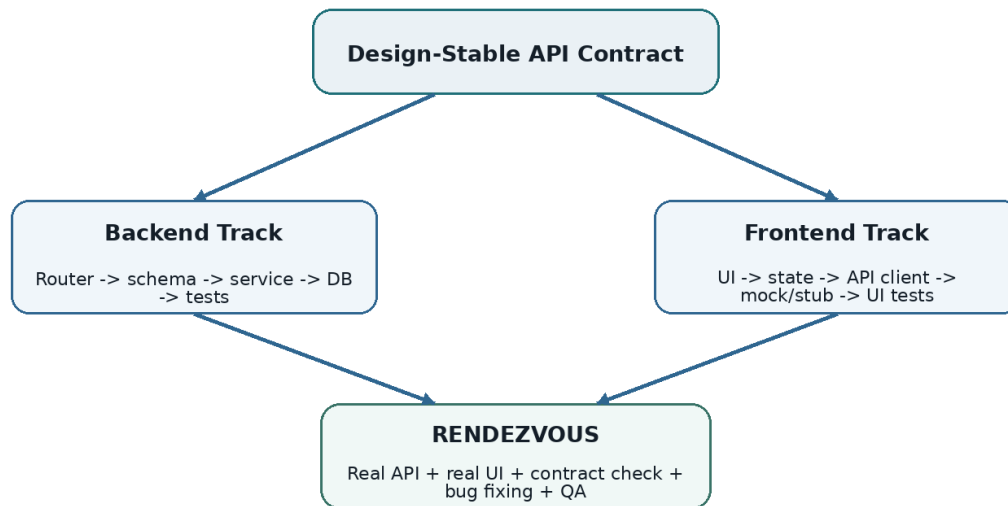
Request
{
  "recipient_id": 7,
  "amount": 2500
}

Response
{
  "id": 21,
  "recipient_id": 7,
  "amount": 2500,
  "status": "completed"
}

```

Backend implementation follows the contract toward authority and persistence: router -> request schema -> service/use case -> business rules -> database transaction -> response schema. Frontend implementation follows the same contract toward the user: form -> state -> event handler -> API client -> loading/success/error states -> visible result. QA derives the contract test surface before either side is fully complete.

Contract-Driven Parallelism and the Integration Rendezvous



15. How Frontend and Backend Work at the Same Time

Frontend waits for contract clarity, not for a complete backend. Once the contract is design-stable, the Backend Engineer can implement the real endpoint while the Frontend Engineer builds the client behavior against the expected shape. If the real API is not yet available, the frontend may use a narrow mock or stub that mirrors the approved contract.

```

async function createTransfer(data) {
  return {
    id: 21,
    recipient_id: data.recipient_id,
    amount: data.amount,
    status: "completed"
  };
}

```

Mocks are temporary scaffolding. They help the frontend implement state transitions and rendering without blocking on server timing, but they are dangerous if they become a substitute for integration. Replace them as soon as the real endpoint is sufficiently ready, then test the real path.

Analogy

Two bridge teams can build from opposite banks after agreeing on the exact connection geometry. The API contract is the meeting geometry. Implementation-stable means the concrete bridge ends actually reach that point.

16. Different Completion Times: Asynchrony Is Normal

16.1 Backend Finishes First

1. Backend owner runs focused endpoint/service/database tests and checks contract adherence.
2. A safe implementation is merged to main after integration review.
3. Frontend owner updates their branch from main and replaces the mock/stub with the real endpoint.
4. Real UI-to-API integration starts immediately.

- Backend owner moves to the next READY dependency-aware workstream or assists a critical blocker.

16.2 Frontend Finishes First

- Frontend completes form behavior, loading/error/success states, and mock-based checks.
- The workstream remains IN PROGRESS because real integration is not done.
- Frontend moves to another READY task, helps QA, or prepares integration rather than creating random optional features.
- When backend lands, frontend updates from main, removes the mock, and performs the real rendezvous.

16.3 QA Finishes Test Design First

QA can begin API checks as soon as an endpoint appears, inspect contract ambiguity, prepare database assertions, or design later regression/E2E scenarios. Test design finishing early is useful because it gives the team a ready definition of what must be proven when implementation arrives.

16.4 Both Sides Finish Together

They meet at the rendezvous: use the latest real endpoint, latest real frontend, and approved contract. Remove mocks, run the user action, observe network traffic and backend logs, fix drift, let QA independently challenge the integrated behavior, then merge/close.

Key point

Parallel development does not mean synchronized completion. People flow through the dependency graph. The workstream closes only when its relevant tracks have converged and the integrated behavior is verified.

17. Contract Drift vs Contract Evolution

Contract drift is an implementation defect. If plan.md says recipient_id, the frontend sends recipient_id, but the backend expects receiver_id, the implementation has drifted. A 422 response is evidence of mismatch; the correct fix is normally to restore the approved contract.

Contract evolution is different. If an approved user requirement genuinely requires created_at in the response and the design omitted it, the contract itself is insufficient. The affected owner raises NEED CONTRACT DECISION. The Lead and affected owners approve or reject the change, update plan.md, notify the team, and then change both implementations. No silent contract changes.

NEED CONTRACT DECISION

Current: TransferResponse has no created_at.

Problem: Approved activity UI requires transaction time.

Proposed: Add created_at to TransferResponse.

Affected: Backend response schema, frontend activity rendering, QA contract tests.

18. Workstream Rendezvous and Vertical-Slice Integration

A capability becomes a true vertical slice when the relevant user action crosses the real layers needed to produce durable behavior. Backend-only and frontend-only completion are partial tracks; integration is the point where the system proves that those tracks meet.

```
TransferForm
-> frontend state
-> API client
-> POST /transfers
-> FastAPI router
-> Pydantic validation
-> service/use case
-> business rules
-> SQLAlchemy/session
-> database
-> commit
```

```

-> response JSON
-> browser
-> frontend state/refetch
-> rerender
-> visible result
-> reload
-> persisted result

```

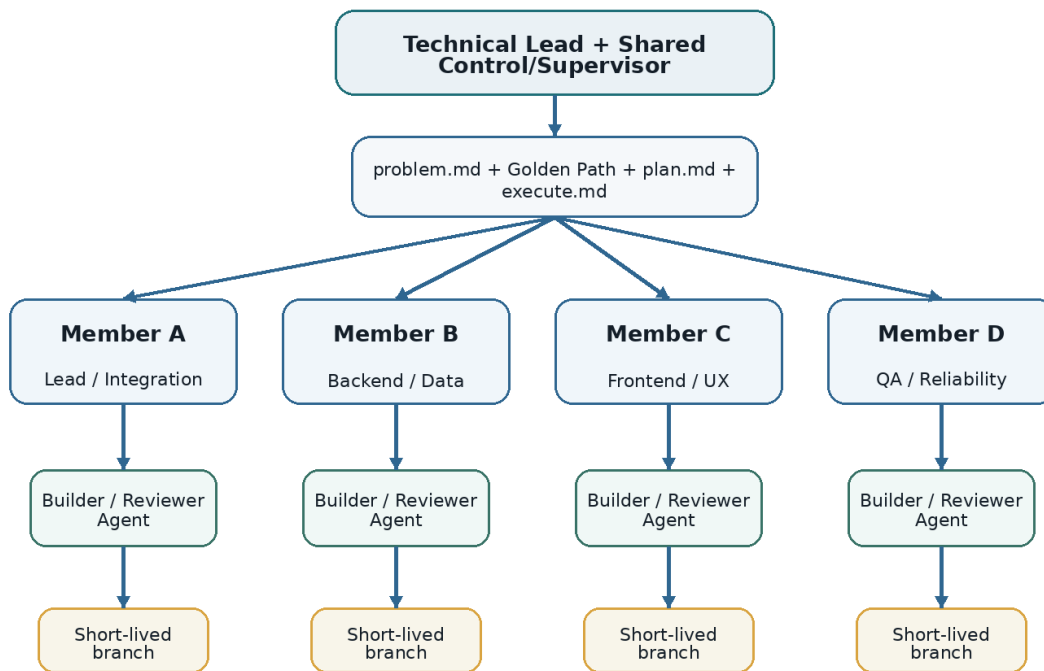
This tracing method is also a debugging tool. If the user sees the wrong state, walk the path and ask where reality first diverges from the contract or invariant.

PART IV - AGENTS, DECISIONS, GIT, AND INCREMENTAL INTEGRATION

19. Individual Agent Setup

Every member may have a local clone and a repository-capable Builder agent. Each member may also use a personal Supervisor for understanding, plan review, and debugging. However, these agents are subordinate to the shared project authority. They do not invent separate problem.md or plan.md files.

One Shared Design, Multiple Bounded Human-Agent Loops



Agents do not need to communicate directly. The coordination medium is stronger and more auditable: Git, shared artifacts, contracts, short human syncs, and explicit escalation for material decisions.

20. Primary Team Supervisor vs Individual Supervisors

Supervisor scope	Responsibilities
Primary shared Control/Supervisor	Challenge interpretation; architecture; Golden Path; workstream decomposition; dependency conflicts; contract disputes; major design changes; scope; release.

Supervisor scope	Responsibilities
Individual Supervisor	Workstream understanding; Builder-plan review; teaching; bounded debugging; local implementation decisions; escalation when shared design would change.

Required instruction to personal supervisors

problem.md, plan.md, execute.md, and the repository are authoritative. Do not independently redesign the shared system. Escalate material shared-design changes to the Technical Lead/team.

21. Builder Agent Workflow

```
Assigned workstream
-> sync main
-> create short-lived branch
-> Builder reconstructs relevant repository state
-> Builder Planning Phase / Plan Mode
-> bounded implementation plan
-> human owner review
-> approve
-> implement
-> test/debug
-> record evidence
-> update authorized execute.md state
-> integration review
-> merge
```

plan.md and Builder Planning Phase are different. plan.md is the persistent whole-system design. Builder Planning Phase is a repo-grounded plan for one bounded workstream: which files change, how the implementation will proceed, what tests prove it, and what integration or conflict risks exist.

22. Universal Builder Prompt

```
ROLE:
You are the Builder for one bounded workstream in a shared multi-person hackathon repository.

WORKSTREAM:
WS-XX - <name>

HUMAN OWNER:
<Member>

OBJECTIVE:
<capability>

GOLDEN PATH RELEVANCE:
<Critical / Supporting / Optional>

READ FIRST:
- AGENTS.md
- problem.md
- plan.md
- execute.md
- relevant code
- relevant tests
- relevant migrations

TEAM CONTEXT:
Other humans and Builder agents are working concurrently on separate workstreams.
```

```

Do not silently change:
- shared architecture
- public API contracts
- business invariants
- shared schemas
- another member's owned scope

DEPENDENCIES:
...

APPROVED CONTRACTS:
...

IN SCOPE:
...

OUT OF SCOPE:
...

FIRST:
Reconstruct relevant repository state.

THEN:
Enter Builder Planning Phase / Plan Mode.

Return:
1. current-state summary
2. files likely to change
3. implementation steps
4. testing plan
5. integration implications
6. dependency concerns
7. possible merge conflicts

STOP before implementation.

After human approval:
- implement only approved scope;
- test/debug;
- preserve shared contracts;
- record evidence.

STOP AND REPORT material architecture/schema/API/business-policy/dependency/scope changes.

```

23. Decision Authority

Level	Who decides	Examples
Level 1 - Local	Workstream owner	Function names; internal implementation detail; local refactor; test organization; small design-preserving fixes.
Level 2 - Shared	Lead + affected owners	API contract; shared schema; DB relationship; new dependency; cross-workstream behavior; core invariant.
Level 3 - Product	Team/Lead + shared Supervisor	MVP; Golden Path; architecture; major scope change; release/deployment strategy.

This avoids two opposite failures. If every tiny choice requires a team meeting, execution stalls. If every agent can change shared design freely, parallel work diverges. Local autonomy is broad inside a protected shared boundary.

24. Git and Branching

main is the best-known integrated state. Workstreams use short-lived branches such as feat/ws03-transfer-api, feat/ws03-transfer-ui, test/ws03-transfer, or fix/ws03-idempotency. A branch is an isolated workbench, not a long-running alternate product.

```
git checkout main
git pull
git checkout -b feat/ws03-transfer-api

# work, test, commit

git fetch origin
git merge origin/main # or rebase if the team is comfortable
# resolve conflicts on the feature branch
# retest
git push
```

Long-lived “frontend” and “backend” branches are dangerous because each accumulates assumptions about an increasingly stale world. Prefer small capability branches and integrate every 1-3 hours when practical.

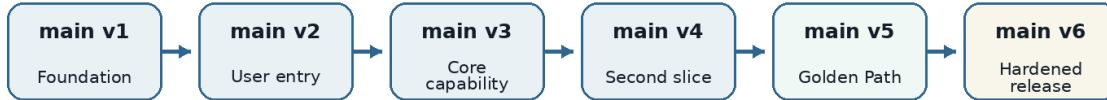
25. Shared File Ownership

Shared artifact	Default owner	Reason
problem.md	Technical Lead	One reconciler protects requirement consistency.
plan.md	Technical Lead	Affected owners contribute; Lead reconciles shared architecture.
execute.md	Workstream owners edit their section or Lead reconciles	Avoid multiple people rewriting the same global status text.
review.md	QA/Reviewer owner	Keep findings separate from execution state.
README.md	Designated documentation owner	Final project-facing truth should be coherent.
Migrations	Backend/persistence owner	Avoid simultaneous migration-number/schema conflicts.
Central API client	Frontend owner	Protect a common request/response integration layer.

26. Incremental Merging

Do not wait for every slice to finish and then merge everything. Verified increments should reach main as soon as they are safe. Later workstreams then build on the real integrated state instead of assumptions.

Incremental Integration: main Becomes the Product Gradual



Verified workstream -> merge -> next workstream builds on the best-known integrated state

A backend capability may merge before its UI if it is independently useful and contract-stable. The frontend branch then updates from main and performs the real integration. If a frontend branch contains temporary mocks, either keep the mocked path isolated until real integration or ensure mocks cannot accidentally become the production path.

PART V - QA, REVIEW, EVIDENCE, AND SYSTEM CONFIDENCE

27. QA Throughout the Lifecycle: Shift Left

QA begins at requirements, not after coding. At intake, QA asks whether expected behavior is testable. At contract design, QA challenges ambiguous fields and errors. During implementation, developers run focused checks while QA prepares independent scenarios. At integration, QA verifies the real frontend/backend slice. At system stage, QA runs regression and Golden-Path E2E. Near release, QA helps decide whether the evidence supports shipping.

Stage	QA activity
Requirement	Identify testable outcomes and ambiguous behavior.
Contract	Challenge edge cases, error semantics, and response assumptions.
Implementation	Developer-focused tests; QA prepares risk cases.
Integration	Verify real API/client behavior, persistence, and failures.
System	Regression and Golden-Path E2E.
Release	Chosen-runtime verification and final defect triage.

28. QA Test Matrix

Capability	Happy path	Boundary	Negative	Persistence	Integration	Concurrency/Retry	Regression	E2E
Transfer	valid transfer	entire balance / minimal positive	0, negative, insufficient, missing recipient	balances + transaction durable	UI/API response	duplicate/retry, concurrency	old balance/history flows	critical
Request	create request	min/max amount	self/invalid user if prohibited	pending request durable	request appears in target UI	repeat submit	existing request views	critical
Dashboard	correct values	empty history / many rows	backend error	reload preserves server truth	refetch/state	n/a	core actions still visible	supporting

Risk prioritization matters. A cosmetic alignment issue and a duplicate financial transaction are not equal. Spend the strongest QA effort on invariants, irreversible state changes, cross-system boundaries, and anything that would make the Golden Path lie to the user.

29. Reviewer Agent

A Reviewer is an independent verification role, not a mandatory ceremony after every tiny change. Use fresh Reviewer context for high-risk business logic, concurrency, money/data integrity, state machines, security-sensitive behavior, shared schema changes, deployment, or the final repository review.

```

ROLE:
You are an independent Reviewer for a bounded workstream in a shared hackathon repository.

WORKSTREAM:
WS-XX

READ:
- problem.md
- plan.md
- execute.md
- relevant implementation
- tests
- diff
- migrations where relevant

VERIFY:
- requirement coverage
- business invariants
- API contract adherence
- persistence correctness
- concurrency/integrity if relevant
- frontend/backend integration
- important negative paths
- missing verification
- regression risk

Initially do not implement changes.

CLASSIFY FINDINGS:
BUG / DESIGN ISSUE / CONTRACT DRIFT / INTEGRATION FAILURE / MISSING VERIFICATION / DOC
DRIFT / DEFERRED / IMPROVEMENT

SEVERITY:
P0 / P1 / P2

RETURN:
- finding
- evidence
- impact
- recommendation
- verdict: PASS / PASS WITH NON-BLOCKING FINDINGS / BLOCKED

```

30. Workstream Definition of Done

```

DESIGNED
-> PLANNED
-> APPROVED
-> IMPLEMENTED
-> INTEGRATED where relevant
-> VERIFIED

```

```
-> EVIDENCE RECORDED
-> MERGED
-> EXIT CRITERIA SATISFIED
-> CLOSED
```

Not every workstream touches every layer. A migration-only foundation may have no frontend. An algorithm workstream may expose its result through an existing endpoint. The Definition of Done should apply only relevant criteria, but “code exists” is never enough.

31. Swarming Critical Blockers

Ownership is the default, not a prison. If a critical Golden-Path capability fails, temporarily swarm it. Backend inspects service/logs and database behavior. Frontend inspects browser Network/state/rendering. QA reproduces systematically and checks expected vs actual state. The Lead traces the contract and dependency chain and prevents random redesign.

Swarm trigger

A critical blocker is consuming disproportionate time, blocks multiple downstream workstreams, or threatens the Golden Path. Announce BLOCKED early rather than hiding the issue for hours.

32. Coordination and Sync Rhythm

```
WORKSTREAM: WS-03
STATUS: IN PROGRESS
DONE: service + endpoint
BLOCKED: need DB constraint decision
CONTRACT CHANGE: none
NEED DECISION: unique request rule
TESTS: 18 passing
MERGE: not ready
NEXT: resolve constraint, add integration test
```

Use 5-10 minute team syncs approximately every 2-3 hours, plus immediate sync for material contract/schema/product decisions. Each sync asks: What works? What is blocked? Did a shared contract change? What merged? What threatens the Golden Path? What is the next highest-value move?

33. Systematic Full-Stack Integration

Once critical slices exist on main, the team transitions from “my feature” to “our system.” Systematic integration checks cross-feature assumptions that isolated workstream tests do not expose: CORS, base URLs, authentication headers, stale frontend state, error-shape consistency, migration order, shared database constraints, configuration, runtime startup, and interactions between features.

- Frontend/backend contract consistency
- CORS and environment configuration
- Cross-feature state transitions
- Persistence and reload behavior
- Frontend cache/refetch/stale state
- Shared error handling
- Migration correctness
- Startup/build/run commands
- Regression of earlier critical behavior

34. Golden-Path E2E

```
Known state
-> open real frontend
```

```

-> perform critical user action
-> browser sends real request
-> backend executes real business logic
-> database changes
-> response returns
-> UI visibly updates
-> reload/refetch
-> persisted state remains correct
-> run one important invalid path

```

E2E is stronger evidence than isolated unit or API tests because it proves the actual user-visible chain works. It does not replace focused tests; it proves the assembled product honors the story those tests were supposed to support.

35. Regression Strategy

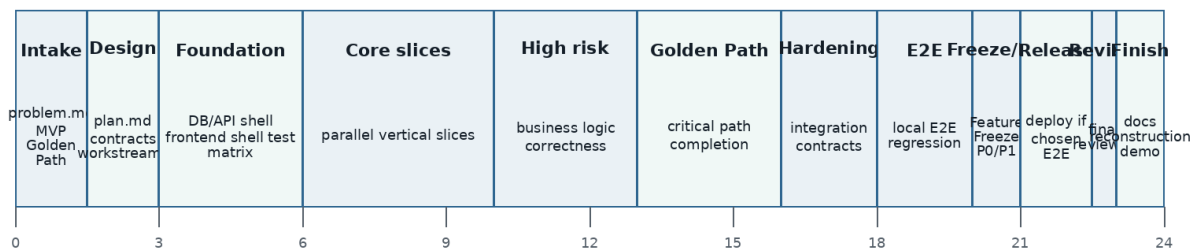
Every merge can break previously working behavior. The team therefore keeps a small risk-based regression set centered on the Golden Path and cross-cutting contracts. Run the smallest set that can catch likely damage after meaningful changes, then broaden near Feature Freeze.

- Critical API happy paths
- One or two important negative paths
- DB migration and persistence checks
- Golden-Path UI actions
- Authentication/session path if present
- High-risk concurrency/retry checks if present

PART VI - THE 24-HOUR OPERATING RHYTHM AND RELEASE

36. Recommended 24-Hour Timeline

Recommended 24-Hour Operating Rhythm



Fatigue is an engineering risk: stagger meals and short recovery windows; keep critical system knowledge available.

Hour	Focus	Exit condition
0-1.5	Challenge Intake	All members understand challenge; problem.md, MVP, Golden Path approved.

Hour	Focus	Exit condition
1.5-3	Master System Design	plan.md, contracts, workstreams, owners, execute.md.
3-6	Foundation	DB/API skeleton, frontend shell, test matrix, first real API/UI contact.
6-10	Core slices	Parallel critical capability implementation and incremental merging.
10-13	High-risk logic	Business/decision logic, integrity, concurrency, optimization as required.
13-16	Golden Path completion	Close missing critical capabilities; avoid uncontrolled optional expansion.
16-18	Hardening	Systematic full-stack integration and cross-feature fixes.
18-20	Local E2E + regression	Prove real Golden Path, refresh/persistence, major negative path.
~20	Feature Freeze	Stop optional features.
20-21	P0/P1 correction	Focused stability and regression.
21	Release decision	Local vs deployment based on rules, stability, and remaining risk.
21-22.5	Deployment if selected	Production config, migrations, CORS, deployed E2E.
22.5-23	Final independent review	Whole-system P0/P1 findings.
23-23.5	Docs reconciliation	README, execute.md, review.md, known limitations.
23.5-23.8	Whole-project reconstruction	Team explanation and judge questions.
23.8-24	Demo Freeze	Rehearsal, Git/source checkpoint, stable submission.

The exact clock is flexible. The important structure is not: understand -> design -> implement in increments -> assemble Golden Path -> harden -> prove -> freeze -> release -> review -> explain.

37. Rest and Fatigue Management

Fatigue is a software risk because it increases bad merges, forgotten contracts, destructive refactors, and poor judgment. Do not romanticize continuous work for 24 hours. Stagger meals and short recovery periods so the team retains at least one system-level person and one implementation-capable person during important windows. Avoid major architecture changes when cognitive quality is clearly degraded. Aim to have everyone available for final integration, review, and demo rehearsal.

Practical rule

Protect the final 4-6 hours from maximum fatigue. A fresh enough team that can debug and explain a smaller working MVP often beats an exhausted team with more features.

38. Golden Path Assembly

The Golden Path is assembled incrementally as critical slices merge. Optional slices are not prerequisites unless the challenge explicitly makes them so. The team should be able to maintain a visible checklist of the journey and answer what currently prevents it from running end to end.

```
[x] user entry
[x] create/submit action
[x] core processing
[~] state transition
[ ] persisted result
[ ] final dashboard state
[ ] reload proof
```

39. Feature Freeze and Demo Freeze

Freeze	Meaning
Feature Freeze	Stop new optional capabilities. Continue integration, bug fixes, tests, docs, and release work.

Freeze	Meaning
Demo Freeze	Stop risky modifications before submission/demo. Only emergency changes with clear evidence and rollback plan.

Feature Freeze protects the system from scope expansion when integration risk is highest. Demo Freeze protects the demonstrated artifact from last-minute “improvements” that have not received enough verification.

40. Release / Deployment Decision

Deployment is not automatically the next step. First prove the Golden Path locally. Then ask whether the event requires a hosted runtime, whether providers and credentials are ready, whether enough time remains, and whether deployment risk is lower than the value it adds.

- Production frontend API base URL
- Environment variables and secrets
- DATABASE_URL and database reachability
- Migrations applied in the chosen environment
- CORS origins
- Frontend build command and artifacts
- Backend start command
- Provider networking/port behavior
- Deployed Golden-Path E2E

If local demonstration is allowed and deployment is unstable, the team may deliberately choose a local release. The choice should be explicit, not the accidental result of spending the final hour debugging hosting while the local product was already good.

41. Final Review and P0/P1 Corrections

The final Reviewer looks at the repository as one product. Priority order is Golden-Path correctness, business invariants, data integrity, API contracts, critical negative paths, integration, runtime/deployment, and misleading documentation. P0 blocks release; P1 should normally be fixed before submission; P2 is documented unless it becomes strategically important. Avoid large late refactors.

42. README and Document Reconciliation

The final README is not the original template plan. It must describe the software that actually exists: product summary, actual features, architecture, tech stack, API overview, setup/run, migrations, tests, demo, deployment if used, known limitations, and intentional deferrals. Remove claims for features that never shipped.

43. Final Whole-Project Reconstruction and Judge Readiness

Agents accelerate implementation, but judges question humans. Before submission, reconstruct the final system as a group. Member A should explain the architecture and dependency story. Member B should explain authoritative backend behavior, data integrity, and difficult business logic. Member C should explain the user flow, client state, and API interaction. Member D should explain how correctness was challenged and proven. Every member should understand the problem, MVP, Golden Path, main runtime flow, tradeoffs, and major bugs fixed.

- Why did you choose this architecture?
- What is your Golden Path and why did you prioritize it?
- How does the frontend know what the backend returns?
- What happens when an invalid or duplicate request arrives?
- What state is persisted and why?
- What was the highest-risk part of the system?
- How did you test it?
- What did you intentionally defer?
- What would you change with another day?

- Show the path from a user click to the database and back.

44. Why This Workflow Converges into a Complete Project

Mechanism	How it contributes to completion
problem.md	Prevents solving the wrong problem.
Golden Path	Prevents scope from outrunning the event.
plan.md	Prevents architectural fragmentation.
API/data contracts	Allow safe parallel frontend/backend/QA work.
Capability workstreams	Create bounded ownership and measurable progress.
Builder agents	Accelerate implementation while remaining constrained.
Human gates	Keep material decisions accountable.
Short-lived Git branches	Isolate concurrent work.
Incremental merges	Prevent final-hour integration shock.
Shift-left QA	Finds ambiguity and failures before they become expensive.
Reviewer agents	Add independent challenge where risk justifies it.
Golden Path integration	Proves individual capabilities form one system.
E2E	Proves the actual user journey and persistence.
Feature/Demo Freeze	Protect stability near submission.
Final reconstruction	Turns agent-assisted code into a human-explainable project.

PART VII - FULL 4-MEMBER CASE STUDY

45. Fictional Challenge: RapidRelief Resource Allocation Platform

Challenge: Build a web application for emergency coordinators to register incidents, see available response teams, assign an appropriate team to an incident, update assignment status, and preserve an auditable history. The system should avoid double-booking a response team and should remain understandable under concurrent coordinator actions.

45.1 Challenge Reconstruction

Lens	Result
Users	Emergency coordinator; response team operator.
Core problem	Match incidents to available teams and preserve trustworthy assignment state.
Frontend	Incident form; team availability view; assignment action; status/history view.
Backend	Validate incidents; enforce team availability; create/update assignments; expose history.
Persistence	Incidents, teams, assignments, status transitions.
Invariants	A team cannot be active on two assignments; invalid transition rejected; history reflects durable state.
Technical risk	Concurrent coordinators assigning the same team; contract drift between status UI and backend state machine.

45.2 problem.md Summary

- Coordinators can create incidents with severity, location label, affected people, and description.
- System shows teams with capability and availability.
- Coordinator can assign one available team to one incident.
- Active team cannot be assigned again until released.
- Assignment status can move through approved transitions.
- All critical state survives refresh and restart.
- Activity/history reflects real committed state.

45.3 MVP and Golden Path

MVP: incident creation + team list + safe assignment + status transition + visible persisted result. Golden Path: coordinator creates a high-severity incident -> sees an available medical team -> assigns it -> backend verifies availability and commits assignment -> UI shows assigned state -> coordinator marks the assignment complete -> team becomes available again -> reload proves persistence.

45.4 Master Design

```
Frontend (React or equivalent)
  -> API client
Backend (FastAPI)
  -> routers
  -> Pydantic contracts
  -> services/use cases
  -> assignment state logic
  -> SQLAlchemy
Database (PostgreSQL-ready)
  -> incidents
  -> response_teams
  -> assignments
  -> assignment_events
```

45.5 Example Contracts

```
POST /incidents
Request: {severity, affected_people, location, description}
Response: {id, severity, affected_people, location, status}

POST /assignments
Request: {incident_id, team_id}
Response: {id, incident_id, team_id, status, created_at}
Errors: 404 missing entity; 409 team unavailable / assignment conflict

PATCH /assignments/{id}
Request: {status}
Response: {id, status, updated_at}
```

45.6 Workstreams and Ownership

ID	Capability	Implementation owner(s)	Verification	Integration
WS-01	Persistence foundation	B	D	A
WS-02	Incident creation	B + C	D	A
WS-03	Team availability	B + C	D	A
WS-04	Safe assignment	B + C	D + Reviewer	A
WS-05	Status transitions	B + C	D	A
WS-06	Dashboard/history	C + B	D	A
WS-07	Concurrency hardening	B	D + Reviewer	A
WS-08	Golden Path E2E	All	D	A

45.7 Parallel Implementation Example

At Hour 4, the assignment API contract is design-stable. Member B starts the service, locking/integrity strategy, and POST /assignments endpoint. Member C builds the incident detail page, team selector, loading state, 409 conflict message, and a temporary createAssignment() stub that returns the exact approved response shape. Member D writes tests for missing team, already-active team, same request repeated, and two coordinators racing for one team. Member A watches the dependency and keeps the next state-transition contract aligned.

Member C finishes the UI first. The workstream is not complete. C moves to the status-history view while keeping the assignment action isolated behind the contract. One hour later B completes the endpoint and backend tests. B merges. C updates from main, removes the stub, and runs the real request. The first integration returns 422 because the backend accidentally named the field `responder_team_id`. The approved contract says `team_id`, so B fixes the implementation. QA re-runs the request and verifies the committed assignment in the database. The integrated slice then merges.

45.8 Contract Evolution Example

Later the dashboard requirement clearly needs `assigned_at` to sort recent work. The current contract lacks it. This is not implementation drift; the design is incomplete. Member C raises NEED CONTRACT DECISION. A, B, and C approve adding `assigned_at`, `plan.md` is updated, B updates the response schema, C updates rendering, and D extends contract tests. The change is deliberate and traceable.

45.9 QA and Reviewer Findings

Source	Finding	Impact	Action
QA	Two rapid clicks create two client requests.	Potential duplicate assignment attempt.	Client disables button after submit; backend still enforces invariant.
Reviewer	Two concurrent coordinators can both observe team available before write.	P0/P1 integrity risk depending implementation.	Use transactional locking/constraint strategy and add concurrency test.
QA	Completing assignment updates assignment but not team availability.	Golden Path fails after reload.	Fix service transaction and regression test.

45.10 Incremental Merge Story

```
main v1 - starter + runtime
main v2 - incidents/teams persistence
main v3 - incident creation vertical slice
main v4 - team availability slice
main v5 - safe assignment slice
main v6 - status transition slice
main v7 - history/dashboard
main v8 - concurrency hardening
main v9 - Golden Path hardened
```

45.11 Golden-Path E2E

10. Reset/seed known state with an available medical team.
11. Open the real frontend and create an incident.
12. Select the team and assign it.
13. Observe successful API response and visible assigned state.
14. Confirm database assignment and team unavailable state.
15. Attempt a second active assignment for the same team and verify rejection with no corrupt state.
16. Complete the assignment.
17. Refresh the page and verify assignment history and team availability persist.

45.12 Final Demo Story

The demo is now a coherent narrative, not a feature tour: “We receive an incident, safely allocate an available team, prevent conflicting allocation, transition the work to completion, and preserve an auditable history.” The Lead can explain architecture, B can explain the concurrency invariant, C can trace the client/API state flow, and D can show how duplicate/concurrent behavior was verified.

PART VIII - SAME CASE STUDY WITH THREE MEMBERS

46. Role Redistribution

With three members, the product goal and shared artifacts do not change. The team reduces parallel width, cuts optional scope earlier, and deliberately preserves verification responsibilities.

Member	Role	Case-study allocation
A	Lead + integration + selected backend	Own contracts, merges, architecture, integration, and user-entry/coordination APIs. Avoid absorbing dashboard polish.
B	Backend + DB + core logic	Own persistence, assignment/state logic, concurrency, backend tests.
C	Frontend + QA + E2E	Own all critical UI, test matrix, contract checks, real integration, regression, and E2E.

46.1 Reduced Parallelism Strategy

Member C cannot simultaneously build every screen and perform all QA. Therefore the team narrows the Golden Path UI to the smallest credible set: incident create, team list, assignment, completion, history. Optional analytics and advanced filters are deferred. Member A helps frontend integration after architecture stabilizes rather than starting another backend feature. Member B remains protected for the highest-risk integrity work.

46.2 Preserving Review Independence

Member C may own QA, but the team can still use a fresh Reviewer agent for WS-04 safe assignment and the final repository. Member A can reproduce one critical backend negative path independently. Independence is not binary; the goal is to avoid relying only on the author's own mental model for the riskiest behavior.

46.3 3-Member Timeline Adjustment

The team should reach Feature Freeze with fewer optional capabilities and a larger safety margin. A reasonable adaptation is to spend the same first three hours on shared understanding/design, but target Golden Path completion around Hour 15 rather than assuming a fourth person can absorb QA/integration work. If the critical path is late, drop optional dashboard polish before dropping concurrency or E2E verification.

PART IX - REUSABLE PROMPT LIBRARY

47. Shared Control/Supervisor Startup Prompt

You are supervising a 3/4-person 24-hour hackathon team.

The team uses:

- one authoritative problem.md;
- one plan.md;
- one Golden Path;
- capability-oriented workstreams;
- multiple parallel Builder agents;
- separate short-lived branches;
- selective Reviewer agents;
- incremental integration into main.

Your responsibilities:

- maintain architecture consistency;
- identify workstream dependencies;
- help normalize the challenge before problem.md approval;
- protect Golden Path scope;
- prepare bounded workstream prompts;

- detect conflicting parallel work;
- protect API/data contracts;
- coordinate integration and merge order;
- help resolve material design decisions;
- supervise release and final reconstruction.

Do not become the default Builder.

48. Individual Supervisor Prompt

You are supervising Member <X>'s assigned workstream in a shared multi-member hackathon repository.

Authoritative shared state:

problem.md
plan.md
execute.md
actual code/tests/migrations/runtime evidence

Do not independently redesign the shared system.

Help the member:

- reconstruct their bounded workstream;
- review the Builder's implementation plan;
- understand why the code works;
- debug local failures;
- verify relevant behavior;
- prepare concise status/evidence;
- escalate material API/schema/architecture/business-policy changes to the Technical Lead/team.

49. QA Test-Design Prompt

Read problem.md and plan.md for WS-XX.

Derive a concise risk-prioritized test matrix covering:

- happy paths
- boundary values
- invalid inputs
- persistence checks
- API contract risks
- frontend/backend integration risks
- concurrency/retry cases where relevant
- regression risks
- Golden Path relevance

Separate mandatory evidence from nice-to-have coverage.
Do not implement changes yet.

50. Integration Prompt

ROLE:

Integration Builder/Reviewer.

OBJECTIVE:

Systematically harden the assembled Golden Path on main.

Read problem.md, plan.md, execute.md, and the actual merged repository.

Trace each critical user journey across:

```
frontend -> API -> backend -> business logic -> database -> response -> frontend.
```

Look for:

- contract mismatch
- CORS/configuration errors
- stale frontend state
- persistence failures
- wrong error handling
- cross-feature regressions
- migration/runtime issues

Do not add unrelated features.

51. Final Reviewer Prompt

Review the repository as an independent final engineering Reviewer.

Prioritize:

1. Golden Path correctness
2. business invariants
3. persistence/data integrity
4. frontend/backend contracts
5. important negative paths
6. deployment/runtime correctness if used
7. misleading documentation

Do not optimize style unless it materially affects correctness or maintainability.
Classify findings P0/P1/P2 and return a release verdict.

PART X - OPERATING CHECKLISTS AND FAILURE MODES

52. Challenge Intake Checklist

- Every member read the official challenge and relevant rules.
- Plain-English product and actors agreed.
- Explicit vs inferred vs optional requirements separated.
- Critical business invariants identified.
- MVP is small enough for 24 hours.
- Golden Path is one coherent user journey, not a list of features.
- problem.md approved before implementation starts.

53. plan.md Approval Checklist

- Architecture supports the Golden Path without unnecessary complexity.
- Entities/relationships and persistence needs are defined.
- Critical invariants have an enforcement location.
- Important API contracts are design-stable.
- Frontend screens and backend capabilities map to the same workflows.
- Workstreams have dependencies and owners.
- High-risk work has a verification plan.
- Release strategy has clear decision criteria.

54. API Contract Readiness Checklist

- HTTP method and path agreed.
- Request fields, types, optionality, and units agreed.

- Response fields and types agreed.
- Success status defined.
- Important application errors defined.
- Auth/context requirements defined if relevant.
- Frontend, backend, and QA are using the same version.

55. Workstream Readiness and Builder Plan Checklist

- Workstream objective is bounded.
- Dependencies are COMPLETE or the preparatory work is explicitly safe.
- Approved contracts are referenced.
- In-scope and out-of-scope are explicit.
- Builder identified expected files and merge-risk areas.
- Testing plan covers the capability risk.
- No silent shared-design change is hidden in the implementation plan.

56. Frontend/Backend Rendezvous Checklist

- Real backend endpoint available.
- Frontend branch updated from current main.
- Temporary mock/stub removed or bypassed.
- Actual request matches approved contract.
- Actual response matches approved contract.
- Loading/success/error states work with real responses.
- Backend logs and browser Network tab show expected behavior.
- Persistence is checked where relevant.
- QA runs the critical integrated scenarios.

57. Merge Readiness Checklist

- Relevant tests pass.
- Integrated behavior is verified where required.
- No unauthorized contract/schema changes.
- Feature branch contains no accidental/unrelated files.
- Branch is updated with current main and conflicts are resolved.
- execute.md evidence/status is updated or reported to the Lead.
- Known non-blocking issues are recorded.

58. Golden Path / E2E Checklist

- Known initial state exists.
- Real frontend and backend are running.
- Critical user action reaches the database/business logic.
- Visible result matches expected outcome.
- Reload/refetch preserves authoritative state.
- At least one important invalid path is proven.
- No P0/P1 defects remain on the journey.

59. Deployment Checklist

- Local Golden-Path E2E passed first.
- Deployment is required or strategically justified.
- Environment variables configured.
- DATABASE_URL correct and reachable.
- Migrations applied.

- Frontend API base URL correct.
- CORS correct.
- Backend and frontend start/build commands work.
- Deployed Golden-Path E2E passes.
- Rollback/local-demo fallback understood.

60. Demo Freeze / Judge Readiness Checklist

- README reflects actual software.
- Git/source checkpoint is clean and pushed.
- No risky unverified change remains.
- Golden Path demo rehearsed.
- Each member knows their deep area and the whole Golden Path.
- Team can explain architecture, contracts, persistence, key risks, testing, tradeoffs, and deferrals.
- Known limitations are stated accurately rather than hidden.

61. Common Failure Modes and Prevention

Failure mode	Why it hurts	Prevention
Everyone starts coding immediately	Team solves different interpretations.	Require shared intake and problem.md approval.
Multiple architectures	Agents produce incompatible systems.	One plan.md; material changes are gated.
Backend/frontend invent contracts independently	Integration fails late.	Design-stable contracts in plan.md first.
Silent API/schema changes	Other branches become wrong without knowing.	NEED DECISION protocol + shared update.
Long-lived layer branches	Huge merge and stale assumptions.	Short-lived capability branches + frequent main updates.
QA starts late	Ambiguity and high-risk failures surface at the end.	Shift-left test design and incremental verification.
Agents roam outside scope	Conflicts and accidental redesign.	Bounded prompts with stop conditions.
No migration owner	Schema/migration conflicts.	One persistence/migration coordinator at a time.
Optional features dominate	Golden Path remains incomplete.	Tag CRITICAL/SUPPORTING/OPTIONAL and cut optional first.
"My part is done"	Disconnected partial features.	Definition of Done includes integration and verification.
Mocks survive too long	Frontend demo is disconnected from backend truth.	Integration rendezvous as soon as endpoint is ready.
Deploy before local E2E	Hosting hides basic product failures.	Local Golden Path must pass first.
Late architecture refactor	High regression risk under fatigue.	Feature Freeze and final P0/P1 focus.
README overclaims	Judges find mismatch between docs and runtime.	Final reconciliation against code/tests/runtime.

62. Analogy Map

Concept	Analogy	Technical mapping
API contract	Two bridge teams agree exactly where their halves meet.	The contract defines the independent integration boundary.
Technical Lead	Orchestra conductor.	Coordinates timing and coherence without playing every instrument.
Database	Durable ledger/archive.	Authoritative state survives refresh/restart.
QA	Engineering inspection throughout construction.	Quality is designed and checked continuously, not only at handover.
Vertical slice	One complete elevator from button to motor to floor arrival.	A capability works across necessary layers instead of finishing one layer for the whole app.
Feature Freeze	Stop adding rooms.	Focus changes on stability and completion.
Demo Freeze	Stop moving furniture before inspection.	Protect the verified demonstration state.
Git branch	Isolated workbench on one assembly	Parallel changes are separated until

Concept	Analogy	Technical mapping
	line.	verified and merged.

63. Final One-Page Team Operating Rules

18. One repository. One problem.md. One Golden Path. One plan.md.
19. All members understand the challenge before implementation.
20. API/data contracts become design-stable before parallel consumers/producers diverge.
21. Each workstream has one accountable owner plus explicit frontend/QA/integration responsibility where relevant.
22. Agents work on bounded scopes and stop for material shared-design changes.
23. Every developer verifies their own work; QA independently challenges important behavior.
24. Use short-lived branches and merge verified increments frequently.
25. When completion times differ, move people to READY work and rendezvous as soon as dependencies are available.
26. When the Golden Path is blocked, swarm the blocker.
27. Cut optional scope before cutting core correctness or E2E proof.
28. Feature Freeze before release; Demo Freeze before submission.
29. Every team member must be able to explain the final product, not only the agent output.

64. Master Causal Chain

The full system can be remembered as one causal chain:

```

Challenge understanding
-> correct problem.md
-> disciplined MVP
-> Golden Path priority
-> shared Master System Design
-> design-stable contracts
-> dependency-aware workstreams
-> bounded human-agent implementation
-> frontend/backend rendezvous
-> QA + selective independent review
-> incremental merge into main
-> Golden Path assembled
-> systematic integration + regression
-> local E2E
-> Feature Freeze
-> release/deployment decision
-> chosen-runtime verification
-> final review + P0/P1 fixes
-> accurate README
-> whole-project reconstruction
-> Demo Freeze
-> submission

```

Final principle

A hackathon team wins reliability by repeatedly turning assumptions into shared artifacts, shared artifacts into bounded implementation, implementation into integrated evidence, and integrated evidence into a stable, explainable Golden Path.

Appendix A - What Each Member Watches Through the 24 Hours

Window	Member A	Member B	Member C	Member D
0-3	A: intake/design	B: invariants/data/API input	C: user flows/screens/contracts	D: testability/risks
3-8	A: integration skeleton/merges	B: backend foundation/core APIs	C: frontend shell + first contracts	D: test matrix + early API checks
8-14	A: dependency/scope + critical implementation	B: core logic/high-risk services	C: core UI + real integration	D: workstream verification + Reviewer runs
14-18	A: Golden Path integration	B: finish critical correctness	C: finish critical UI/integration	D: regression + blocker reproduction
18-21	A: system hardening/release choice	B: logs/DB/fixes	C: browser/UI/fixes	D: E2E/regression
21-24	A: release/docs/reconstruction/demo	B: production/backend support + explanation	C: production/frontend support + explanation	D: final review/E2E/judge evidence

Appendix B - Lightweight Decision Log Template

```

D-01 <decision title>
Date/Hour: ...
Decision Level: 1 / 2 / 3
Context: ...
Decision: ...
Reason: ...
Affected workstreams/contracts: ...
Owner: ...

```

Only record meaningful shared decisions. Do not create bureaucracy around local variable names or small design-preserving fixes.

Appendix C - Evidence Examples

- Unit/API test names and pass counts
- Manual reproduction steps with expected/actual result
- Browser Network evidence for request/response
- Database query showing committed state
- Screenshot or short note showing UI result where useful
- Concurrency/retry test output
- Migration status
- Deployed runtime URL/status where applicable
- Reviewer verdict and resolved P0/P1 findings

BUILD ONE SYSTEM, NOT FOUR PARTS

**Shared truth -> bounded parallelism -> frequent integration -> verified Golden
Path -> explainable submission**

PART XI - FIRST-THREE-HOURS COMMAND SHEET AND STARTUP SEQUENCE

The first three hours determine whether the remaining twenty-one hours are spent building one coherent product or repairing four different interpretations. This section is intentionally operational: it tells the team what to do, who speaks, what must be written down, what can start in parallel, and what must not start yet.

Operating rule

No member starts challenge-specific implementation merely because they have free hands. The team first creates a shared product truth and enough design stability to make parallel work safe.

65. Minute 0-15: Rules, Environment, and Evidence Capture

The team begins by separating official authority from assumptions. The Technical Lead opens the official rulebook, challenge statement, organizer clarifications, and submission requirements. Member B confirms backend/runtime constraints, Member C confirms required user-facing platform constraints, and Member D creates a short list of testable submission obligations. This is not bureaucracy: a technically excellent project can still fail if it violates a format, platform, runtime, or submission rule.

Owner	Action	Output	Why it matters
A - Lead	Capture official rules and clarifications	Authority notes / challenge source	Prevents building against an invalid assumption
B - Backend	Check runtime, DB, networking, provided starter constraints	Environment risk notes	Prevents late infrastructure surprises
C - Frontend	Check required web/mobile form, demo/browser constraints	UI/runtime notes	Prevents wrong client target
D - QA	Convert submission requirements into checks	Submission checklist	Makes compliance verifiable rather than remembered

- Confirm repository access for all members.
- Confirm every member can run the starter locally.
- Confirm Git identity and remote access.
- Confirm one team communication channel.
- Confirm where official clarifications will be recorded.
- Do not create feature branches yet unless the challenge has already been normalized.

66. Minute 15-45: Independent Read, Then Shared Reconstruction

Each member reads the problem independently before the lead explains it. Independent reading reduces anchoring: if the lead misunderstands one sentence, the entire team is less likely to inherit the same mistake. After the read, each person summarizes the product in one minute from their role perspective.

Member	One-minute answer
A	What product are we actually being asked to deliver, and what is the smallest credible outcome?
B	What state, invariants, data, and hard backend behavior appear unavoidable?
C	What user actions and screens must exist for the product to feel complete?
D	What claims will need proof, and what failure cases could invalidate the demo?

The shared Supervisor then performs Challenge-Intake Understanding Reconstruction. The output is explanation first, not code: actors, core workflow, inputs, outputs, business rules, persistence, edge cases, explicit requirements, inferred requirements, optional ideas, and risk. Any disagreement is labeled rather than silently resolved.

67. Minute 45-75: problem.md Approval Gate

The Technical Lead reconciles the challenge into one canonical problem.md. The team reviews it against the official statement line by line for mandatory behavior. This is the first formal human gate.

Approval questions:

- Can every member explain the product in plain language?
- Does every official requirement appear somewhere in problem.md?
- Are inferences clearly distinguished from explicit requirements?
- Are optional ideas clearly marked optional?
- Are important business invariants stated without prematurely choosing implementation mechanisms?
- Would a judge reading problem.md recognize the intended product?

Stop condition

If the team cannot agree on a requirement that materially changes architecture or MVP, do not hide the uncertainty. Record it as NEED CLARIFICATION / ASSUMPTION and make the safest reversible choice until organizer clarification is available.

68. Minute 75-100: MVP and Golden Path Lock

The MVP is not simply a list of features. It is the smallest useful product boundary that satisfies the challenge. The Golden Path is the one successful user journey that proves the MVP produces value. The team writes both explicitly because they become the reference for every later scope decision.

MVP: <smallest useful product boundary>

GOLDEN PATH:

1. <user begins from known state>
2. <user performs critical action>
3. <backend validates/processes>
4. <data changes persist>
5. <result becomes visible>
6. <reload/revisit still shows correct state>

Member D adds at least one important invalid-path companion to the Golden Path. A system that only works when every input is perfect is not yet trustworthy.

69. Minute 100-150: Master System Design Workshop

Now the team designs one architecture. Member A facilitates rather than dictating every decision. Member B proposes data and backend boundaries, Member C proposes screens and client state, Member D challenges testability and failure behavior. The shared Supervisor helps identify hidden coupling and missing dependencies.

Design topic	Primary voice	Questions that must be answered
Entities/data	B	What persists? What relates? Which invariants belong in data/business logic?
API contracts	A+B+C	Which method/path/request/response/err or behavior lets both sides work independently?

UI/user flow	C	Which screen owns each user action? How does loading/error/success appear?
Quality/risk	D	What would be costly to discover late? How will we prove it?
Dependencies/order	A	Which capability unlocks others? What can safely run in parallel?

The output is plan.md: the shared HOW. The team does not attempt to specify every private function. It specifies the architecture and contracts strongly enough that independent Builders cannot accidentally invent different systems.

70. Minute 150-180: Workstream Assignment and Agent Launch

Only after the design is stable enough does the team split. The lead creates dependency-aware workstreams and marks each READY or BLOCKED. Each member receives a bounded assignment, not their own architecture. Every member syncs the same main branch, creates a short-lived workstream branch, and starts their Builder Planning Phase.

At Hour 3	Member A	Member B	Member C	Member D
Primary action	Lead/integration foundation or first cross-cutting capability	Persistence/backend foundation	Frontend shell + contract-based first UI	QA matrix + contract/requirement checks
Agent mode	Supervisor + bounded Builder if coding	Builder Plan Mode	Builder Plan Mode	QA/Test agent; Reviewer only when risk justifies
Must not do	Rewrite approved architecture silently	Change API/schema unilaterally	Invent response fields independently	Wait for a finished app before testing

71. First-Three-Hours Exit Criteria

- problem.md approved and committed.
- MVP and Golden Path written and understood by all members.
- plan.md contains architecture, entities, contracts, workstreams, dependencies, and verification strategy.
- execute.md has ownership and status for the first wave of workstreams.
- Shared API contracts for the first critical slices are design-stable.
- At least one person owns migrations/shared schema changes.
- Every active member knows their current branch and bounded scope.
- No one is maintaining a private competing project plan.

PART XII - DETAILED 4-MEMBER ROLE OPERATING MANUALS

The four roles below are primary ownership models, not permanent silos. Each role has a control loop: what it watches continuously, what artifacts it owns, what decisions it may make locally, when it must escalate, and how it hands work to the next person.

72. Member A - Technical Lead / System Integrator Operating Manual

Member A is the convergence owner. The lead may write code, but their irreplaceable job is to prevent parallel speed from becoming parallel divergence. A lead who spends six uninterrupted hours inside one feature branch can accidentally leave the team without architecture supervision.

72.1 Continuous lead dashboard

Signal	Question	Action if unhealthy
Golden Path	What exact step is not executable yet?	Move capacity toward that blocker or cut optional scope
Dependency graph	Which READY capability unlocks the most downstream work?	Prioritize/assign it
Contracts	Did any request/response/schema change?	Run shared-decision protocol immediately
main	Is it runnable and better than the previous checkpoint?	Stop merges until restored
Team load	Who is blocked, idle, overloaded, or duplicating work?	Reassign/swarm
Clock	Are we still on the planned phase?	Reduce scope before reducing core verification

72.2 Lead rhythm during implementation

30. Check execute.md and branch status every 45-90 minutes.
31. Review Level-2/Level-3 decisions as they arise.
32. Coordinate merge order around dependencies, not personality or arrival time.
33. Pull and run main at meaningful checkpoints.
34. Trigger rendezvous when both sides of a slice are sufficiently ready.
35. Prevent the team from opening optional workstreams while a critical Golden-Path step is blocked.
36. Protect Feature Freeze and Demo Freeze.

72.3 Lead anti-patterns

- Becoming the sole coder and neglecting integration.
- Rewriting teammates work instead of establishing contracts.
- Allowing every disagreement to become a full-team meeting.
- Approving architecture changes through chat but failing to update plan.md.
- Waiting until the final hours to run main personally.

73. Member B - Backend / Data / Core Logic Operating Manual

Member B owns server-side correctness and usually the migration lane. The backend engineer is not simply an endpoint factory. They translate business invariants into authoritative server behavior and durable state transitions.

73.1 Before implementing an endpoint

- Confirm the endpoint belongs to an approved workstream.
- Confirm method/path/request/response/error expectations in plan.md.
- Identify authoritative business rules.
- Identify transaction boundary and persistence effects.

- Identify concurrency, retry, idempotency, or ordering risks if relevant.
- Identify which tests can prove the invariant independently of the UI.

73.2 Backend completion packet

WORKSTREAM: WS-XX
 BACKEND STATUS: READY FOR INTEGRATION
 CONTRACT: unchanged | changed with approved decision D-XX
 MIGRATION: none | <revision>
 TESTS: <command + result>
 MANUAL CHECK: <endpoint / docs evidence>
 KNOWN LIMITATION: <none or explicit>
 NEXT CONSUMER: frontend / integration / QA

This packet lets Member C and Member D consume the implementation without needing a verbal reconstruction of every commit.

74. Member C - Frontend / UX / Client Integration Operating Manual

Member C owns the user-facing proof that the system is usable. They work from design-stable contracts and user journeys, not from guesses about whatever backend code happens to exist today.

74.1 Contract-first frontend loop

37. Read Golden Path step and approved contract.
38. Build the UI state machine: idle, input, loading, success, failure.
39. Implement the API-client boundary with one known request/response shape.
40. Use a narrow mock/stub only if the real endpoint is unavailable.
41. Tag the mock clearly so it cannot be mistaken for integrated completion.
42. When backend becomes implementation-stable, sync main and replace the mock.
43. Use browser Network + visible UI + server evidence to verify the real path.

74.2 Frontend completion packet

WORKSTREAM: WS-XX
 FRONTEND STATUS: READY FOR RENDEZVOUS
 SCREEN/COMPONENT: <name>
 CONTRACT EXPECTED: <method path + version/decision>
 MOCKS REMAINING: yes/no
 LOADING/ERROR/SUCCESS STATES: verified
 BROWSER CHECK: <what was observed>
 BLOCKER: <none or dependency>

75. Member D - QA / Integration / Reliability / Utility Operating Manual

Member D owns independent confidence, not all quality. They start before implementation by turning requirements and contracts into testable claims, then move toward the areas with highest consequence and highest uncertainty.

75.1 QA queue priority

44. PO Golden-Path blockers and data corruption risks.
45. New or changed shared contracts.
46. High-risk state transitions, concurrency, retry, authorization, or persistence.
47. Newly integrated vertical slices.
48. Regression on previously working Golden-Path steps.
49. Optional feature polish only after critical confidence is established.

75.2 QA evidence packet

CAPABILITY: WS-XX
 BUILD/COMMIT TESTED: <sha or branch>
 ENVIRONMENT: local/deployed
 HAPPY PATH: PASS/FAIL
 NEGATIVE/BORDER CASES: <results>
 PERSISTENCE: PASS/FAIL/N-A
 INTEGRATION: PASS/FAIL/N-A
 REGRESSION IMPACT: <none / details>
 FINDINGS: P0/P1/P2
 VERDICT: PASS / PASS WITH NON-BLOCKING FINDINGS / BLOCKED

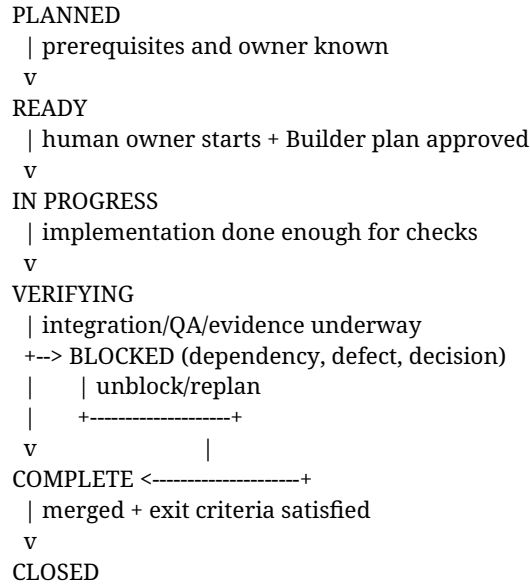
76. Role-to-Role Handoff Map

From	To	Handoff object	Receiver checks first
A Lead	B Backend	Approved workstream + contract + dependencies	Is scope READY and contract design-stable?
A Lead	C Frontend	Golden Path step + UI responsibility + contract	Can UI proceed with real API or temporary contract-faithful mock?
B Backend	C Frontend	Implementation-stable endpoint + test evidence	Does actual response match plan.md?
B/C	D QA	Integrated or testable capability + commit evidence	Which claims are highest risk?
D QA	A Lead	Findings + severity + reproduction	Fix, defer, replan, or release?
A Lead	Whole team	Merged main checkpoint	What Golden Path step is now executable?

PART XIII - SYNCHRONIZATION, HANDOFF, RENDEZVOUS, AND COMPLETION-TIME PROTOCOLS

A multi-member hackathon rarely proceeds at equal speed. The team therefore needs explicit synchronization rules. The goal is to avoid idle waiting without allowing dependent work to drift.

77. Workstream State-Transition Protocol



A frontend track may be COMPLETE while its parent workstream remains IN PROGRESS because backend or integration is not ready. Track-level completion is not workstream completion.

78. Completion-Time Matrix: What Each Person Does Next

Situation	Backend	Frontend	QA	Lead
Backend finishes first	Merge verified server work; move to next READY dependency	Sync endpoint when ready; continue UI	Start API/persistence verification	Confirm contract, merge order, next dependency
Frontend finishes first	Continue backend	Finish states, mark mock, move to READY task	Verify UI logic/mock assumptions	Prevent fake-data path from being mistaken as integrated
QA test design finishes first	Expose endpoint as soon as testable	Expose UI as soon as testable	Test incrementally; prepare regression	Use QA findings to refine risk order
One member blocked >20-30 min	Share evidence, not vague status	Assist if cross-boundary	Reproduce independently	Swarm/reassign/replan
Critical slice ready on both sides	Freeze contract	Remove mock/connect real API	Run focused integration checks	Coordinate rendezvous and merge
Optional task finishes while critical blocked	Move to critical support	Move to critical support	Prioritize blocker	Do not open another optional feature

79. The Frontend/Backend Rendezvous Runbook

A rendezvous is a short synchronization event for one capability, not a general meeting. It occurs when the backend contract is implemented enough to be consumed and the frontend path is ready enough to use it.

50. Backend owner posts implementation-stable contract evidence and any approved decision IDs.
51. Frontend owner updates their branch from main/merge-ready backend state.
52. Frontend removes or bypasses the temporary mock for this path.
53. Run the real request from browser to backend.
54. Inspect browser Network request/response.
55. Inspect backend log/service behavior.
56. Inspect database state where relevant.
57. Compare actual request/response with plan.md.
58. Fix contract drift or integration defects in the owning layer.
59. QA independently reruns the happy path and the most important negative path.
60. Record evidence and mark Integration/Verification complete.
61. Merge remaining work and close the workstream only when exit criteria are met.

Rendezvous success

The capability is no longer two separately working halves. A real user action crosses the actual client/server boundary and produces the intended durable/visible outcome.

80. Handoff Message Templates

BACKEND -> FRONTEND
 WS: WS-03
 Endpoint: POST /transfers
 Contract decision: D-04
 Implementation status: stable for integration
 Tests: 21 passed
 Migration: rev_003 applied
 Known limitation: none
 Branch/merge: merged main @ <sha>

FRONTEND -> QA
 WS: WS-03
 Screen: TransferForm
 Real backend connected: yes
 Mock remaining: no
 Happy-path manual check: pass
 Error state implemented: yes
 Commit: <sha>
 Please prioritize: insufficient funds + duplicate submit

QA -> LEAD
 WS: WS-03
 Verdict: BLOCKED
 Finding: duplicate submit creates two transfers
 Severity: P0
 Reproduction: <steps>
 Evidence: <test/log/db state>
 Suggested owner: backend
 Golden Path impact: direct

81. Communication Without Constant Meetings

The team should communicate state, not stream-of-consciousness. A five-line structured update is more useful than fifteen minutes of narration. Sync meetings are for decisions and cross-boundary risks; routine progress belongs in execute.md and status messages.

WS: WS-04
 STATUS: IN PROGRESS
 DONE: request create endpoint + tests
 BLOCKED: incoming list contract needs decision

CONTRACT CHANGE: proposed created_at field
 NEED DECISION: D-07
 NEXT: frontend rendezvous after decision

PART XIV - GIT, BRANCH, MERGE, MIGRATION, AND CONFLICT-RESOLUTION RUNBOOK

Git is the coordination substrate for parallel human and agent work. The purpose is not to imitate enterprise ceremony; it is to preserve a runnable integrated truth while allowing bounded work to move independently.

82. Repository Ground Rules

- main is the best-known integrated state, not a shared scratchpad.
- One workstream branch should contain one coherent capability or bounded correction.
- Agents must inspect git status/diff before reporting completion.
- Generated junk, secrets, local DB files, editor metadata, and unrelated changes must not enter commits.
- Shared design changes are committed before dependent branches continue when practical.
- No force-push to shared main.
- No destructive reset of another member's published work without explicit coordination.

83. Standard Branch Lifecycle

```
git checkout main
git pull --ff-only
git checkout -b feat/ws03-transfer-api

# work + focused tests
git status
git diff
git add <intentional-files>
git commit -m "implement WS-03 transfer API"
git fetch origin
git merge origin/main # or team-approved rebase strategy
# resolve on feature branch, rerun tests
git push -u origin feat/ws03-transfer-api
```

A team may use rebase instead of merge-to-update, but it should choose one habit before the event. During a hackathon, familiarity and recoverability matter more than history aesthetics.

84. Merge Readiness Gate

Check	Minimum evidence
Scope	Diff contains only approved workstream/supporting changes
Tests	Relevant automated/manual checks pass
Contract	No silent public API/schema drift
Migration	Revision order known and applied successfully if relevant
Integration	Required consumer path tested or explicitly marked pending
Docs/status	execute.md/evidence updated according to ownership policy
Secrets	No credentials, local env values, or private artifacts added

85. Conflict Resolution Protocol

62. Stop and identify whether the conflict is textual or semantic.
63. Pull the latest authoritative plan/decision before choosing a side.
64. For purely textual conflicts, preserve both intended changes and rerun tests.
65. For semantic conflicts involving API/schema/business rules, do not guess. Escalate to Level-2/Level-3 decision authority.
66. Resolve on the feature branch, not by experimenting directly on main.
67. Run the affected slice after resolution, not just unit tests.
68. Record a decision if the resolution changes shared behavior.

A merge tool telling you which lines conflict does not tell you which architecture is correct. Git can reveal collision; only the shared design and human owners can resolve meaning.

86. Migration Lane Protocol

Database migrations create ordering and merge risks, so one person - normally Member B - should act as migration coordinator. Other workstreams may request schema changes, but simultaneous independent migration histories should be minimized.

SCHEMA CHANGE REQUEST

Workstream: WS-05

Needed change: add request.status

Reason: approved request lifecycle requires durable state

Compatibility impact: existing rows need default PENDING

Requested by: C

Decision owner: A+B

Migration owner: B

- Before generating a migration, sync latest migration head.
- Avoid two unrelated branches creating migrations from the same old head when possible.
- If parallel migrations are unavoidable, explicitly reconcile/merge heads before release.
- Test both fresh database setup and upgrade path when feasible.
- Never manually edit production-like data as a substitute for a missing migration without documenting the exception.

87. When to Merge Partial Work

Partial work may merge when it is safe, non-misleading, and useful to downstream work. A stable backend endpoint can merge before frontend completion. A reusable frontend shell can merge before one API exists. A temporary fake-success path should generally not merge into main unless clearly gated and impossible to confuse with real behavior.

Partial item	Usually safe to merge?	Condition
Verified backend endpoint	Yes	Contract approved; tests pass
Frontend component shell	Often	Does not pretend integration is complete
Temporary mock data path	Usually no	Unless feature-gated/test-only and explicit
Migration needed by other workstreams	Yes, carefully	Migration owner + compatibility verified
Half-refactor across shared modules	No	Finish or revert; high merge risk

PART XV - QA, DEBUGGING, REVIEW, EVIDENCE, AND INCIDENT-RESPONSE RUNBOOK

Quality engineering in this workflow is continuous confidence-building. The team does not ask only "did the code run?" It asks whether the approved behavior is correct, reproducible, integrated, persistent, and still correct after later changes.

88. QA Test-Design Ladder

Layer	Question	Example
Requirement	What claim must be true?	A paid request cannot be paid twice
Equivalence	Which input classes behave similarly?	valid positive amount / invalid non-positive amount
Boundary	Where does behavior change?	balance exactly equal to amount
State	Which transitions are legal?	PENDING -> PAID allowed; PAID -> PAID rejected
Persistence	Does restart/reload preserve truth?	paid status remains after refresh
Integration	Do layers agree?	frontend field names match backend schema
Concurrency/retry	What if actions overlap/repeat?	double click / network retry
Regression	What old behavior can new change break?	new auth blocks existing transfer

89. Debugging Escalation Ladder

69. Reproduce with exact steps and known starting state.
70. Classify the failing layer: UI, Network/contract, backend validation, service/business logic, DB/persistence, environment/config.
71. Collect evidence before changing code.
72. Compare actual behavior against problem.md/plan.md, not against what one developer remembers.
73. Use the smallest owner capable of fixing it.
74. If local fix preserves design, fix/test locally.
75. If fix requires shared contract/schema/invariant change, stop and run decision protocol.
76. Rerun focused test plus one nearby regression.
77. Record evidence and close only when reproduction no longer fails.

DEBUG TRIAGE

Symptom: <visible failure>

Reproduction: <steps>

Expected: <contract/requirement>

Actual: <response/log/db/UI>

Likely layer: frontend | contract | backend | logic | DB | env

Owner: <member>

Shared-design change required: yes/no

Regression set after fix: <tests>

90. Browser-to-Database Trace

For a full-stack defect, trace the request without skipping layers: user event -> frontend state -> event handler -> API client -> browser Network -> HTTP route -> request validation -> service/use case -> business rule -> ORM/session -> database -> commit -> response serialization -> Network response -> client state -> rerender -> visible result. The first layer where actual behavior diverges from expected behavior is usually the best starting point.

91. Reviewer Trigger Matrix

Change/risk	Independent Reviewer?	Reason
Simple text/UI polish	Usually no	Low correctness consequence
New critical API contract	Often yes	Cross-team boundary
Money/resource invariant	Yes	High consequence if wrong
Concurrency/idempotency	Yes	Difficult failure modes
Migration affecting core entities	Often yes	Durable state risk
Small design-preserving bug fix	Maybe not	Owner tests may be sufficient
Pre-release whole project	Yes	Independent final confidence

92. Evidence Standard

Evidence should answer "why do we believe this is complete?" It can include automated test output, API response, browser Network evidence, DB row/state evidence, migration success, E2E steps, or Reviewer verdict. Evidence is lightweight but concrete.

Claim	Weak evidence	Stronger hackathon evidence
Endpoint works	"I tested it"	pytest output + example response
Persistence works	UI looked right	DB state + refresh/reload result
Contract matches	frontend compiles	real Network request/response matches plan.md
Concurrency handled	code has a lock	concurrent/retry test + invariant result
Deployment works	build succeeded	deployed Golden-Path E2E

93. P0/P1/P2 Operational Meaning

Severity	Meaning	Team action
P0	Blocks core Golden Path, corrupts critical state, security/integrity catastrophe, cannot submit/demo reliably	Stop optional work; swarm immediately
P1	Important incorrect behavior or major reliability/usability defect that should be fixed before release	Schedule before freeze/release; may interrupt current task
P2	Non-blocking issue, polish, minor docs drift, low-risk improvement	Fix only if time remains; document otherwise

PART XVI - DETAILED 3-MEMBER OPERATING MODEL AND REDUCED-CAPACITY STRATEGY

The three-member model is not the four-member process with QA deleted. It is a deliberate redistribution of ownership with more selective parallelism and stronger use of fresh Reviewer agents for independence.

94. Three-Member Responsibility Split

Responsibility	Member A - Lead/Integration/Backend	Member B - Backend/Data/Logic	Member C - Frontend/QA/E2E
Challenge/problem.md	Primary	Contribute invariants	Contribute user/testability
plan.md	Primary	Data/backend/contracts	UI/contracts/test design
Migrations	Backup	Primary	Request changes
Core backend	Shared critical/cross-cutting	Primary	API consumer/testing
Frontend	Integration support	Support when free	Primary
QA design	Risk prioritization	Backend tests	Primary manual/integration/E2E
Reviewer	Trigger/orchestrate	May request fresh reviewer	May run fresh reviewer
Merges/main	Primary	Prepare merge-ready branches	Prepare merge-ready branches

95. Three-Member Parallelism Rules

- Do not run three unrelated feature tracks if one critical dependency is unstable.
- Member A should keep at least 30-40% attention on system integration rather than disappearing into backend implementation.
- Member C begins QA design before frontend completion, but uses fresh Reviewer agents for high-risk backend logic to preserve independence.
- Member B should absorb backend support and testing work before starting optional server features.
- Cut optional features earlier than the four-member model because verification capacity is lower.

96. Three-Member 24-Hour Adjustment

Window	Primary focus	Capacity rule
0-3h	Same shared intake/design	All three collaborate; no premature split
3-7h	Foundation + first Golden-Path slice	A integration/backend, B data/core, C UI + QA matrix
7-13h	Critical capabilities	Only 2-3 active critical workstreams; merge frequently
13-16h	Golden Path completion	Defer optional scope aggressively
16-20h	Integration + regression + local E2E	C QA-heavy; A integration; B correctness/fixes
20-24h	Freeze/release/review/demo	All converge; no isolated feature ownership

97. Preserving Review Independence with Three People

When Member C helped implement a frontend capability, they should not be the only final authority on that same capability. For high-risk work, use a fresh Reviewer agent with repository context, then have another human confirm findings. Independence is about a fresh perspective and evidence, not a fourth job title.

98. Lead Overload Protection

- Member A owns decisions, not every implementation detail.
- Member B may directly approve Level-1 backend implementation choices.
- Member C may directly approve Level-1 frontend/test choices.
- Use structured decision requests instead of interrupting A with every question.
- If A becomes the critical coding bottleneck, reassign coding and return A to integration oversight.

PART XVII - EXPANDED AGENT PROMPT LIBRARY

These prompts are templates. Replace placeholders with the current workstream and repository evidence. The important property is role clarity: an agent is either supervising, building, reviewing, testing, integrating, or investigating - not doing everything without a boundary.

99. Team-Level Supervisor Startup Prompt

ROLE

You are the shared Control/Supervisor for a 24-hour multi-member hackathon team.

AUTHORITY

Official rules/clarifications > official challenge > approved problem.md > approved plan.md > actual code/tests/migrations/runtime evidence > execute.md/review.md/README.

TEAM MODEL

Four members by default: Lead/Integrator, Backend/Data, Frontend/Client, QA/Integration.

YOUR JOB

- reconstruct the challenge before implementation;
- help the humans create one problem.md, MVP, Golden Path, and one Master System Design;
- detect requirement ambiguity and architecture fragmentation;
- create dependency-aware capability workstreams;
- keep API/data contracts explicit;
- prepare bounded Builder/Reviewer prompts;
- supervise cross-workstream decisions and scope;
- prioritize Golden Path movement;
- supervise integration, E2E, release, and judge readiness.

DO NOT

Become the default repository Builder. Do not invent separate plans for members. Do not approve silent contract/schema/business-rule changes.

100. Backend Builder Workstream Prompt

ROLE: Backend Builder for WS-XX.

Read AGENTS.md, problem.md, plan.md, execute.md, relevant migrations/tests.

Reconstruct current backend/data state before proposing code.

OBJECTIVE: <capability>.

APPROVED CONTRACT: <method/path/request/response/errors>.

INVARIANTS: <list>.

DEPENDENCIES: <list>.

OUT OF SCOPE: frontend ownership, unrelated refactors, unapproved schema/contract changes.

FIRST RETURN A PLAN:

- files;
- data/migration impact;

- service/transaction boundary;
 - tests including negative/concurrency cases if relevant;
 - integration implications;
 - merge-conflict risk.
- STOP BEFORE IMPLEMENTATION.

AFTER APPROVAL: implement, test, debug, record evidence. Stop and escalate material shared-design changes.

101. Frontend Builder Workstream Prompt

ROLE: Frontend Builder for WS-XX.

Read problem.md, plan.md, execute.md, approved API contract, current API client and UI patterns.

OBJECTIVE: <user-facing capability>.

CONTRACT: <method/path/request/response/errors>.

BACKEND STATUS: design-stable only | implementation-stable.

PLAN MUST COVER:

- components/pages;
- local/server state;
- loading/success/error states;
- API-client changes;
- temporary mock strategy if needed;
- exact mock removal/rendezvous step;
- frontend verification;
- integration risks.

STOP BEFORE IMPLEMENTATION.

Do not invent response fields or silently compensate for backend contract drift.

102. QA Test-Design Prompt

ROLE: QA/Test Designer for WS-XX.

Use problem.md and plan.md as expected behavior.

Return a risk-prioritized test matrix covering happy path, boundaries, negative inputs, state transitions, persistence, integration, concurrency/retry where relevant, and regression impact.

Separate tests that can run before integration from tests requiring the full slice.

Flag ambiguous requirements/contracts as NEED DECISION instead of guessing.

103. Contract-Drift Reviewer Prompt

Compare the approved contract in plan.md with actual backend schemas/routes/responses and frontend API-client/request usage for WS-XX.

Identify every mismatch.

Classify each as implementation drift, approved evolution, or ambiguous.

Do not change code. Return evidence, impact, owner, and whether a shared decision is required.

104. Integration Rendezvous Prompt

ROLE: Integration Supervisor for WS-XX.

Trace the real user action from frontend event through API client, browser HTTP, backend route/validation/service, persistence, response, and rerender.

The goal is to replace assumptions/mocks with real behavior.

Check contract adherence, configuration/CORS, error mapping, persistence, stale client state, and reload behavior.

Return: first divergence, reproduction, owner, smallest design-preserving fix, and regression checks. Do not add unrelated features.

105. Blocker Investigation Prompt

ROLE: bounded investigator, not automatic fixer.

BLOCKER: <symptom>.

WORKSTREAM: WS-XX.

EXPECTED: <requirement/contract>.

REPRODUCTION: <steps>.

Read relevant code/tests/log evidence.

Return hypotheses ranked by evidence, likely failing layer, minimal experiments to distinguish them, and whether any fix would change shared design. Stop before broad refactor.

106. Final Whole-Repository Reviewer Prompt

Review the integrated main branch as an independent release Reviewer. Prioritize Golden Path correctness, business invariants, data integrity, API contract consistency, critical negative paths, migration/runtime correctness, deployment if used, regression evidence, and misleading documentation.

Classify findings P0/P1/P2 and return PASS / PASS WITH NON-BLOCKING FINDINGS / BLOCKED. Avoid style-only suggestions unless they affect correctness, maintainability, or judge explanation.

PART XVIII - COPY-READY REPOSITORY ARTIFACT TEMPLATES

These templates make the workflow executable under time pressure. They are intentionally lightweight: enough structure to synchronize humans and agents without turning the repository into documentation work.

107. problem.md Template

```
# Problem
## Plain-English Product
## Users / Actors
## Required User Workflows
## Inputs and Outputs
## Frontend Requirements
## Backend Requirements
## Data / Persistence Requirements
## Business Rules and Invariants
## Constraints
## Explicit Requirements
## Logically Inferred Requirements
## Assumptions / Clarifications Needed
## Optional Ideas (Not MVP Commitments)
## Acceptance Criteria
## MVP Boundary
```

108. plan.md Template

```
# Master System Design
## MVP and Golden Path
## Architecture Overview
## Frontend Structure
## Backend Structure
## Entities / Relationships
## Business Invariants
## API Contracts
### <METHOD /path>
Request:
Response:
Errors:
Consumer:
## Business / Decision Logic
## Workstream Map
## Dependency Graph
## Ownership
## Integration Strategy
## Verification Strategy
## Release / Deployment Strategy
## Major Decisions
```

109. execute.md Workstream Template

```
## WS-XX - <Capability>
Golden Path Relevance: CRITICAL | SUPPORTING | OPTIONAL
Primary Owner:
Frontend Owner:
Verification Owner:
Integration Owner:
Dependencies:
```

Approved Contracts:

Status: PLANNED | READY | IN PROGRESS | BLOCKED | VERIFYING | COMPLETE

Backend:

- []

Frontend:

- []

Persistence:

- []

Integration:

- []

Verification:

- []

Evidence:

Blocker:

Need Decision:

Next:

Exit Criteria:

110. review.md Finding Template

Finding R-XX

Workstream / Scope:

Category: BUG | DESIGN ISSUE | CONTRACT DRIFT | INTEGRATION FAILURE | MISSING VERIFICATION | DOC DRIFT
| DEFERRED | IMPROVEMENT

Severity: P0 | P1 | P2

Evidence:

Expected:

Actual:

Impact:

Owner:

Resolution / Deferral:

Verification after fix:

Status:

111. API Contract Template

<Capability>

Method + Path:

Authentication:

Request Headers:

Path Params:

Query Params:

Request Body:

Success Status:

Success Response:

Validation Errors:

Business Conflict Errors:

Not Found Behavior:

Idempotency / Retry Notes:

Frontend Consumer:

Persistence Effect:

Contract Decision ID:

112. Lightweight Decision Record

D-XX - <short decision>

Time:

Context:

Options considered:

Decision:

Reason:

Affected workstreams:

Contract/schema/doc updates required:

Owner responsible for propagation:

PART XIX - FAILURE-MODE AND RECOVERY PLAYBOOKS

The workflow should tell the team what to do when reality deviates from the plan. Recovery quality is often more important than perfect planning because a hackathon guarantees surprises.

113. Backend Is Two Hours Late

- Do not let frontend invent a second contract.
- Frontend completes contract-faithful UI states and moves to another READY task.
- QA prepares tests and can test any already-live backend subparts.
- Lead asks whether the backend scope can be split into an earlier integration-safe subset.
- Swarm if the late endpoint is the current Golden-Path blocker.
- Cut optional backend behavior before cutting core correctness.

114. Frontend Is Two Hours Late

- Merge verified backend contracts so other work can proceed.
- Backend moves to next dependency-aware core capability.
- QA exercises APIs directly and builds regression evidence.
- Lead prevents backend from overbuilding features the UI cannot expose.
- If Golden Path is blocked by UI, swarm basic integration before polish.

115. Contract Was Wrong

78. Identify the approved requirement that makes the current contract insufficient.
79. Create NEED CONTRACT DECISION with impact.
80. Lead + affected owners approve the smallest coherent change.
81. Update plan.md/decision record first or in the same coordinated change.
82. Backend and frontend update against the new contract.
83. QA adds regression around the changed boundary.
84. Do not preserve backward compatibility unless it is genuinely needed; unnecessary dual contracts waste hackathon time.

116. main Is Broken After Merge

85. Stop additional merges.
86. Identify the last known good commit and the merge that introduced failure.
87. Reproduce on main.
88. Decide revert vs forward fix based on time/risk.
89. Restore runnable main before continuing unrelated merges.
90. Rerun Golden-Path smoke test.

91. Record the failure if it reveals a missing merge gate.

117. Two Builders Changed the Same Shared File

First decide whether both changes are legitimately required. If yes, appoint one human owner to reconcile on a branch with both contexts. If one change violates ownership or shared design, discard/rework it. Do not ask an agent to "merge both" without explaining semantic intent.

118. Database Migration Conflict

- Pause deployment/main migration until heads are understood.
- Have migration owner inspect latest revision graph.
- Rebase/regenerate or create a proper merge migration according to the migration tool.
- Verify fresh setup and upgrade path.
- Notify dependent branches of the canonical migration head.

119. QA Finds P0 Near Feature Freeze

Freeze all optional work immediately. Lead owns triage. Assign one fix owner, one reproducer/verifier, and one integration observer. If the fix requires architecture redesign, prefer the smallest safe behavior-preserving workaround that satisfies the core requirement, provided it remains explainable and does not falsify project claims.

120. Deployment Fails Late

- Do not destroy a working local runtime while chasing hosting.
- Determine whether deployment is mandatory.
- If optional and local demo is acceptable, preserve local release as fallback.
- If mandatory, reduce deployment variables: one known DB, explicit env, migrations, start command, CORS, frontend API URL.
- Run deployed Golden-Path E2E; a green build alone is not release proof.

121. One Member Becomes Unavailable

Cross-understanding and shared artifacts are contingency controls. Lead checks execute.md, branch, latest handoff packet, and workstream evidence. Another member reconstructs the bounded scope from repository truth rather than from memory. Critical work is reassigned; optional owned work may be deferred.

122. Team Is Behind at Hour 14

Keep	Reduce/Defer
Golden Path critical capabilities	Optional dashboards/analytics
Core invariants and persistence	Aesthetic polish beyond demo clarity
Real frontend/backend integration	Nice-to-have secondary workflows
Critical negative path	Large refactors
Regression on working core	Extra technology demonstrations

The rule is to reduce breadth before reducing confidence in the core.

PART XX - FINAL FOUR-HOUR WAR ROOM, DEMO, SUBMISSION, AND JUDGE-DEFENSE PROTOCOL

The final four hours are not simply the end of coding. The operating mode changes from feature throughput to release confidence. Everyone should know exactly which activities are allowed and which are too risky.

123. Hour 20-21: Feature Freeze and Defect Queue

- Stop optional capability development.
- Snapshot current main and verify startup commands.
- Run risk-based regression on all critical Golden-Path steps.
- Sort findings P0/P1/P2.
- Assign P0/P1 owners and explicit verification owner.
- Do not mix unrelated cleanup into critical fixes.

124. Hour 21-22: Release / Deployment Decision

Question	If YES	If NO
Deployment mandatory?	Deploy immediately using known plan	Evaluate value vs risk
Local E2E passing?	Proceed	Do not deploy yet; fix local system
Production config rehearsed?	Proceed cautiously	Deployment risk high
Enough buffer for deployed E2E?	Proceed	Prefer stable local release if allowed

Whichever runtime is chosen becomes the release candidate. From this point, all fixes must be justified against release confidence.

125. Hour 22-23: Final Independent Review and Documentation Reconciliation

92. Run final Reviewer against integrated main/release candidate.
93. Fix P0 and justified P1 findings.
94. Update README to actual features and actual run commands.
95. Reconcile execute.md and review.md; remove false COMPLETE claims.
96. Verify migrations, env template, seed/demo data, and startup instructions.
97. Tag or record the release commit.

126. Hour 23-23:30: Whole-Project Reconstruction

Each member explains the project from their role, while the lead checks that explanations agree. The point is not memorization; it is discovering understanding gaps while there is still time to inspect the repository.

Member	Must be able to explain
A	problem, MVP, Golden Path, architecture, workstream decisions, tradeoffs, integration, release
B	entities, transactions, business logic, invariants, migrations, concurrency/retry, API behavior
C	user journey, component/state flow, API client, loading/error handling, real integration
D	test strategy, highest risks, key failures found, regression, E2E evidence, release verdict

127. Hour 23:30-24: Demo Freeze Protocol

- No new features.
- No dependency upgrades.

- No broad refactors.
- No schema changes unless submission-blocking.
- Keep the demo dataset/known state reproducible.
- Run the exact demo once from clean start.
- Confirm source pushed and correct branch/commit submitted.
- Keep local fallback even if deployed demo is primary.
- Prepare one sentence for each known limitation rather than hiding it.

128. Demo Story Structure

98. Problem: one concise sentence about the real pain.
99. User: who experiences it.
100. Golden Path: perform the central workflow live.
101. Technical value: explain one or two non-trivial engineering choices.
102. Quality proof: mention how correctness/integration was verified.
103. Tradeoff: explain one deliberate scope choice.
104. Outcome: show persistent/visible final state.

129. Judge Question Bank

Question	Strong answer direction
Why this architecture?	Tie the answer to hackathon constraints, domain invariants, simplicity, and future extension - not fashion.
How do frontend and backend coordinate?	Explain design-stable contracts, independent implementation, rendezvous, and real integration.
How do you prevent conflicting balances/state?	Explain the specific invariant and actual transaction/concurrency mechanism used in this project.
What did AI agents do?	Explain bounded Builder/Reviewer roles and human approval of important decisions.
How do you know it works?	Point to focused tests, QA findings, integrated Golden-Path E2E, persistence/reload, and release evidence.
What would you build next?	Choose the highest-value deferred capability or hardening step, not a random technology.
What failed during development?	Describe one real integration/contract/test failure and how evidence led to the fix.

PART XXI - PRINTABLE AT-THE-TABLE CHEAT SHEETS

These pages are intentionally compressed. They are designed to remain visible during the event while the rest of the handbook serves as the deeper reference.

130. One-Page Team Operating Loop

CHALLENGE -> problem.md -> MVP -> GOLDEN PATH -> plan.md -> execute.md

FOR EACH READY WORKSTREAM:

assign owner -> sync main -> branch -> Builder plan -> human approve -> implement/test -> backend/frontend rendezvous if needed -> QA -> Reviewer if risk justifies -> merge -> evidence -> close

ALWAYS ASK:

1. What blocks the Golden Path?
2. Did any contract/schema/invariant change?
3. Is main runnable?
4. What is the next READY dependency?
5. Are we adding optional scope while core is blocked?

131. Member Quick Cards

Member	Always watching	Immediate escalation trigger
A Lead	Golden Path, dependencies, contracts, main, clock	architecture/API/schema/MVP/release change
B Backend	invariants, transactions, data, endpoint evidence	shared schema/contract/business-rule change
C Frontend	user flow, UI states, API client, mocks	contract insufficient/drift, backend dependency blocked
D QA	highest-risk claims, regression, integration evidence	P0/P1, ambiguous expected behavior

132. Decision Quick Guide

Question	Decision level
Can I rename a private helper?	Level 1 - local
Can I change endpoint response field?	Level 2 - shared
Can I add/change DB relationship used by other workstreams?	Level 2 - shared
Can I replace core architecture or database?	Level 3 - product
Can I cut an optional feature?	Lead/team scope decision; usually safe if Golden Path unaffected
Can I change Golden Path?	Level 3 - product

133. Integration Quick Guide

BEFORE RENDEZVOUS

- ☐ contract design-stable
- ☐ backend endpoint testable
- ☐ frontend consumer ready
- ☐ mock identified

RENDEZVOUS

- ☐ sync latest code
- ☐ remove/bypass mock
- ☐ real browser request
- ☐ compare Network with plan.md
- ☐ inspect backend/log/DB
- ☐ verify visible result

- ☐ verify error path
- ☐ QA independent check
- ☐ evidence + merge

134. Merge Quick Guide

BEFORE MERGE

- ☐ scope clean
- ☐ tests pass
- ☐ no silent contract/schema drift
- ☐ migrations coherent
- ☐ required integration checked
- ☐ no secrets/junk
- ☐ feature branch updated with main
- ☐ conflicts resolved and retested

AFTER MERGE

- ☐ pull/run main
- ☐ smoke Golden Path if affected
- ☐ update execute.md/evidence

135. QA Quick Guide

FOR EVERY CRITICAL CAPABILITY ASK:

- Happy path?
- Boundary?
- Negative/invalid?
- State transition?
- Persistence/reload?
- Frontend-backend contract?
- Retry/concurrency if relevant?
- Regression risk?
- Evidence?
- Verdict?

136. Hour-by-Hour Phase Card

Hours	Mode	Do not do
0-3	Understand + design + assign	blind feature coding
3-6	Foundation + first contact	isolated long-lived branches
6-13	Core slices + high-risk logic	silent contract changes
13-16	Complete Golden Path	open large optional features
16-20	Integrate + harden + E2E	treat layer tests as enough
20-22.5	Freeze + release/deploy	large refactors
22.5-24	Final review + docs + demo	risky last-minute changes

137. Final Submission Card

- Correct repository/branch/commit submitted.
- README run instructions tested from a clean terminal.
- Required credentials/demo data handled safely.
- Local fallback preserved.
- Golden Path rehearsed.
- Known limitations stated accurately.
- Every member can explain the project and their own work.
- No last-minute unpushed change exists only on one laptop.

138. Closing Principle

A strong 24-hour team does not win by maximizing simultaneous typing. It wins by converting uncertainty into shared decisions, shared decisions into bounded work, bounded work into integrated capabilities, and integrated capabilities into a verified Golden Path. Agents increase implementation throughput; human governance keeps that throughput pointed at one product.